

RoboPainting: Indirect Force Control for Stable Brush–Wall Contact

Ruohan He, Shixuan Sun, Jiawen Wang, and Xiaoyang Zhao

Team 13, MAE C163C/C263C

Department of Mechanical and Aerospace Engineering

University of California, Los Angeles

github.com/chiawenwang/robot_painting

Abstract—This report presents RoboPainting, a contact-rich robotic painting system designed to maintain stable brush–wall contact while following prescribed letter trajectories. The project focuses on feedback controller design for robotic contact rather than mechanical design alone. In simulation, a MuJoCo model of a six-degree-of-freedom Piper manipulator regulates brush normal force through an outer-loop admittance controller that maps force error into a wall-normal position command, which is then converted to joint references through damped inverse kinematics. The simulated controller maintained contact throughout a 6.00 mm wall-retreat event and achieved a mean pen-down tangential tracking error of 0.497 mm. On the real Piper platform, a joint-level MIT-mode impedance controller is used for safer compliant contact. The hardware task was implemented as reverse painting: the wall was colored first, and an eraser mounted at the end effector removed color along the commanded path to write “UCLA.” The recommended hardware stiffness scale is $s = 0.3$, because $s = 0.1$ was too soft for reliable letter formation, whereas $s \geq 0.5$ made the arm too stiff for safe contact. The final recommended architecture uses simulation to tune force-regulation behavior and hardware joint impedance to preserve physical compliance during real-world contact.

Keywords—robotic painting, indirect impedance control, contact-force regulation.

I. INTRODUCTION AND LITERATURE REVIEW

Robotic painting is a representative contact-rich manipulation problem because successful execution requires both geometric path tracking and stable physical interaction. The robot must move the tool tangentially to reproduce a desired pattern while regulating normal interaction so that contact remains continuous without excessive compression. Similar requirements occur in polishing, sanding, grinding, wiping, and other surface-treatment operations. Pure position control is vulnerable to surface uncertainty: inward surface motion can create excessive contact force, whereas outward motion can produce separation and incomplete strokes.

Contact-rich manipulation broadly requires robots to reason about both motion and contact forces. Suomalainen, Karayiannidis, and Kyrki survey robotic manipulation in contact and show that reliable task execution often depends on controlling the interaction with the environment, not only the free-space trajectory [1]. Their survey motivates this project by placing painting in a broader class of tasks where the environment is part of the control loop. Unlike many manipulation examples in that survey, this project studies a visually simple but physically challenging surface-contact



Fig. 1: RoboPainting hardware demonstration. The mobile platform carries a Piper manipulator that maintains contact with a vertical writing surface while an eraser-style end effector traces the UCLA letters. This teaser summarizes the central goal of the project: stable wall-normal contact together with accurate tangential path tracking.

task: writing letters on a wall while maintaining stable brush or eraser contact.

Bansal *et al.* study robotic painting with human-like applicator behavior and show that painting quality is strongly influenced by the interaction between the applicator and the surface [2]. Their work is similar in application, because both projects use robotic actuation to reproduce painting motions. However, their emphasis is on coating outcomes and human-applicator imitation, whereas this project emphasizes feedback-controller design and gain selection for brush–wall contact.

Hogan’s impedance-control framework established the principle of regulating the dynamic relationship between motion and interaction force rather than commanding position or force independently [3]. This idea directly supports the simulation design: measured force error is passed through a virtual mass–damper–spring model before it becomes a wall-normal motion command. Abu-Dakka and Saveriano review variable impedance control and learning approaches for robots that must adapt compliance in changing environments [4]. The hardware implementation in this project is simpler

TABLE I: Controller design requirements and performance metrics.

Design objective	Requirement	Performance metric
Normal force regulation	Desired force = 10 N	Mean force, peak force, and force RMSE during pen-down motion
Contact maintenance	No separation during the 6 mm retreat	Percentage of retreat samples with nonpositive contact gap
Wall following	Brush responds to wall retreat	Brush-front displacement and displacement-following ratio
Tangential path accuracy	Mean < 1 mm; maximum < 2 mm	Euclidean y - z tracking error
Orientation stability	Brush-axis drift < 1°	Maximum tool-axis angular deviation
Transient stability	No sustained oscillation	Force time history and settling behavior

than the learning-based methods in that review, but it shares the same motivation: the robot should be compliant enough to tolerate uncertain contact.

Classic force-control literature further motivates the distinction between direct force control and indirect force/compliance control. Whitney’s historical review identifies stability as a central issue in robot force control [5], and Salisbury’s Cartesian stiffness control demonstrates that apparent end-effector stiffness can be shaped for contact tasks [6]. The present project applies these ideas pragmatically: the simulation uses a wall-normal admittance model, while the real robot uses low joint stiffness in the MIT interface because high-stiffness position control was not appropriate for safe contact.

II. OBJECTIVES AND CONTROLLER DESIGN REQUIREMENTS

The study objective was to determine whether a position-actuated manipulator could maintain contact with a moving surface while preserving a prescribed painting path. The wall was commanded to move forward and backward in the x direction to represent local surface-height variation. The controller was required to regulate a nominal normal force of 10 N, maintain contact throughout a 6 mm wall-retreat event, and avoid sustained force oscillation after contact transitions.

Geometric and orientation requirements were defined to ensure that normal compliance did not degrade the painted shape. During pen-down motion, the mean Euclidean y - z tracking error was required to remain below 1 mm and the maximum error below 2 mm. The brush-axis orientation error was required to remain below 1° . These numerical thresholds are engineering targets for this simulation rather than universal painting standards.

A later hardware study should revise these thresholds according to tool dimensions, surface material, sensor resolution, actuator bandwidth, and the process-quality requirements of the intended application. In the present hardware demonstration, no external force/torque sensor was used;

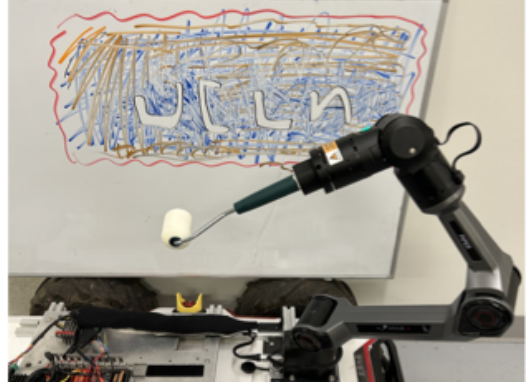


Fig. 2: Close-up of the Piper manipulator during reverse painting. The eraser-style tip is mounted to the end effector so that tangential motion writes the letters while compliant normal motion maintains contact with the wall.

therefore, hardware evaluation focuses on contact stability, safety, visible stroke quality, and the effect of joint-impedance stiffness.

III. ROBOT DESIGN AND SYSTEM OVERVIEW

The system consists of a six-degree-of-freedom AgileX Piper manipulator mounted on a mobile base and carrying a custom eraser-style end effector. The task is to write the word “UCLA” on a vertical whiteboard or wall while maintaining stable physical contact. Fig. 1 shows the complete hardware platform and final visual outcome, and Fig. 2 shows the end-effector contact configuration used during reverse painting.

The wall-normal direction is defined as the x axis, and the painting plane is the wall-tangential y - z plane. The controller intentionally separates these directions. Compliance is primarily introduced in the wall-normal x direction, while the letter trajectory is specified in the y - z plane. This separation is useful because painting quality depends on both letter geometry and stable tool pressure.

In hardware, the final demonstration used reverse painting rather than depositing paint. The wall was first covered with color, and the eraser attached to the end effector removed the color along the commanded path. This produced a negative-space “painting” effect while preserving the same contact-rich control problem as brush painting: the robot still had to maintain controlled normal interaction while tracking a tangential path.

IV. METHODS

A. Simulation Model and Task Definition

The robot was simulated in MuJoCo using a six-degree-of-freedom Piper model adapted from open-source AgileX Robotics model resources [7]. MuJoCo is designed for model-based control and contact-rich articulated simulation [8]. A virtual planar wall was represented as a moving body whose prescribed displacement was along the global x direction. The wall first moved toward the robot and then retreated, providing a repeatable approximation of a locally

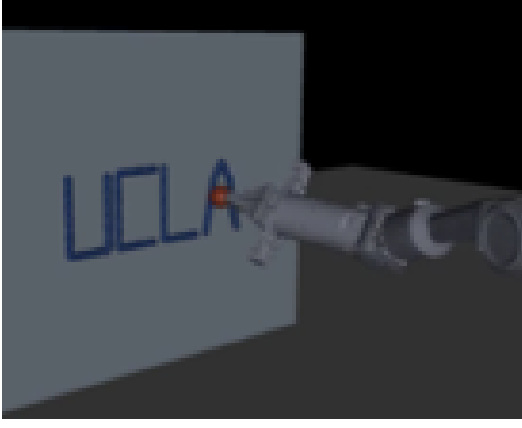


Fig. 3: MuJoCo visualization of the Piper manipulator writing the prescribed UCLA path on the moving wall.

uneven surface and enabling evaluation of both inward-disturbance force response and outward-disturbance contact maintenance.

The letters were represented as piecewise-linear trajectories in the wall-tangential y - z plane. Between disconnected strokes, a pen-up state reduced the desired force and moved the tool away from the wall to avoid unintended connecting lines. The brush front was represented by the end-effector position plus a constant tool-radius offset. Fig. 3 shows the Piper manipulator executing the prescribed UCLA trajectory in MuJoCo.

B. Directionally Decomposed Control Architecture

The controller separates the contact task into three coordinated objectives. The wall-normal x channel regulates contact through a compliant force-to-motion mapping. The y - z channel supplies the nominal writing trajectory, and the orientation channel keeps the tool axis aligned with the wall normal. The resulting Cartesian target is converted to six joint-position references through damped inverse kinematics.

Let F_{des} denote the desired normal force and F_{meas} the estimated force. The outer-loop compliant displacement $e = x_{\text{imp}} - x_{\text{ref}}$ is governed by the virtual mass-damping-stiffness dynamics

$$M\ddot{e} + B\dot{e} + Ke = F_{\text{des}} - F_{\text{meas}}, \quad (1)$$

where M , B , and K are virtual controller parameters rather than physical properties of the robot. The implemented values were

$$M = 1.6 \text{ kg}, \quad B = 145 \text{ N s m}^{-1}, \quad K = 560 \text{ N m}^{-1}. \quad (2)$$

At each time step, Eq. (1) is rearranged as

$$\ddot{e} = \frac{F_{\text{des}} - F_{\text{meas}} - B\dot{e} - Ke}{M}, \quad (3)$$

and numerically integrated to obtain virtual velocity and position. Velocity, displacement, and command-rate limits are applied after integration to improve safety and numerical stability. The x command is then combined with the

Algorithm 1 Simulation indirect force controller

- 1: Generate nominal letter position $\mathbf{x}_{\text{nom}}(t)$ in the wall plane.
 - 2: Estimate F_{meas} using Eq. (6).
 - 3: Integrate Eq. (1) to update the wall-normal compliant state.
 - 4: Rate-limit and clamp the normal command to obtain x_{cmd} .
 - 5: Set $\mathbf{p}_{\text{cmd}} = [x_{\text{cmd}}, y_{\text{traj}}, z_{\text{traj}}]^T$.
 - 6: Solve damped inverse kinematics and send q_{cmd} to the simulated robot.
-

prescribed tangential trajectory to form

$$\mathbf{p}_{\text{cmd}} = [x_{\text{cmd}}, y_{\text{traj}}, z_{\text{traj}}]^T. \quad (4)$$

A tool-axis orientation error is added to the Cartesian task, and the sixth joint is held near a reference value to suppress unnecessary tool roll. The damped pseudoinverse is used to reduce amplification of joint motion near unfavorable configurations:

$$J^\# = J^T (JJ^T + \lambda^2 I)^{-1}. \quad (5)$$

The resulting joint increment includes posture regularization so that the arm maintains a consistent configuration while preserving the primary Cartesian task. In the reported simulation, a quasi-static inner servo applied the inverse-kinematics solution nearly directly to the joint states. This assumption isolates the outer-loop contact behavior but does not reproduce the finite bandwidth of the physical Piper joint servos.

C. Contact-Force Estimation and Feedback

The simulation did not use a physical six-axis force/torque sensor. Instead, normal contact force was estimated from penetration of the tool front into the wall and relative closing velocity. The unilateral spring-damper model was

$$F_{\text{meas}} = \max(0, K_c \delta + B_c v_{\text{rel}}), \quad (6)$$

where δ is normal penetration, v_{rel} is relative normal velocity, $K_c = 3000 \text{ N m}^{-1}$ is contact stiffness, and $B_c = 60 \text{ N s m}^{-1}$ is contact damping. The maximum operator prevents negative tensile force and represents nonadhesive contact: the wall can push the tool but cannot pull it after separation.

The wall pose is known to the force estimator because it is needed to calculate penetration and relative velocity. However, the instantaneous wall position is not directly copied into the x -position reference. When the wall retreats, penetration and estimated force decrease first. The resulting positive force error then drives Eq. (1), causing the tool command to move toward the wall. Thus, wall following is feedback driven, although the feedback itself remains model dependent.

D. Hardware Controller: Joint-Level MIT Impedance

The first real-robot approach attempted to add a virtual spring on top of the robot's Cartesian position command interface. This was not sufficiently compliant because the on-board high-stiffness position servo treated the commanded–actual position gap as a tracking error to eliminate. To achieve safer physical interaction, the final implementation uses the Piper low-level MIT joint interface through the Piper SDK [7]:

$$\tau_i \approx K_{p,i} (q_{\text{ref},i} - q_i) + K_{d,i} (-\dot{q}_i), \quad (7)$$

where $q_{\text{ref},i}$ is obtained from Pinocchio inverse kinematics and each joint behaves approximately like a virtual spring–damper. Pinocchio is an open-source library for rigid-body kinematics and dynamics algorithms [9]. In the real controller, the gripper joints are locked in the reduced kinematic model, leaving six controllable joints.

The real-world controller uses the following nominal unscaled gains from the final script:

$$K_p = [10, 8, 8, 8, 6, 6], \quad (8)$$

$$K_d = [0.8, 0.6, 0.6, 0.6, 0.5, 0.5]. \quad (9)$$

The single stiffness scale s modifies both stiffness and damping:

$$K_p^{\text{eff}} = sK_p, \quad K_d^{\text{eff}} = \sqrt{s}K_d. \quad (10)$$

The recommended setting is $s = 0.3$, which gives

$$K_p^{\text{eff}} = [3.0, 2.4, 2.4, 2.4, 1.8, 1.8] \quad (11)$$

and

$$K_d^{\text{eff}} \approx [0.44, 0.33, 0.33, 0.33, 0.27, 0.27]. \quad (12)$$

The command rate is 200 Hz, the ramp time is 2 s, the letter speed is 0.02 m s^{-1} , and the maximum per-joint position error is 0.35 rad, approximately 20° . The error limit bounds the approximate proportional torque contribution:

$$|\tau_{p,i}| \leq K_{p,i}^{\text{eff}} e_{q,\text{max}}. \quad (13)$$

This bound is a safety-oriented design choice: increasing stiffness improves tracking but increases the risk of excessive contact force and end-effector damage.

E. Explicit Assumptions

The simulation and hardware controllers were designed under the following assumptions:

- 1) Robot kinematic parameters are exact, and the wall is planar and rigid except for its prescribed translational motion in simulation.
- 2) The tool-radius offset is constant; bristle deformation, eraser deformation, paint dynamics, and color-removal mechanics are not modeled.
- 3) The simulation contact stiffness and damping are constant, linear, and known to the force estimator.
- 4) Sensor noise, quantization, network latency, and computation delay are neglected in simulation.

TABLE II: Simulation contact-force metrics for the recommended gains.

Metric	Value	Interpretation
Task duration	38.99 s	Reduced from approximately 96 s
Desired pen-down force	10.00 N	Reference
Mean pen-down force	10.99 N	0.99 N positive bias
Force RMSE	1.45 N	Pen-down samples
Peak pen-down force	13.86 N	Initial contact transient

- 5) The quasi-static inner joint-position servo in simulation tracks inverse-kinematics commands without meaningful delay or error.
- 6) The real robot does not directly measure normal force; real-world contact regulation is therefore indirect and based on joint compliance.

V. RESULTS

A. Simulation Force Regulation

Figure 4 shows the response after the painting trajectory was conservatively accelerated. The complete task duration was reduced from approximately 96 s to 38.99 s. Over pen-down samples, the mean force was 10.99 N, the force-error RMSE was 1.45 N, and the peak force was 13.86 N. The largest peak occurred during initial contact establishment. The response remained strongly damped and did not exhibit sustained oscillation.

During intentional pen-up intervals, the desired force was set to zero and the estimated force correspondingly fell to zero. The approximately 0.99 N positive difference between the mean pen-down force and the 10 N target indicates a remaining steady force bias. This offset is consistent with the spring term in the virtual dynamics and the absence of integral compensation.

B. Wall Following and Contact Maintenance

As shown in Fig. 5, the prescribed wall motion included both forward and backward displacement to emulate a change in surface height. During the 6.00 mm retreat event, the brush front followed 4.972 mm, producing a displacement-following ratio of

$$\text{Following ratio} = \frac{\Delta x_{\text{brush}}}{\Delta x_{\text{wall}}} \times 100\% = 82.86\%. \quad (14)$$

The contact gap remained negative for 100 % of the retreat samples, satisfying the primary contact-maintenance requirement. A negative gap denotes finite tool compression rather than separation. Positive gap excursions outside the retreat interval correspond to intentional pen-up transitions and are not classified as controller failure.

During the retreat interval, the mean force was 12.05 N and the force RMSE relative to the 10 N target was 2.27 N. Continuous contact was therefore achieved at the cost of a temporary positive force bias. The result demonstrates that the outer compliant loop can respond to a retreating surface through force feedback, even though it does not reproduce the full wall displacement.

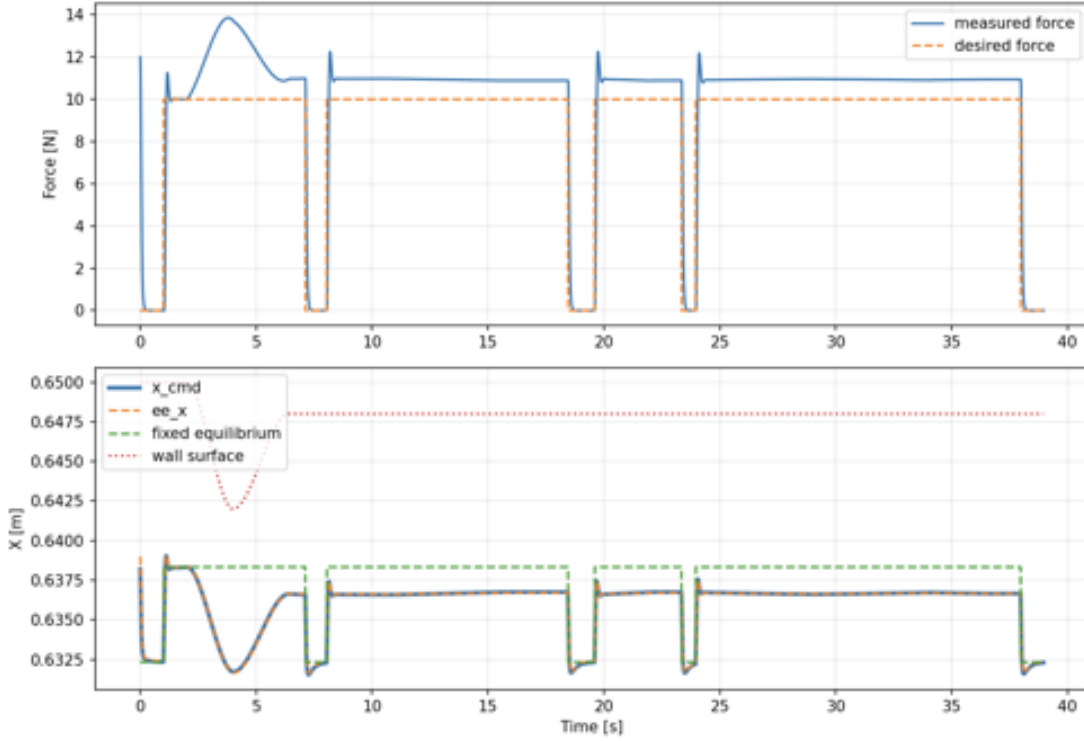


Fig. 4: Contact-force response and wall-normal controller states. The dashed orange line denotes the desired force, and the lower plot shows commanded, measured, equilibrium, and wall-surface positions.

TABLE III: Simulation path, contact, and orientation summary.

Metric	Measured	Requirement	Status
Mean pen-down e_{yz}	0.497 mm	< 1 mm	Pass
Maximum pen-down e_{yz}	1.013 mm	< 2 mm	Pass
Maximum brush-axis drift	0.062°	$< 1^\circ$	Pass
Contact during retreat	100 %	100 %	Pass
Wall-following ratio	82.86 %	Response metric	Reported

C. Tangential Path and Orientation Performance

Figure 6 shows that the nominal and actual UCLA trajectories remain closely aligned. For pen-down samples, the mean Euclidean y - z error was 0.497 mm and the maximum was 1.013 mm. Both values satisfy the specified requirements of less than 1 mm mean error and less than 2 mm maximum error. The tangential error is

$$e_{yz} = \sqrt{(y - y_{\text{des}})^2 + (z - z_{\text{des}})^2}. \quad (15)$$

The full time history contains transition peaks of approximately 2.4 mm. These peaks occur mainly during pen-up travel and abrupt changes between path segments; they are excluded from the reported pen-down maximum because they do not represent the accuracy of painted strokes. The maximum brush-axis drift was 0.062°, substantially below the 1° requirement.

D. Hardware Reverse-Painting Result

The hardware test executed the four letters “UCLA.” The tool did not deposit fresh paint; instead, the wall was colored in advance, and the end-effector eraser removed color along the commanded strokes. This reverse-painting method made the final path visible while preserving the same contact-control challenge: the robot had to maintain sufficient normal pressure for erasing without becoming overly stiff against the wall.

The final hardware run used the joint-level MIT impedance controller with a 200 Hz control rate, a 2 s ramp time, a 0.02 ms^{-1} letter speed, and a 0.35 rad joint-error limit. The stiffness scale was selected as $s = 0.3$. This value was a practical compromise: $s = 0.1$ was too soft and produced weak or distorted contact because the tool did not press consistently enough against the colored wall; $s = 0.3$ provided sufficient contact and legibility while remaining compliant; $s \geq 0.5$ made the arm too stiff, increasing the risk of surface damage, tool damage, and unsafe contact forces. Because no force/torque sensor was used in the hardware setup, these results are evaluated qualitatively through visible letter formation, contact stability, and safety.

VI. DISCUSSION

A. Effects of Controller Gains

The virtual mass, damping, and stiffness determine the trade-off among responsiveness, force stability, and geometric compliance. The mass M is a virtual controller parameter,

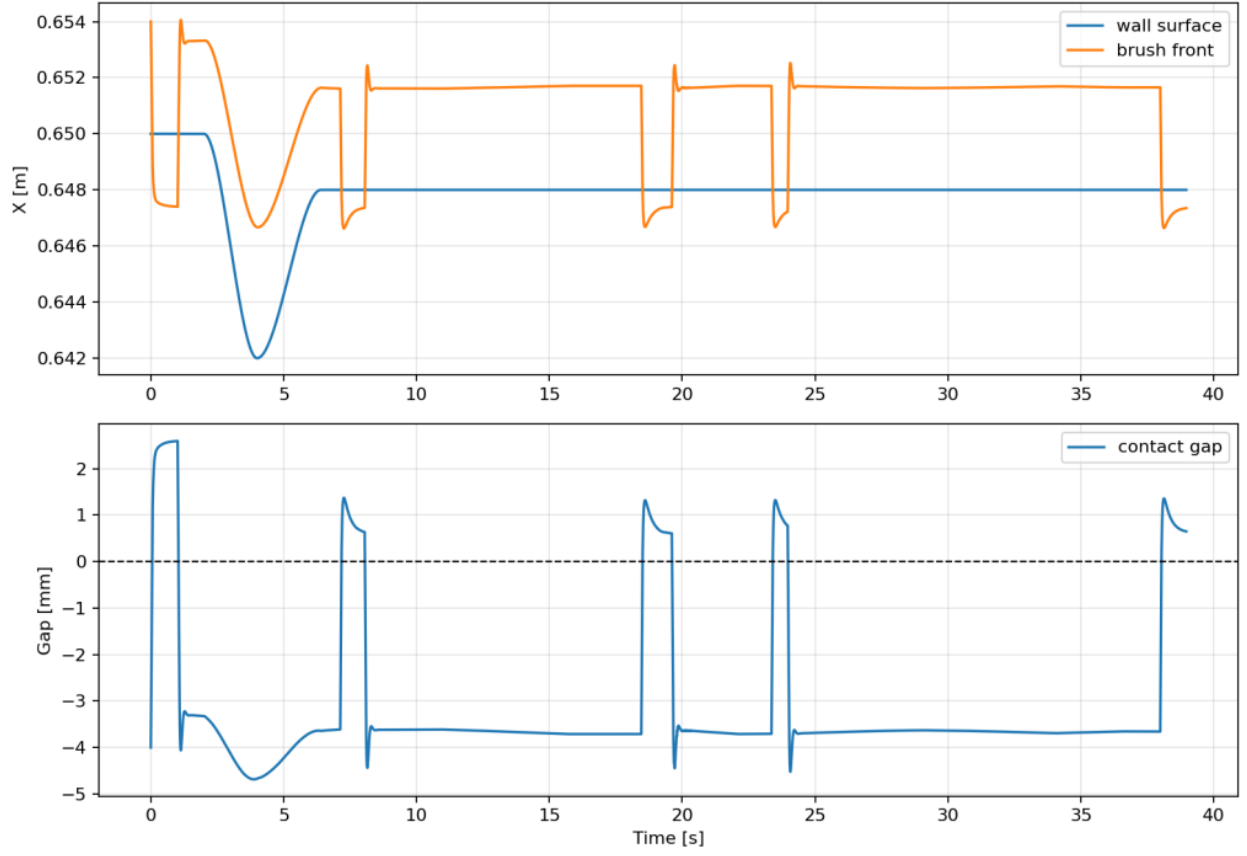


Fig. 5: Wall position, brush-front response, and contact gap. The zero-gap dashed line separates compression from geometric separation.

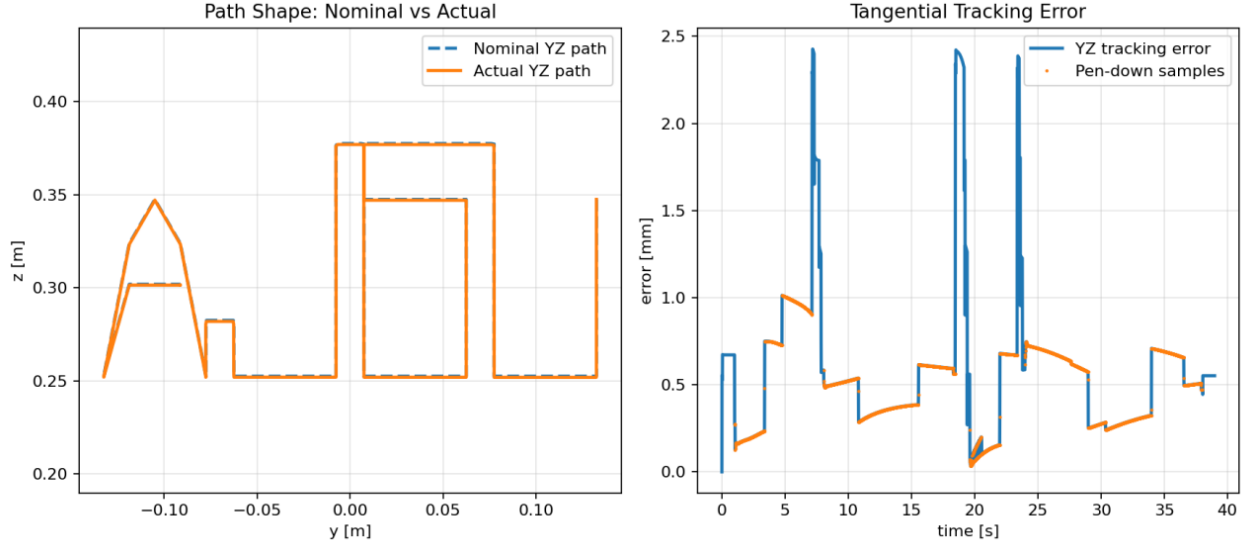


Fig. 6: Nominal and actual y - z paths on the wall plane and Euclidean tangential tracking error. Pen-down samples are shown as individual markers.

not the physical mass of the robot. Increasing M reduces the acceleration produced by a given force error, yielding smoother but slower x -direction correction. Decreasing M

improves responsiveness but increases sensitivity to discrete-time noise and transient oscillation.

Increasing damping B suppresses overshoot and ringing

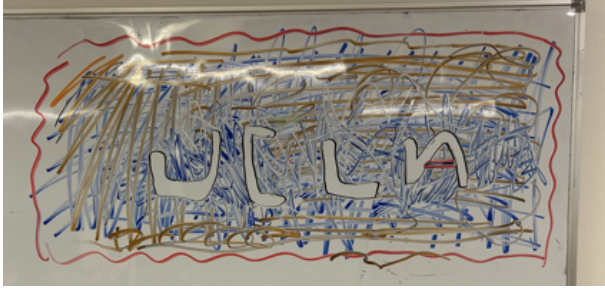


Fig. 7: Final hardware reverse-painting result. The colored whiteboard background was erased along the commanded path to form the four letters “UCLA.”

TABLE IV: Hardware controller settings and stiffness observations.

Quantity	Value or observation
Control rate	200 Hz
Ramp time	2 s
Letter speed	0.02 m s^{-1}
Joint error limit	$0.35 \text{ rad} \approx 20^\circ$
Letters tested	Full “UCLA”
Hardware process	Reverse painting by erasing pre-colored wall
Too soft	$s = 0.1$
Recommended	$s = 0.3$
Too stiff	$s \geq 0.5$

during contact establishment. Excessive damping, however, creates lag and reduces the tool’s ability to follow rapid wall retreat. Decreasing B improves response speed but can produce force peaks, repeated contact switching, and unstable-looking motion.

Increasing stiffness K pulls the compliant state more strongly toward the fixed equilibrium and reduces normal displacement. This can reduce drift, but it also weakens compliance and makes the response resemble rigid position control. Decreasing K improves wall-following ability but allows larger x excursions and can make force-error rejection slower.

The final simulation setting, $M = 1.6 \text{ kg}$, $B = 145 \text{ N s m}^{-1}$, and $K = 560 \text{ N m}^{-1}$, was intentionally highly damped. It prevented sustained oscillation, preserved contact during the complete retreat, and maintained submillimeter mean pen-down error. The remaining startup overshoot and steady force bias suggest that future improvement should focus on compliant-state initialization, bias compensation, and testing with nonideal actuator dynamics rather than simply increasing stiffness.

The hardware stiffness scale s was the most important real-world tuning parameter. With $s = 0.1$, the arm was too compliant to reliably remove color from the wall; the eraser could deflect away from the desired contact state and the letters became weak or inconsistent. With $s \geq 0.5$, the arm behaved more like a stiff position servo, which improved geometric tracking but made contact less forgiving and increased the risk of damaging the wall, eraser, or arm. The selected value $s = 0.3$ is therefore recommended for the

TABLE V: Observed gain effects.

Gain	Effect when increased	Main trade-off
Virtual mass M	Smoother acceleration	Slower wall following
Damping B	Less overshoot and oscillation	Greater lag
Stiffness K	Less normal drift	Lower compliance
Hardware scale s	Better letter tracking/pressure	Higher damage risk

current reverse-painting setup because it provides adequate contact pressure while preserving compliance.

B. Selection and Recommendation of Contact-Control Strategy

One major challenge was selecting an appropriate contact-control strategy for a position-actuated robot. Direct torque-level Cartesian impedance control would convert Cartesian motion error into force or torque commands, but the available simulation and hardware interfaces did not both support a reliable, identical low-level torque-control loop. Attempts to reproduce torque-like behavior through additional correction, blending, and filtering increased delay and degraded the written trajectory.

The recommended simulation controller is therefore an outer-loop admittance controller with an inner joint-position loop. The measured force error is passed through the virtual mass-damping-stiffness dynamics to generate an x -direction position correction, which is then combined with the y - z path and mapped to joint positions through damped inverse kinematics. This architecture produced readable letters, responsive wall following, and low oscillation for the available interface.

The recommended hardware controller is the joint-level MIT impedance controller with $s = 0.3$, 200 Hz command rate, 2 s ramp time, and 0.35 rad joint-error limit. It should be described as an indirect impedance implementation rather than direct force control, because hardware normal force is not directly measured. For a future robot with accurate external-wrench sensing and reliable torque control, a genuine torque-level Cartesian impedance or hybrid force/motion controller would be more appropriate.

C. Capabilities, Limitations, and Project Challenges

The primary capability of the controller is directional task separation. Normal compliance responds to wall motion without directly changing the geometry of the y - z letter path. The orientation objective prevents the tool from rotating away from the wall, and damped inverse kinematics reduces numerical instability near unfavorable configurations. The reported simulation satisfies all stated contact, path, and orientation requirements.

The most important development challenges were maintaining stable contact while preserving letter geometry, preventing unnecessary wrist motion over a long trajectory, and interpreting impedance and admittance terminology correctly for a position-controlled robot. The final task decomposition addressed shape distortion by confining compliance mainly to x while retaining explicit y - z and orientation objectives.

TABLE VI: Representative failure modes.

Failure condition	Cause	Likely outcome
Wall retreats too far or too quickly	Displacement, rate, or bandwidth limits exceeded	Positive gap and contact loss
Wall moves inward too quickly	Controller cannot retreat before compression grows	Large overshoot force
Damping too low	Insufficient energy dissipation	Oscillatory force and repeated contact switching
Stiffness too high	Compliance suppressed	Rigid behavior and poor retreat following
IK near singularity	Pseudoinverse amplification	Large or sudden joint commands
Real hardware s too low	Joint compliance too high	Weak erasing and distorted letters
Real hardware s too high	Joint compliance too low	Damage risk and unsafe contact

An additional hardware challenge was the absence of direct force sensing: the system could demonstrate compliant reverse painting, but it could not measure hardware force RMSE in the same way as simulation.

D. Response to Presentation Feedback

The presentation feedback form in Appendix identified four report-level questions that shaped the final discussion. First, the whiteboard was moved in the hardware demonstration because the reachable workspace of the Piper arm was limited for the desired letter size and contact setup from a fixed base pose. This was a practical laboratory workaround rather than a requirement of the control strategy. In a more integrated system, the mobile base could translate the manipulator relative to the wall, allowing the same wall-contact controller to cover a larger writing area without manually moving the whiteboard.

Second, the position plots use meters because the simulation state, wall motion, and virtual trajectory commands are all represented in SI units. The absolute wall and tool displacements are on the centimeter scale, while tracking and contact-gap metrics are reported in millimeters for readability. In the hardware demonstration, the virtual contact target was placed approximately 10 cm from the initial approach point to the final commanded point, with the physical wall generally located between those two positions. This behind-surface virtual target allowed the joint-space impedance controller to generate a sustained normal contact tendency without commanding a rigid penetration of the wall.

Third, the 10 N simulated normal-force target was intentionally conservative for maintaining visible contact and was also motivated by possible rolling-paint applications. However, excessive normal pressure increases tangential friction between the tool and the surface, which can degrade letter tracking, increase actuator effort, and risk damaging the wall or end effector. For the eraser-based reverse-painting hardware task, lower force or adaptive force/stiffness scheduling would likely improve tracking as long as the pressure remains high enough for consistent color removal.

Fourth, joint-space gain tuning was manageable for the final hardware controller because the main tuning variable was a global stiffness scale s after nominal joint gains were selected. Empirically, $s = 0.3$ gave a useful compromise between contact pressure and safety. Operational-space control remains an attractive future direction because it would allow more direct tuning of wall-normal and wall-tangential stiffness. The project did not fully adopt it because the available hardware setup did not provide a reliable measured wrench and torque-level Cartesian control interface for the final deadline.

VII. FUTURE DIRECTIONS

Hardware experiments should compare the recommended joint-impedance architecture with rigid position control using repeated trials. A first improvement is to add a force/torque sensor or calibrated current-based contact estimator so the real system can regulate F_{meas} directly rather than relying only on compliance. A second improvement is to implement adaptive stiffness so that the robot can use low stiffness during contact establishment and higher stiffness only when contact is safe and stable. A third improvement is to evaluate actual painting or erasing quality with quantitative image metrics, such as stroke coverage, letter-shape error, and color-removal uniformity.

Additional future work includes testing curved or textured surfaces, improving the custom roller/eraser CAD so that it passively accommodates wall-angle errors, and validating the same metrics used in simulation—force RMSE, contact-maintenance ratio, wall-following displacement, y - z error, and tool-axis drift—on real hardware with appropriate sensors.

VIII. CONCLUSION

A directionally decomposed compliant controller was developed for a Piper manipulator painting on a moving wall. The wall's forward and backward motion represented local surface-height variation. The simulation controller used an outer-loop admittance model in the wall-normal direction, explicit y - z path tracking, tool-axis orientation control, damped inverse kinematics, and an idealized joint-position inner loop.

After conservative trajectory acceleration, the complete simulated task was executed in 38.99 s. The controller maintained contact throughout a 6.00 mm retreat, with a brush-following displacement of 4.972 mm and a following ratio of 82.86 %. During pen-down motion, the mean force was 10.99 N, the force RMSE was 1.45 N, and the peak force was 13.86 N. The mean and maximum pen-down y - z errors were 0.497 mm and 1.013 mm, and the maximum tool-axis drift was 0.062°.

All declared simulation requirements were satisfied. However, the force was estimated from a known contact model and the inner joint-position loop was idealized. The results therefore demonstrate high-level feasibility and controller-design insight rather than guaranteed hardware performance.

On the real robot, reverse painting was demonstrated by using an end-effector eraser to remove color from a pre-colored wall and write the four letters “UCLA.” The recommended real-world stiffness scale is $s = 0.3$, which balances letter legibility and safe compliance.

ACKNOWLEDGMENT

The authors thank Instructor Veronica J. Santos and TA Benjamin Forbes for their guidance on controller design and evaluation. The course project repository is available at https://github.com/chiawenwang/robot_painting [10]. The authors used OpenAI ChatGPT for language polishing and LaTeX formatting assistance. No technical results, controller design decisions, data interpretation, or final claims were generated without author review. All technical content was verified, revised, and approved by the authors.

REFERENCES

- [1] M. Suomalainen, Y. Karayiannidis, and V. Kyrki, “A survey of robot manipulation in contact,” *Robotics and Autonomous Systems*, vol. 156, p. 104224, 2022.
- [2] K. Bansal, Y. Du, E. Palan, M. Quadir, and M. K. Jawed, “Robotic painting: Mimicking human applicators,” *Journal of Coatings Technology and Research*, vol. 20, no. 4, pp. 1369–1381, 2023.
- [3] N. Hogan, “Impedance control: An approach to manipulation: Part i—theory,” *Journal of Dynamic Systems, Measurement, and Control*, vol. 107, no. 1, pp. 1–7, 1985.
- [4] F. J. Abu-Dakka and M. Saveriano, “Variable impedance control and learning—a review,” *Frontiers in Robotics and AI*, vol. 7, p. 590681, 2020.
- [5] D. E. Whitney, “Historical perspective and state of the art in robot force control,” *The International Journal of Robotics Research*, vol. 6, no. 1, pp. 3–14, 1987.
- [6] J. K. Salisbury, “Active stiffness control of a manipulator in cartesian coordinates,” in *Proceedings of the 19th IEEE Conference on Decision and Control including the Symposium on Adaptive Processes*, 1980, pp. 95–100.
- [7] AgileX Robotics, “piper_sdk: Code for controlling the piper robotic arm,” https://github.com/agilexrobotics/piper_sdk, 2026.
- [8] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 5026–5033.
- [9] J. Carpentier, G. Saurel, G. Buondonno, J. Mirabel, F. Lamiriaux, O. Stasse, and N. Mansard, “The Pinocchio C++ library: A fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives,” in *2019 IEEE/SICE International Symposium on System Integration (SII)*, 2019, pp. 614–619.
- [10] Team 13, “RoboPainting course project repository,” https://github.com/chiawenwang/robot_painting, 2026.

APPENDIX

TABLE VII: Itemized team member contributions.

Member	Contributions
Ruohan He	Completed algorithm validation in simulation and participated in final report writing.
Shixuan Sun	Built the simulation environment and supported simulation setup, testing, and validation.
Jiawen Wang	Built the real hardware platform, improved the real-world MIT impedance algorithm, contributed to the final report, and maintained the GitHub repository.
Xiaoyang Zhao	Designed and fabricated the end-effector using 3D CAD and supported physical integration.

All team members participated in controller-design discussions, including the choice of indirect force-control architecture, simulation-to-hardware transfer, and real-world stiffness selection.

TABLE VIII: Bill of materials used for the hardware demonstration. Costs were not itemized in this report draft.

Item	Qty.	Description	Source
Piper manipulator	1	Six-degree-of-freedom arm	AgileX
Mobile base/platform	1	Robot support platform	AgileX/lab
Custom eraser mount	1	Fabricated end-effector adapter	Team fabricated
Eraser material	1	Contact tip for reverse painting	Lab stock
Whiteboard/wall surface	1	Vertical drawing surface	Lab stock
Control computer/CAN interface	1 set	Real-time command computer and interface	Lab stock

The custom simulation code is `sim.py`. The real-robot MIT impedance controller is `impedance_control_mit.py`. The native files should be submitted in the Google Drive or UCLA Box folder, and the custom code should also be printed into the combined Turnitin PDF according to the course instructions.

TABLE IX: Code artifacts.

File	Purpose
<code>sim.py</code>	MuJoCo simulation, wall model, force estimator, admittance controller, plots, and video rendering.
<code>impedance_control_mit.py</code>	Piper hardware controller using Pinocchio IK and MIT joint impedance commands.
<code>main.tex</code>	IEEE-style report source.
<code>references.bib</code>	Bibliography database.

MAE C163C / C263C Project Presentation Feedback

Team #: 13

Project Title: RoboPainting

TOPIC	COMMENTS
Hard copy of slides?	Yes
Intro	OK
Methods (e.g., concepts applied from class)	- Simulation + hardware implementation - Joint-space impedance control
Results (e.g., video, error assessment) and Discussion	Clever use of a whiteboard for "reverse" painting
Sources cited properly, and on same slide as content?	Yes
Clarity of presentation	OK
Technical competence	OK
Equal presentation effort across all team members?	Yes
Ability to field questions	Why move the whiteboard? Was the workspace too large for the robot arm?
Changes/additions for project report	- Address any questions cited above. - Slide 4: Change y-axis label to be in mm or cm (not m) - By how much (in mm or cm) did you have to command the virtual path to be behind the writing surface? - Do you think you could have gotten better performance if you reduced the contact force? A contact force of 10N is kind of high, but that might be necessary for rolling paint. - How difficult was it for you to tune gains for joint-space control? I would think that operational space control would have been easier for gain tuning.

UCLA MAE C263C, Final Project Presentation Feedback (team)
Spring 2026, Course #210-682-200 / #610-383-200 / #610-383-280 (Santos)

Fig. 8: Instructor presentation feedback form for Team 13.

```
# -*- coding: utf-8 -*-
"""
True joint-level impedance control for the real Piper, using the MIT interface.
```

Why this file

~~~~~  
``impedance\_control.py`` implements a virtual-spring on top of ``EndPoseCtrl``. That call goes through Piper's on-board high-stiffness position servo, so the end-effector cannot actually be back-driven by external force: even though the script caps ``|x\_cmd - x\_actual|`` to ``F\_max/Kp``, the on-board servo treats that gap as a position error to crush at full torque.

To get true compliance, this file uses ``JointMitCtrl`` (Motor Impedance Torque), Piper's low-level joint interface:

```
JointMitCtrl(motor_num, pos_ref, vel_ref, kp, kd, t_ref)
```

with small ``kp`` / ``kd``. Each joint becomes a spring-damper with stiffness ``kp`` (N m / rad), so:

- \* In free space: joint error drives the joint toward ``pos\_ref`` -> the arm walks to the target.
- \* Push it: external force easily back-drives the joint (the spring is weak).
- \* Hold against obstacle: torque saturates at ``kp \* (q\_ref - q\_actual)`` -> bounded.

Cartesian target is converted to a joint target via Pinocchio IK (using the URDF at ``piper\_ros/src/piper\_description/urdf/piper\_description.urdf``).

Run env: conda env ``piper\_work\_env`` (which has ``pin`` installed).  
~~~~~

```
import argparse
import math
import os
import select
import sys
import termios
import time
import tty
from contextlib import contextmanager
from dataclasses import dataclass, field
from typing import List, Optional, Tuple

import numpy as np

try:
    import pinocchio as pin
except ImportError as e:
    raise ImportError(
        "pinocchio is required. In the piper_work_env conda env run:\n"
        "    pip install pin\n"
        f"Original error: {e}"
    )

from piper_sdk import C_PiperInterface_V2

# ----- constants
_DEFAULT_URDF = os.path.join(
    os.path.expanduser("~"), "piper_robot",
    "piper_ros/src/piper_description/urdf/piper_description.urdf",
)

# MotionCtrl_2 mask for MIT mode (move_mode=0x04, mit_mode=0xAD).
_MIT_CTRL_2 = (0x01, 0x04, 0, 0xAD)

# rad -> 0.001 deg, matches the demo convention.
_RAD_TO_MDEG = 1000.0 * 180.0 / math.pi
```

```
# m -> 0.001 mm (the unit EndPoseCtrl expects).
_M_TO_UMM = 1.0e6
```

```
# ===== non-blocking stdin ==
```

```
@contextmanager
def _raw_stdin():
    """Put stdin into cbreak mode so single keypresses are readable without
    waiting for Enter. Restores terminal state on exit. No-op when stdin is
    not a TTY."""
    if not sys.stdin.isatty():
        yield False
        return
    fd = sys.stdin.fileno()
    old = termios.tcgetattr(fd)
    try:
        tty.setcbreak(fd)
        yield True
    finally:
        termios.tcsetattr(fd, termios.TCSADRAIN, old)
```

```
def _key_pressed() -> Optional[str]:
    """Return the most recent buffered key (single char) or None."""
    if not sys.stdin.isatty():
        return None
    last = None
    while True:
        r, _, _ = select.select([sys.stdin], [], [], 0)
        if not r:
            break
        last = sys.stdin.read(1)
    return last
```

```
# ===== kinematics ==
```

```
class PiperKinematics:
    """Pinocchio wrapper for Piper FK / IK.

    Locks the gripper joints (`joint7`, `joint8`) so the reduced model has
    exactly the 6 controllable joints. The EE frame is added at `joint6` with
    identity offset, matching `piper_ros/.../piper_pinocchio.py`.
    """
```

```
def __init__(self, urdf_path: str = _DEFAULT_URDF):
    if not os.path.isfile(urdf_path):
        raise FileNotFoundError(f"URDF not found: {urdf_path}")

    full_model = pin.buildModelFromUrdf(urdf_path)
    locked = [
        full_model.getJointId(name)
        for name in ("joint7", "joint8")
        if full_model.existJointName(name)
    ]
    self.model = pin.buildReducedModel(
        full_model, locked, np.zeros(full_model.nq)
    )
    self.data = self.model.createData()

    if not self.model.existFrame("ee"):
        ee_frame = pin.Frame(
            "ee",
            self.model.getJointId("joint6"),
            pin.SE3.Identity(),
            pin.FrameType.OP_FRAME,
        )
        self.model.addFrame(ee_frame)
        self.data = self.model.createData()
    self.ee_id = self.model.getFrameId("ee")
```

```
assert self.model.nq == 6, f"reduced model has nq={self.model.nq}, expected 6"
print(f"[mit-kin] loaded URDF, nq={self.model.nq}, ee frame id={self.ee_id}")
```

```
def fk(self, q: np.ndarray) -> pin.SE3:
    pin.forwardKinematics(self.model, self.data, q)
    pin.updateFramePlacements(self.model, self.data)
    return self.data.oMf[self.ee_id].copy()
```

```
def ik(
    self,
    target_se3: pin.SE3,
    q_init: np.ndarray,
    max_iters: int = 300,
    eps: float = 1e-4,
    damping: float = 1e-4,
    step: float = 0.6,
) -> Tuple[np.ndarray, bool]:
    """Damped least-squares IK in local (body) frame. Returns (q, success)."""
    q = q_init.copy()
    I6 = np.eye(6)
    for _ in range(max_iters):
        pin.forwardKinematics(self.model, self.data, q)
        pin.updateFramePlacements(self.model, self.data)
        cur = self.data.oMf[self.ee_id]
        err = pin.log(cur.actInv(target_se3)).vector
        if np.linalg.norm(err) < eps:
            return q, True
        J = pin.computeFrameJacobian(
            self.model, self.data, q, self.ee_id, pin.ReferenceFrame.LOCAL
        )
        dq = J.T @ np.linalg.solve(J @ J.T + damping * I6, err)
        q = pin.integrate(self.model, q, step * dq)
    return q, False
```

```
# ===== params ==
@dataclass
class MitImpedanceParams:
    """Per-joint MIT controller gains and behaviour knobs."""

    mit_kp: List[float] = field(default_factory=lambda: [12.0, 18.0, 15.0, 10.0, 8.0, 6.0])
    mit_kd: List[float] = field(default_factory=lambda: [1.2, 1.5, 1.2, 0.8, 0.6, 0.5])

    # Single soft<->stiff knob. kp_eff = mit_kp * stiffness, kd_eff = mit_kd * sqrt(stiffness).
    stiffness: float = 1.0

    # Cap per-joint position error to bound max torque. 0 disables.
    max_joint_err_rad: float = 0.35 # ~20 deg

    rate_hz: float = 200.0
    ramp_s: float = 2.0

    # Re-assert MIT mode every cycle (cheap; matches the demo).
    reassert_mode_each_cycle: bool = True
```

```
# ===== letter path ==
class LetterPath:
    """2D letter path in the XZ plane, defined in letter-local coordinates.
```

Coordinate convention

```
-----
* (0, 0) is the letter's start point (top-left by convention).
* +x → right
* +z → up (so descending strokes use negative z values)
* The Y dimension is fixed in 3D; it is supplied by the world origin at
  evaluation time and never changes within a letter.
```

Usage

Build the path by chaining `.line_to()` and `.arc()` calls, then pass the instance to `PiperMitImpedance.run_letter_mit()`.

Example – letter U (`arc_radius = height = 5 cm`):

```
path = (LetterPath(v_speed=0.03)
        .line_to(0.00, -0.05)      # left arm down
        .arc(0.05, -0.05, 0.05, 180, 360) # bottom semicircle
        .line_to(0.10, 0.00))      # right arm up
.....

def __init__(self, v_speed: float = 0.02):
    self.v_speed = float(v_speed)
    self._segs: List[Tuple] = [] # each: (duration, callable(t)->np.ndarray[x,z])
    self._total_time: float = 0.0
    self._pos = np.zeros(2, dtype=float) # current pen position (x, z)

# ----- builders
def line_to(self, x: float, z: float) -> "LetterPath":
    """Straight stroke to letter-frame point (x, z)."""
    p0 = self._pos.copy()
    p1 = np.array([x, z], dtype=float)
    dist = float(np.linalg.norm(p1 - p0))
    if dist < 1e-9:
        return self
    dur = dist / self.v_speed
    self._segs.append((dur, lambda t, a=p0, b=p1, d=dur: a + (t / d) * (b - a)))
    self._total_time += dur
    self._pos = p1.copy()
    return self

def arc(
    self,
    cx: float, cz: float,
    radius: float,
    start_deg: float, end_deg: float,
) -> "LetterPath":
    """Circular arc centred at (cx, cz) with given radius.

    Angles use math convention: 0° = +x, 90° = +z, 180° = -x, 270° = -z.
    The sweep direction follows the sign of (end_deg - start_deg).
    """
    span_rad = math.radians(end_deg - start_deg)
    arc_len = abs(span_rad) * radius
    if arc_len < 1e-9:
        return self
    dur = arc_len / self.v_speed
    a0 = math.radians(start_deg)
    a1 = math.radians(end_deg)
    c = np.array([cx, cz], dtype=float)
    self._segs.append((
        dur,
        lambda t, c=c, r=radius, a0=a0, a1=a1, d=dur: (
            c + r * np.array([math.cos(a0 + (t / d) * (a1 - a0)),
                             math.sin(a0 + (t / d) * (a1 - a0))])
        ),
    ))
    self._total_time += dur
    self._pos = c + radius * np.array([math.cos(a1), math.sin(a1)])
    return self

def ellipse_arc(
    self,
    cx: float, cz: float,
    rx: float, rz: float,
    start_deg: float, end_deg: float,
) -> "LetterPath":
    """Elliptical arc centred at (cx, cz) with semi-axes rx (+x) and rz (+z).

    Angles use math convention: 0° = +x, 90° = +z.
```

Arc length is approximated numerically for duration calculation.

```
"""
span_rad = math.radians(end_deg - start_deg)
n = 200
thetas = np.linspace(math.radians(start_deg), math.radians(end_deg), n + 1)
dxs = -rx * np.sin(thetas[:-1]) * (span_rad / n)
dzs = rz * np.cos(thetas[:-1]) * (span_rad / n)
arc_len = float(np.sum(np.sqrt(dxs ** 2 + dzs ** 2)))
if arc_len < 1e-9:
    return self
dur = arc_len / self.v_speed
a0 = math.radians(start_deg)
a1 = math.radians(end_deg)
self._segs.append((
    dur,
    lambda t, cx=cx, cz=cz, rx=rx, rz=rz, a0=a0, a1=a1, d=dur: np.array([
        cx + rx * math.cos(a0 + (t / d) * (a1 - a0)),
        cz + rz * math.sin(a0 + (t / d) * (a1 - a0)),
    ]),
))
self._total_time += dur
self._pos = np.array([cx + rx * math.cos(a1), cz + rz * math.sin(a1)])
return self
```

----- query

```
@property
def total_time(self) -> float:
    return self._total_time
```

```
@property
def end_local(self) -> np.ndarray:
    """End point in letter-local (x, z)."""
    return self._pos.copy()
```

----- eval

```
def __call__(self, t: float, origin: np.ndarray) -> np.ndarray:
    """Absolute 3D world position at time t into the letter stroke.

    origin : 3D start point of the letter (e.g. top-left corner in world).
    Returns np.ndarray shape (3,).
    """
    xz = self._eval_xz(t)
    return np.array([origin[0] + xz[0], origin[1], origin[2] + xz[1]])

def _eval_xz(self, t: float) -> np.ndarray:
    for dur, fn in self._segs:
        if t <= dur:
            return fn(t)
        t -= dur
    _, fn = self._segs[-1] # past the end -> hold last point
    return fn(self._segs[-1][0])
```

===== controller ==

```
class PiperMitImpedance:
    """Standalone MIT-mode impedance controller for the Piper arm."""

    def __init__(
        self,
        can_name: str = "can0",
        urdf_path: str = _DEFAULT_URDF,
        home_joints_rad: Optional[List[float]] = None,
        start_z_offset_m: float = 0.0,
    ):
        self.can_name = can_name
        self.piper = C_PiperInterface_V2(can_name)
        self.home_joints_rad: List[float] = (
            list(home_joints_rad) if home_joints_rad is not None else [0.0] * 6
        )
        self.start_z_offset_m: float = float(start_z_offset_m)
```

```

self.kin = PiperKinematics(urdf_path)

# ----- setup
def connect_and_enable(self) -> None:
    self.piper.ConnectPort(start_thread=True, piper_init=True)
    print(f"[mit] connected on {self.can_name}")

    while not self.piper.EnablePiper():
        time.sleep(0.01)
    time.sleep(0.1)
    print("[mit] enabled")

    self.piper.GripperCtrl(0, 1000, 0x02, 0)
    self.piper.GripperCtrl(0, 1000, 0x01, 0)

    self._set_ctrl_mode_can()

def _set_ctrl_mode_can(self) -> bool:
    """Transition to CAN control mode (mirrors piper_control.py:set_ctrl_mode2can)."""
    ctrl_mode_str = {0x00: "STANDBY", 0x01: "CAN_CTRL", 0x02: "TEACH"}
    cur = self.piper.GetArmStatus().arm_status.ctrl_mode
    print(f"[mit] ctrl_mode={ctrl_mode_str.get(cur, hex(cur))}")

    if cur == 0x00:
        print("[mit] STANDBY -> CAN")
        self.piper.MotionCtrl_2(0x01, 0x00, 100, 0x00)
        time.sleep(1.0)
        while not self.piper.EnablePiper():
            time.sleep(0.01)
        self.piper.MotionCtrl_2(0x01, 0x00, 100, 0x00)

    elif cur == 0x02:
        print("[mit] TEACH -> STANDBY -> CAN")
        self.piper.MotionCtrl_1(0x02, 0, 0)
        time.sleep(1.0)
        self.piper.MotionCtrl_2(0x01, 0x00, 100, 0x00)
        time.sleep(1.0)
        while not self.piper.EnablePiper():
            time.sleep(0.01)
        self.piper.MotionCtrl_2(0x01, 0x00, 100, 0x00)

    elif cur == 0x01:
        print("[mit] already in CAN")
        while not self.piper.EnablePiper():
            time.sleep(0.01)
        self.piper.MotionCtrl_2(0x01, 0x00, 100, 0x00)

    else:
        print(f"[mit] unexpected ctrl_mode={hex(cur)}; trying CAN")
        self.piper.MotionCtrl_2(0x01, 0x00, 100, 0x00)
        time.sleep(1.0)
        while not self.piper.EnablePiper():
            time.sleep(0.01)
        self.piper.MotionCtrl_2(0x01, 0x00, 100, 0x00)

    self.piper.GripperCtrl(0, 1000, 0x02, 0)
    self.piper.GripperCtrl(0, 1000, 0x01, 0)

    t0 = time.time()
    while time.time() - t0 < 5.0:
        if self.piper.GetArmStatus().arm_status.ctrl_mode == 0x01:
            print("[mit] CAN control mode confirmed")
            return True
        time.sleep(0.05)

    print("[mit] WARNING: failed to confirm CAN control mode within 5 s")
    return False

def ensure_can_ready(self) -> bool:
    return self._set_ctrl_mode_can()

```

```

# ----- I0 helpers
def read_ee_pose_raw(self) -> Tuple[int, int, int, int, int, int]:
    ep = self.piper.GetArmEndPoseMsgs().end_pose
    return (ep.X_axis, ep.Y_axis, ep.Z_axis, ep.RX_axis, ep.RY_axis, ep.RZ_axis)

def read_ee_position_m(self) -> np.ndarray:
    x, y, z, *_ = self.read_ee_pose_raw()
    return np.array([x, y, z], dtype=float) / _M_TO_UMM

def _wait_reached(self, timeout_s: float, poll_s: float = 0.05) -> bool:
    """Block until motion_status == 0x00 ('reached') or timeout.

    Sleeps 0.15 s before polling so the firmware can register the new
    command; without this delay the function may return True immediately
    because the previous move just finished (motion_status is still 0x00).
    """
    time.sleep(0.15)
    t0 = time.time()
    while time.time() - t0 < timeout_s:
        if self.piper.GetArmStatus().arm_status.motion_status == 0x00:
            return True
        time.sleep(poll_s)
    return False

def read_joint_rad(self) -> np.ndarray:
    js = self.piper.GetArmJointMsgs().joint_state
    j_mdeg = [js.joint_1, js.joint_2, js.joint_3,
              js.joint_4, js.joint_5, js.joint_6]
    return np.array([math.radians(v / 1000.0) for v in j_mdeg], dtype=float)

# ----- motion
def goto_home(self, speed: int = 30, timeout_s: float = 10.0) -> bool:
    """Move to self.home_joints_rad via MOVE_J."""
    if not self.ensure_can_ready():
        print("[mit] first ensure_can_ready failed -> retrying after 1 s")
        time.sleep(1.0)
        self.ensure_can_ready()

    self._print_arm_state("entering goto_home")

    j = [int(round(v * _RAD_TO_MDEG)) for v in self.home_joints_rad]
    print(f"[mit] goto_home (joints rad): {[round(v, 4) for v in self.home_joints_rad]}")
    self.piper.MotionCtrl_2(0x01, 0x01, speed, 0x00) # CAN + MOVE_J
    self.piper.JointCtrl(j[0], j[1], j[2], j[3], j[4], j[5])

    reached = self._wait_reached(timeout_s)
    if reached:
        print(f"[mit] home reached; EE (m) = {self.read_ee_position_m()}")
    else:
        print(f"[mit] WARNING: home not confirmed within {timeout_s:.1f}s")
    return reached

def goto_start(self, speed: int = 30, timeout_s: float = 10.0) -> bool:
    """Move to home, then offset Z by start_z_offset_m via MOVE_P."""
    if not self.goto_home(speed=speed, timeout_s=timeout_s):
        print("[mit] aborting goto_start: home not reached")
        return False

    dz = self.start_z_offset_m
    if dz == 0.0:
        return True

    X, Y, Z, RX, RY, RZ = self.read_ee_pose_raw()
    Z_new = Z + int(round(dz * _M_TO_UMM))
    print(f"[mit] goto_start: Z offset {dz*1000:.0f} mm -> Z={Z_new}")
    self.piper.MotionCtrl_2(0x01, 0x00, 100, 0x00) # CAN + MOVE_P
    self.piper.EndPoseCtrl(X, Y, Z_new, RX, RY, RZ)
    time.sleep(0.01)

```

```

reached = self._wait_reached(timeout_s)
if reached:
    print(f"[mit] start reached; EE (m) = {self.read_ee_position_m()}")
else:
    print(f"[mit] WARNING: start not confirmed within {timeout_s:.1f}s")
return reached

# ----- MIT
def _send_mit(self, q_ref: np.ndarray, kp: List[float], kd: List[float]) -> None:
    for i in range(6):
        self.piper.JointMitCtrl(i + 1, float(q_ref[i]), 0.0, float(kp[i]), float(kd[i]), 0.0)

def _print_arm_state(self, tag: str) -> None:
    st = self.piper.GetArmStatus().arm_status
    ctrl = {0x00: "STANDBY", 0x01: "CAN", 0x02: "TEACH"}.get(st.ctrl_mode, hex(st.ctrl_mode))
    mfeed = {0x00: "MOVE_P", 0x01: "MOVE_J", 0x02: "MOVE_L",
             0x03: "MOVE_C", 0x04: "MIT"}.get(st.mode_feed, hex(st.mode_feed))
    ms = {0x00: "REACHED", 0x01: "MOVING"}.get(st.motion_status, hex(st.motion_status))
    print(f"[mit-debug] {tag}: ctrl_mode={ctrl} mode_feed={mfeed} "
          f"motion_status={ms} err_code={st.err_code}")

def _exit_mit_to_can(self, base_kp: List[float], base_kd: List[float]) -> None:
    """Smooth teardown from MIT mode back to CAN/MOVE_J.

    The core trick: read the actual joint angles while MIT gains are still
    zero (arm is floating), then hand those exact angles to the MOVE_J
    position controller. The controller sees ~zero error on its first cycle
    and applies no sudden torque, preventing the disable-on-switch.
    """
    self._print_arm_state("before exit-MIT teardown")

    # 1. Ramp gains to zero so joints float before mode switch.
    try:
        for alpha in (0.6, 0.3, 0.1, 0.0):
            q_now = self.read_joint_rad()
            kp_step = [k * alpha for k in base_kp]
            kd_step = [k * math.sqrt(max(alpha, 0.0)) for k in base_kd]
            self.piper.MotionCtrl_2(*MIT_CTRL_2)
            self._send_mit(q_now, kp_step, kd_step)
            time.sleep(0.05)
        q_now = self.read_joint_rad()
        self.piper.MotionCtrl_2(*MIT_CTRL_2)
        self._send_mit(q_now, [0.0] * 6, [0.0] * 6)
        time.sleep(0.1)
    except Exception as e:
        print(f"[mit-debug] gain-rampdown error (ignored): {e!r}")

    # 2. Snapshot actual joint angles while the arm is still floating.
    q_actual = self.read_joint_rad()
    j_actual = [int(round(v * _RAD_TO_MDEG)) for v in q_actual]
    print(f"[mit-debug] handoff joints (rad): {[round(v, 4) for v in q_actual]}")

    # 3. Switch to CAN + MOVE_J at low speed, target = current position.
    # Sending the target before and after EnablePiper keeps error ≈ 0.
    for _ in range(3):
        self.piper.MotionCtrl_2(0x01, 0x01, 10, 0x00) # MOVE_J, 10 % speed
        self.piper.JointCtrl(*j_actual)
        time.sleep(0.08)

    t0 = time.time()
    while not self.piper.EnablePiper():
        if time.time() - t0 > 2.0:
            print("[mit-debug] EnablePiper timed out after MIT exit")
            break
        time.sleep(0.01)

    # 4. Reaffirm target after enable so the controller latches onto it.
    self.piper.MotionCtrl_2(0x01, 0x01, 10, 0x00)
    self.piper.JointCtrl(*j_actual)
    time.sleep(0.3)

```

```

self._print_arm_state("after exit-MIT teardown")

def run_to_target_mit(
    self,
    target_m: np.ndarray,
    params: MitImpedanceParams = MitImpedanceParams(),
    duration_s: Optional[float] = 30.0,
    quit_joints_rad: Optional[List[float]] = None,
) -> None:
    """Drive EE toward target_m and hold it with joint-level compliance.

    quit_joints_rad: if provided, the arm moves to these joint angles after
    exiting MIT mode (on 'q', duration reached, or Ctrl-C).
    """
    target_m = np.asarray(target_m, dtype=float).reshape(3)

    q_start = self.read_joint_rad()
    ee_start = self.kin.fk(q_start)
    target_pose = pin.SE3(ee_start.rotation, target_m)
    q_target, ok = self.kin.ik(target_pose, q_start)
    if not ok:
        print(f"[mit] WARNING: IK did not converge to target {target_m}. "
              f"Using best-effort q_target = {q_target}.")
    print(f"[mit] q_start (rad): {q_start.round(4)}")
    print(f"[mit] q_target (rad): {q_target.round(4)}")
    print(f"[mit] ee_start (m) : {ee_start.translation.round(4)}")
    print(f"[mit] target (m) : {target_m.round(4)}")

    if params.stiffness <= 0.0:
        raise ValueError(f"stiffness must be > 0, got {params.stiffness}")
    scale_p = float(params.stiffness)
    scale_d = math.sqrt(scale_p)
    kp_eff = [k * scale_p for k in params.mit_kp]
    kd_eff = [k * scale_d for k in params.mit_kd]
    tau_max_eff = [k * params.max_joint_err_rad for k in kp_eff]
    print(f"[mit] stiffness={scale_p:.2f} "
          f"kp_eff={[round(k, 2) for k in kp_eff]} "
          f"kd_eff={[round(k, 2) for k in kd_eff]} "
          f"tau_max={[round(t, 2) for t in tau_max_eff]} Nm")

    print("[mit] entering MIT mode")
    self.piper.MotionCtrl_2(*MIT_CTRL_2)
    time.sleep(0.05)
    self._send_mit(q_start, kp_eff, kd_eff)
    time.sleep(0.05)

    dt = 1.0 / params.rate_hz
    t0 = time.time()
    last_log = -1.0

    try:
        with _raw_stdin() as kb_active:
            if kb_active:
                print("[mit] press 'q' at any time to abort")
            while True:
                loop_start = time.time()
                now = loop_start - t0

                key = _key_pressed()
                if key in ("q", "Q"):
                    print("[mit] 'q' pressed -> returning to zero")
                    break

                q = self.read_joint_rad()

                if params.ramp_s > 0 and now < params.ramp_s:
                    alpha = now / params.ramp_s
                    q_d = q_start + alpha * (q_target - q_start)
                else:
                    q_d = q_target.copy()

```

```

        if params.max_joint_err_rad > 0:
            err = q_d - q
            np.clip(err, -params.max_joint_err_rad, params.max_joint_err_rad, out=err)
            q_d_clamped = q + err
        else:
            q_d_clamped = q_d

        if params.reassert_mode_each_cycle:
            self.piper.MotionCtrl_2(*MIT_CTRL_2)
        self._send_mit(q_d_clamped, kp_eff, kd_eff)

        if now - last_log >= 0.5:
            last_log = now
            x_act = self.kin.fk(q).translation
            err_cart = float(np.linalg.norm(target_m - x_act))
            err_joint = q_target - q
            tau_approx = np.array(kp_eff) * (q_d_clamped - q)
            print(f"[mit] t={now:5.2f}s ee_err={{err_cart*1000:6.1f}}mm "
                  f"|q_err|max={{float(np.max(np.abs(err_joint))):5.3f}}rad "
                  f"|tau_p|max={{float(np.max(np.abs(tau_approx))):5.2f}}Nm "
                  f"(press q to quit)")

        if duration_s is not None and now >= duration_s:
            print(f"[mit] duration reached ({duration_s:.1f}s)")
            break

        sleep_left = dt - (time.time() - loop_start)
        if sleep_left > 0:
            time.sleep(sleep_left)

    finally:
        print("[mit] exiting MIT mode -> CAN/MOVE_P")
        try:
            self._exit_mit_to_can(params.mit_kp, params.mit_kd)
        except Exception as e:
            print(f"[mit] teardown failed: {e!r}")

        if quit_joints_rad is not None:
            print(f"[mit] going to quit position: {[round(v, 4) for v in quit_joints_rad]}")
            _saved = self.home_joints_rad
            self.home_joints_rad = list(quit_joints_rad)
            self.goto_home(speed=30, timeout_s=10.0)
            self.home_joints_rad = _saved

def run_trajectory_mit(
    self,
    traj_fn,
    ramp_target_m: np.ndarray,
    params: MitImpedanceParams = MitImpedanceParams(),
    duration_s: Optional[float] = 30.0,
    quit_joints_rad: Optional[List[float]] = None,
) -> None:
    """Drive EE along a time-varying Cartesian trajectory with joint-level compliance.

    During ramp_s the arm interpolates (joint-space) toward ramp_target_m.
    After ramp_s, traj_fn(t_after_ramp) is called each cycle to get the
    Cartesian target; IK is solved with warm-starting from the previous result.

    traj_fn: callable(float) -> np.ndarray shape (3,)
        Returns desired EE position in metres at time t seconds after the
        ramp phase ends.
    ramp_target_m: Cartesian position to ramp toward before the trajectory starts.
    quit_joints_rad: joint angles to move to after exiting MIT mode.
    """
    ramp_target_m = np.asarray(ramp_target_m, dtype=float).reshape(3)

    q_start = self.read_joint_rad()
    ee_start = self.kin.fk(q_start)

```

```

ramp_pose = pin.SE3(ee_start.rotation, ramp_target_m)
q_ramp, ok = self.kin.ik(ramp_pose, q_start)
if not ok:
    print(f"[mit] WARNING: IK did not converge to ramp target {ramp_target_m}. "
          f"Using best-effort.")
print(f"[mit] q_start      (rad): {q_start.round(4)}")
print(f"[mit] q_ramp_target(rad): {q_ramp.round(4)}")
print(f"[mit] ee_start      (m)  : {ee_start.translation.round(4)}")
print(f"[mit] ramp_target   (m)  : {ramp_target_m.round(4)}")

if params.stiffness <= 0.0:
    raise ValueError(f"stiffness must be > 0, got {params.stiffness}")
scale_p = float(params.stiffness)
scale_d = math.sqrt(scale_p)
kp_eff = [k * scale_p for k in params.mit_kp]
kd_eff = [k * scale_d for k in params.mit_kd]
tau_max_eff = [k * params.max_joint_err_rad for k in kp_eff]
print(f"[mit] stiffness={scale_p:.2f} "
      f"kp_eff={round(k, 2) for k in kp_eff} "
      f"kd_eff={round(k, 2) for k in kd_eff} "
      f"tau_max={round(t, 2) for t in tau_max_eff} Nm")

print("[mit] entering MIT mode (trajectory)")
self.piper.MotionCtrl_2(*_MIT_CTRL_2)
time.sleep(0.05)
self._send_mit(q_start, kp_eff, kd_eff)
time.sleep(0.05)

dt = 1.0 / params.rate_hz
t0 = time.time()
last_log = -1.0
# Warm-start IK from the ramp target after ramp phase.
q_ik_prev = q_ramp.copy()

try:
    with _raw_stdin() as kb_active:
        if kb_active:
            print("[mit] press 'q' at any time to abort")
        while True:
            loop_start = time.time()
            now = loop_start - t0

            key = _key_pressed()
            if key in ("q", "Q"):
                print("[mit] 'q' pressed -> returning to zero")
                break

            q = self.read_joint_rad()

            if params.ramp_s > 0 and now < params.ramp_s:
                alpha = now / params.ramp_s
                q_d = q_start + alpha * (q_ramp - q_start)
                cart_target = ramp_target_m
            else:
                t_traj = now - params.ramp_s
                cart_target = np.asarray(traj_fn(t_traj), dtype=float).reshape(3)
                tgt_pose = pin.SE3(ee_start.rotation, cart_target)
                q_ik, ik_ok = self.kin.ik(tgt_pose, q_ik_prev)
                if ik_ok:
                    q_ik_prev = q_ik
                    q_d = q_ik_prev.copy()

            if params.max_joint_err_rad > 0:
                err = q_d - q
                np.clip(err, -params.max_joint_err_rad, params.max_joint_err_rad, out=err)
                q_d_clamped = q + err
            else:
                q_d_clamped = q_d

            if params.reassert_mode_each_cycle:

```

```

        self.piper.MotionCtrl_2(*MIT_CTRL_2)
        self._send_mit(q_d_clamped, kp_eff, kd_eff)

        if now - last_log >= 0.5:
            last_log = now
            x_act = self.kin.fk(q).translation
            err_cart = float(np.linalg.norm(cart_target - x_act))
            tau_approx = np.array(kp_eff) * (q_d_clamped - q)
            t_traj_display = max(0.0, now - params.ramp_s)
            print(f"[mit] t={now:5.2f}s  t_traj={t_traj_display:5.2f}s  "
                  f"ee_err={err_cart*1000:6.1f}mm  "
                  f"target(m)={cart_target.round(4)}  "
                  f"|tau_p|max={float(np.max(np.abs(tau_approx))):5.2f}Nm  "
                  f"(press q to quit)")

        if duration_s is not None and now >= duration_s:
            print(f"[mit] duration reached ({duration_s:.1f}s)")
            break

        sleep_left = dt - (time.time() - loop_start)
        if sleep_left > 0:
            time.sleep(sleep_left)

    finally:
        print("[mit] exiting MIT mode -> CAN/MOVE_P")
        try:
            self._exit_mit_to_can(params.mit_kp, params.mit_kd)
        except Exception as e:
            print(f"[mit] teardown failed: {e!r}")

        if quit_joints_rad is not None:
            print(f"[mit] going to quit position: {[round(v, 4) for v in quit_joints_rad]}")
            _saved = self.home_joints_rad
            self.home_joints_rad = list(quit_joints_rad)
            self.goto_home(speed=30, timeout_s=10.0)
            self.home_joints_rad = _saved

def run_letter_mit(
    self,
    letter: LetterPath,
    forward_m: np.ndarray,
    params: MitImpedanceParams = MitImpedanceParams(),
    repeat: int = 1,
    duration_s: float = 300.0,
    quit_joints_rad: Optional[List[float]] = None,
) -> None:
    """Run the full sequence: ramp forward -> draw letter -> return to start.

    Parameters
    -----
    letter      : LetterPath instance defining the stroke in XZ-local coords.
    forward_m   : 3D offset from current EE position to the letter origin
                  (top-left corner), e.g. np.array([0, 0.10, 0]) for 10 cm in Y.
    params      : MIT impedance gains / timing.
    repeat      : how many times to draw the letter (ramp only once; letter ->
                  return cycles repeat times, then holds at x0).
    duration_s  : how long to hold at x0 after all repetitions finish
                  (press 'q' to exit early).
    quit_joints_rad : joint angles to move to after MIT mode ends.
    """
    x0 = self.read_ee_position_m()
    letter_origin = x0 + np.asarray(forward_m, dtype=float)

    end_xz = letter.end_local
    letter_end_3d = np.array([
        letter_origin[0] + end_xz[0],
        letter_origin[1],
        letter_origin[2] + end_xz[1],
    ])

```

```

return_dist = float(np.linalg.norm(letter_end_3d - x0))
t_return = return_dist / letter.v_speed if return_dist > 1e-6 else 0.1
t_letter = letter.total_time
t_cycle = t_letter + t_return # one full letter + return
t_all = t_cycle * max(repeat, 1) # total trajectory time

print(f"[mit] letter_origin={letter_origin.round(4)} "
      f"letter_end={letter_end_3d.round(4)} "
      f"t_letter={t_letter:.1f}s t_return={t_return:.1f}s "
      f"repeat={repeat} t_all={t_all:.1f}s")

def full_traj(t: float) -> np.ndarray:
    if t >= t_all: # all reps done → hold at x0
        return x0.copy()
    t_in_cycle = t % t_cycle
    if t_in_cycle < t_letter:
        return letter(t_in_cycle, letter_origin)
    t_r = t_in_cycle - t_letter
    frac = t_r / t_return
    return letter_end_3d + frac * (x0 - letter_end_3d)

self.run_trajectory_mit(
    full_traj,
    ramp_target_m=letter_origin,
    params=params,
    duration_s=duration_s,
    quit_joints_rad=quit_joints_rad,
)

```

===== demo

```

if __name__ == "__main__":
    """

```

Motion sequence:

- 1) connect + enable;
- 2) go to INIT position: joint1 = home_j1, joints 2-6 = 0;
- 3) go to HOME (= impedance start): full home_joints_rad;
- 4) MIT impedance hold toward target for duration_s;
- 5) return to INIT position on exit.

Usage:

```

python impedance_control_mit.py --letter U # default
python impedance_control_mit.py --letter C
python impedance_control_mit.py --letter L
python impedance_control_mit.py --letter A # downward-facing C (arch)
"""

```

```

parser = argparse.ArgumentParser(
    description="Draw a letter with MIT impedance control."
)
parser.add_argument(
    "--letter",
    choices=["U", "C", "L", "A"],
    default="U",
    help=(
        "Letter to draw in the XZ plane: "
        "U, C, L, or A (A = downward-facing C / arch shape). "
        "Default: U."
    ),
)
args = parser.parse_args()

```

----- positions

```

# Keep this in sync with piper_control.py home_joints_rad.
_HOME_JOINTS_RAD = [1.57, 0.866, -0.960, 0.0, 0.182, 0]

```

Initial and final resting pose: only joint 1 matches home, rest at zero.

```

_INIT_JOINTS_RAD = [_HOME_JOINTS_RAD[0], 0.0, 0.0, 0.0, 0.0, 0.0]

```

----- setup

```

time.sleep(5.0)

```

```

ctrl = PiperMitImpedance(
    can_name="can0",
    urdf_path=_DEFAULT_URDF,
    home_joints_rad=_HOME_JOINTS_RAD,
    start_z_offset_m=0,
)
ctrl.connect_and_enable()

# Step 1: go to initial position
print("[mit] moving to init position...")
ctrl.home_joints_rad = _INIT_JOINTS_RAD
ctrl.goto_home(speed=30, timeout_s=10.0)
input("[mit] reached init position. Press Enter to continue to home...")

# Step 2: go to home = impedance start
print("[mit] moving to home (start) position...")
ctrl.home_joints_rad = _HOME_JOINTS_RAD
ctrl.goto_start(speed=30, timeout_s=10.0)
input("[mit] reached home position. Press Enter to start drawing...")

# ----- letter
# All paths are defined in local XZ coords (origin = pen start).
# +x → right +z → up (down strokes use negative z)
# Letter height / width: ~10 cm.
print(f"[mit] drawing letter: {args.letter}")
if args.letter == "U":
    # Three straight segments: left arm down → bottom → right arm up. Width: 6 cm, height: 8 cm.
    letter = (LetterPath(v_speed=0.02)
        .line_to(0.00, -0.08) # left arm down
        .line_to(0.06, -0.08) # bottom across
        .line_to(0.06, 0.00)) # right arm up
elif args.letter == "C":
    # Origin (0,0) = top-right corner. Three straight segments. Width: 8 cm, height: 8 cm.
    letter = (LetterPath(v_speed=0.02)
        .line_to(-0.04, 0.00) # top stroke left
        .line_to(-0.04, -0.08) # left side down
        .line_to(0.00, -0.08)) # bottom stroke right
elif args.letter == "L":
    # Vertical stroke down → horizontal stroke right. Width: 6 cm, height: 8 cm.
    letter = (LetterPath(v_speed=0.02)
        .line_to(0.00, -0.08) # down 8 cm
        .line_to(0.06, -0.08)) # right 6 cm
elif args.letter == "A":
    # Descend 8 cm, then draw a narrow-tall downward-facing C (n arch).
    # rx=0.03 → width 6 cm; rz=0.07 → arch rises 7 cm above base.
    letter = (LetterPath(v_speed=0.02)
        .line_to(0.00, -0.08) # down 8 cm
        .ellipse_arc(0.03, -0.08, 0.03, 0.07, 180, 0)) # narrow-tall arch

# ----- params
params = MitImpedanceParams(
    mit_kp=[10.0, 8.0, 8.0, 8.0, 6.0, 6.0],
    mit_kd=[0.8, 0.6, 0.6, 0.6, 0.5, 0.5],
    stiffness=0.3,
    max_joint_err_rad=0.35,
    rate_hz=200.0,
    ramp_s=2.0,
)

# ----- run
try:
    ctrl.run_letter_mit(
        letter,
        forward_m=np.array([0.0, 0.10, 0.0]), # advance 10 cm in +Y
        params=params,
        repeat=1, # draw the letter twice
        duration_s=300.0, # hold at x0 after; press 'q' to exit
        quit_joints_rad=_INIT_JOINTS_RAD,
    )

```

```
except KeyboardInterrupt:  
    print("[mit] interrupted")
```

```

#!/usr/bin/env python3
from __future__ import annotations

# %% [markdown]
# # MuJoCo Admittance-Control Painting Demo
#
# This script is written in a notebook-friendly style so it can be copied into
# Google Colab or run directly as a `.py` file. It demonstrates a simplified
# 6-DOF "Piper-like" arm painting on a moving wall while regulating the normal
# contact force with a wall-referenced impedance-style controller.
#
# Key ideas:
# - The wall first moves toward the robot, then eases back while staying no farther
#   than its original `x` position along the surface normal.
# - The brush follows a nominal tangential painting path in the wall's `y-z` plane.
# - A wall-referenced impedance controller offsets the brush in `x` to regulate contact force.
# - If contact is lost, the controller performs a slow, safe search toward the wall.

# %% Imports and optional package installation
import importlib
import argparse
import json
import math
import os
import subprocess
import sys
import tempfile
import time
import xml.etree.ElementTree as ET
from pathlib import Path
from textwrap import dedent

SCRIPT_DIR = Path(__file__).resolve().parent if "__file__" in globals() else Path.cwd()
IN_COLAB = "google.colab" in sys.modules or os.getenv("COLAB_GPU") is not None
ALLOW_RUNTIME_PIP_INSTALL = IN_COLAB
FORCE_NONINTERACTIVE_MPL = sys.platform == "darwin" and not IN_COLAB

def choose_writable_dir(preferred_dir: Path, fallback_name: str) -> Path:
    """Return a writable directory, falling back to cwd or the system temp dir."""
    candidates = [
        preferred_dir,
        Path.cwd() / fallback_name,
        Path(tempfile.gettempdir()) / fallback_name,
    ]

    for candidate in candidates:
        try:
            candidate.mkdir(parents=True, exist_ok=True)
            if os.access(candidate, os.W_OK):
                return candidate
        except Exception:
            continue

    raise RuntimeError(f"Could not create a writable directory for {fallback_name}.")

MATPLOTLIB_CACHE_DIR = choose_writable_dir(SCRIPT_DIR / ".matplotlib-cache", ".matplotlib-cache")
os.environ.setdefault("MPLCONFIGDIR", str(MATPLOTLIB_CACHE_DIR))
if FORCE_NONINTERACTIVE_MPL:
    os.environ.setdefault("MPLBACKEND", "Agg")

def ensure_package(import_name: str, pip_name: str | None = None):
    """Import a package, installing it first if needed."""
    try:
        return importlib.import_module(import_name)
    except ImportError:
        install_name = pip_name or import_name
        if ALLOW_RUNTIME_PIP_INSTALL:

```

```
print(f"Installing missing package: {install_name}")
subprocess.check_call([sys.executable, "-m", "pip", "install", install_name])
return importlib.import_module(import_name)
```

```
raise RuntimeError(
    f"Missing Python package '{install_name}' in interpreter:\n"
    f" {sys.executable}\n\n"
    "This script does not auto-install packages in local desktop runs.\n"
    "In PyCharm, set the interpreter to the same Python that works in your terminal,\n"
    "for example `/opt/miniconda3/bin/python3`, then install:\n"
    " pip install mujoco numpy matplotlib imageio\n"
) from None
```

```
mujoco = ensure_package("mujoco")
np = ensure_package("numpy")
ensure_package("matplotlib")
plt = importlib.import_module("matplotlib.pyplot")
animation = importlib.import_module("matplotlib.animation")
imageio = ensure_package("imageio.v2", "imageio")
mujoco_viewer = importlib.import_module("mujoco.viewer")
try:
    imageio_ffmpeg = importlib.import_module("imageio_ffmpeg")
except ImportError:
    imageio_ffmpeg = None
```

```
try:
    ipython_display = importlib.import_module("IPython.display")
    IPyImage = ipython_display.Image
    display = ipython_display.display
except ImportError:
    IPyImage = None
    display = None
```

```
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
```

```
def set_vis_flag(flag_container, flag_name: str, enabled: bool):
    """Set a MuJoCo viewer flag only when the enum exists in the installed version."""
    vis_flag = getattr(mujoco.mjtVisFlag, flag_name, None)
    if vis_flag is not None:
        flag_container.flags[vis_flag] = enabled
```

```
# %% Tuning parameters
# Desired normal contact force at the brush tip [N].
F_DES = 10.0

# Impedance command smoothing for IK-friendly position commands.
IMP_XCMD_LPF_ALPHA = 0.35
IMP_MAX_XCMD_RATE = 0.10
IMP_REENGAGE_GAIN = 0.00025
IMP_MAX_REENGAGE_BIAS = 0.0025
PEN_DOWN_ENGAGE_TIME = 0.25
PEN_DOWN_ENGAGE_BIAS = 0.0020
IMP_CONTACT_LOSS_SEARCH_BIAS = 0.0010
IMP_RECOVERY_RATE_BOOST = 2.0
CONTROL_MODE = "unknown_wall"
```

```
CONTACT_THRESHOLD = 0.5
CONTACT_LOST_THRESHOLD = 0.3
F_MAX_SAFE = 20.0
V_SEARCH = 0.003
V_RECOVERY = 0.004
K_FORCE = 0.0004
MAX_DX_PER_STEP = 0.001
MAX_APPROACH_DISTANCE = 0.06
MAX_RECOVERY_DISTANCE = 0.04
CONTACT_CONFIRM_STEPS = 5
CONTACT_LOST_CONFIRM_STEPS = 5
```

UNKNOWN_XCMD_LPF_ALPHA = IMP_XCMD_LPF_ALPHA

```
UNKNOWN_WALL_PRESETS = {
    "stable": {
        "contact_threshold": 0.5,
        "contact_lost_threshold": 0.3,
        "f_max_safe": 20.0,
        "v_search": 0.003,
        "v_recovery": 0.004,
        "k_force": 0.0004,
        "max_dx_per_step": 0.001,
        "max_approach_distance": 0.06,
        "max_recovery_distance": 0.04,
        "contact_confirm_steps": 5,
        "contact_lost_confirm_steps": 5,
        "unknown_xcmd_lpf_alpha": UNKNOWN_XCMD_LPF_ALPHA,
    },
    "aggressive": {
        "contact_threshold": 0.3,
        "contact_lost_threshold": 0.15,
        "f_max_safe": 20.0,
        "v_search": 0.010,
        "v_recovery": 0.012,
        "k_force": 0.0006,
        "max_dx_per_step": 0.0015,
        "max_approach_distance": 0.10,
        "max_recovery_distance": 0.08,
        "contact_confirm_steps": 2,
        "contact_lost_confirm_steps": 3,
        "unknown_xcmd_lpf_alpha": UNKNOWN_XCMD_LPF_ALPHA,
    },
}
UNKNOWN_WALL_PATH_TIME_BUDGET_SCALE = 4.0
UNKNOWN_MAX_XCMD_RATE = 0.22
```

Moving wall parameters:
The wall surface normal points along world +x, which is also the approach direction.
For a clean contact-control demo, the wall uses a one-sided motion profile:
it advances toward the brush, then retreats partway without going past its
initial x position, which helps keep the paint stroke continuous.

```
WALL_X_CENTER = 0.66
WALL_HALF_THICKNESS = 0.01
WALL_HALF_WIDTH_Y = 0.26
WALL_HALF_HEIGHT_Z = 0.22
WALL_PUSH_DISTANCE = 0.011
WALL_RETREAT_DISTANCE = 0.007
WALL_MOTION_START = 2.0
WALL_PUSH_DURATION = 2.0
WALL_RETURN_DURATION = 2.4
WALL_MOTION_ENABLED = True
WALL_PROFILE_MODE = "rigid"
CONCAVE_CENTER_Y = 0.0
CONCAVE_CENTER_Z = 0.30
CONCAVE_DEPTH = 0.0
CONCAVE_SIGMA = 0.05
BUMPY_AMP = 0.004
BUMPY_FREQ_Y = 22.0
BUMPY_FREQ_Z = 18.0
BUMPY_GRID_NY = 16
BUMPY_GRID_NZ = 12
```

Contact model parameters for the virtual spring-damper estimate.
CONTACT_STIFFNESS = 2500.0
CONTACT_DAMPING = 45.0
CONTACT_LOSS_FORCE_THRESHOLD = 0.4

Brush/tool geometry.
BRUSH_RADIUS = 0.015

```

# Simulation timing.
DT = 0.01
SIM_DURATION = 14.0

# Safety clamps for commanded normal motion.
MAX_DELTA_X = 0.05

RUN_PARAMETER_SWEEP = False

# Nominal painting path in the wall's tangential plane.
PAINT_Y_CENTER = 0.0
PAINT_Z_CENTER = 0.30
PAINT_Y_AMPLITUDE = 0.04
PAINT_Z_AMPLITUDE = 0.02
PAINT_FREQUENCY = 0.12

# UCLA letter painting parameters.
PAINT_TEXT = "UCLA"
LETTER_WIDTH = 0.085
LETTER_HEIGHT = 0.135
LETTER_SPACING = 0.030
LETTER_STROKE_SPEED = 0.085
PEN_UP_TRAVEL_SPEED = 0.12
PEN_UP_CLEARANCE = 0.012
PAINT_START_DELAY = 0.8
PAINT_END_HOLD = 1.0
PAINT_MARK_SPACING = 0.005
PAINT_MARK_RADIUS = 0.005
PAINT_MARK_COUNT = 900
PAINT_MARK_OFFSET = 0.0015
PAINT_MARK_HIDDEN_Y = 2.0
PAINT_MARK_RGBA = np.array([0.15, 0.34, 0.68, 1.0], dtype=float)
TEXT_READABLE_FROM_ROBOT_SIDE = True

# IK parameters.
IK_MAX_ITERS = 25
IK_DAMPING = 2.5e-3
IK_TOL = 1.0e-4
IK_STEP_LIMIT = 0.12

# Rendering parameters.
RENDER_VIDEO = True
RENDER_WIDTH = 480
RENDER_HEIGHT = 360
RENDER_FPS = 20
RENDER_EVERY_N_STEPS = max(1, int(round(1.0 / (RENDER_FPS * DT))))
SAVE_ROBOT_3D_ANIMATION = True
ROBOT_ANIMATION_FPS = 20
ROBOT_ANIMATION_EVERY_N_STEPS = max(1, int(round(1.0 / (ROBOT_ANIMATION_FPS * DT))))

# Native MuJoCo viewer parameters.
SHOW_NATIVE_MUJOCO_VIEWER = True
VIEWER_REALTIME = True
VIEWER_SHOW_LEFT_UI = False
VIEWER_SHOW_RIGHT_UI = False
KEEP_VIEWER_OPEN_AFTER_SIM = True
SHOW_CONTACT_VISUALS = False
SHOW_TOUCH_SITE_VISUAL = False
CAMERA_AZIMUTH = -38.0
CAMERA_ELEVATION = -11.0
CAMERA_DISTANCE = 0.96
CAMERA_LOOKAT = np.array([0.59, 0.0, PAINT_Z_CENTER + 0.02], dtype=float)
ROBOT_ANIMATION_ELEVATION = 18.0
ROBOT_ANIMATION_AZIMUTH = -40.0

# Saved render output.
SAVE_RENDER_GIF = True
SAVE_RENDER_MP4 = True

# Optional touch sensor usage. The manual contact estimate is usually more stable

```

```

# for this class-demo setup, so it is the default source.
USE_TOUCH_SENSOR = False

# If True, the demo approximates a very high-bandwidth inner joint servo by
# writing the IK solution directly into qpos before stepping MuJoCo. This keeps
# the notebook example stable and makes the impedance behavior easier to see.
USE_QUASI_STATIC_SERVO = True

# Paths and names for swapping in a real robot later.
EXTERNAL_MODEL_PATH = None
USE_REAL_PIPER_MODEL_IF_AVAILABLE = True
AUTO_FIND_LOCAL_PIPER_MODEL = True
AUTO_CLONE_PIPER_REPO = False
PIPER_REPO_URL = "https://github.com/agilexrobotics/piper_isaac_sim.git"
PIPER_REPO_BRANCH = "master"
PIPER_MODEL_RELATIVE_CANDIDATES = [
    Path("src/piper_description/mujoco_model/piper_description.xml"),
    Path("src/piper_description/mujoco_model/piper_no_gripper_description.xml"),
    Path("piper_description/mujoco_model/piper_description.xml"),
    Path("piper_description/mujoco_model/piper_no_gripper_description.xml"),
]
PIPER_ATTACHMENT_BODY_CANDIDATES = ["link6", "link7", "gripper_base", "tool_mount"]
EXTERNAL_BRUSH_OFFSET = np.array([0.0, 0.0, 0.165], dtype=float)
JOINT_NAMES = [f"joint{i}" for i in range(1, 7)]
EE_SITE_NAME = "brush_tip_site"
TOUCH_SENSOR_NAME = "brush_touch"
WALL_BODY_NAME = "moving_wall"

# Derived normal-direction references.
WALL_SURFACE_CENTER = WALL_X_CENTER - WALL_HALF_THICKNESS
DESIRED_PENETRATION = F_DES / CONTACT_STIFFNESS
X_NOMINAL_BASE = WALL_SURFACE_CENTER - BRUSH_RADIUS + DESIRED_PENETRATION
X_CMD_MIN = X_NOMINAL_BASE - 0.04
X_CMD_MAX = X_NOMINAL_BASE + 0.01

def paint_mark_sites_xml(indent=""):
    lines = []
    for index in range(PAINT_MARK_COUNT):
        lines.append(
            f'{indent}<site name="paint_mark_{index:04d}" '
            f'pos="{-WALL_HALF_THICKNESS - PAINT_MARK_OFFSET:.6f} {PAINT_MARK_HIDDEN_Y:.6f} 0" '
            f'size="{PAINT_MARK_RADIUS:.6f}" '
            f'rgba="0.15 0.34 0.68 0"/>'
        )
    return "\n".join(lines)

def wall_surface_offset_local(y_local, z_local):
    if WALL_PROFILE_MODE == "concave":
        dy = float(y_local - CONCAVE_CENTER_Y)
        dz = float(z_local - CONCAVE_CENTER_Z)
        r2 = dy * dy + dz * dz
        sigma2 = max(CONCAVE_SIGMA * CONCAVE_SIGMA, 1e-9)
        return float(-CONCAVE_DEPTH * math.exp(-r2 / sigma2))
    if WALL_PROFILE_MODE == "bumpy":
        return float(BUMPY_AMP * math.sin(BUMPY_FREQ_Y * y_local) * math.cos(BUMPY_FREQ_Z * (z_local - PAINT_Z_CENTER)))
    return 0.0

def wall_geoms_xml(indent=""):
    if WALL_PROFILE_MODE != "bumpy":
        return (
            f'{indent}<geom name="paint_wall" type="box" '
            f'size="{WALL_HALF_THICKNESS} {WALL_HALF_WIDTH_Y} {WALL_HALF_HEIGHT_Z}" '
            f'rgba="0.75 0.82 0.92 1" friction="0.8 0.02 0.002" contype="1" conaffinity="1"/>'
        )
    lines = []

```

```

dy = (2.0 * WALL_HALF_WIDTH_Y) / BUMPY_GRID_NY
dz = (2.0 * WALL_HALF_HEIGHT_Z) / BUMPY_GRID_NZ
tile_half_y = 0.5 * dy
tile_half_z = 0.5 * dz
for iy in range(BUMPY_GRID_NY):
    y_local = -WALL_HALF_WIDTH_Y + (iy + 0.5) * dy
    for iz in range(BUMPY_GRID_NZ):
        z_local = -WALL_HALF_HEIGHT_Z + (iz + 0.5) * dz
        z_world = PAINT_Z_CENTER + z_local
        x_offset = wall_surface_offset_local(y_local, z_world)
        thickness = max(0.002, WALL_HALF_THICKNESS - x_offset)
        x_center_local = x_offset
        lines.append(
            f'{indent}<geom name="paint_wall_{iy:02d}_{iz:02d}" type="box" '
            f'pos="{x_center_local:.6f} {y_local:.6f} {z_local:.6f}" '
            f'size="{thickness:.6f} {tile_half_y:.6f} {tile_half_z:.6f}" '
            f'rgba="0.75 0.82 0.92 1" friction="0.8 0.02 0.002" contype="1" conaffinity="1"/>'
        )
    return "\n".join(lines)

def local_to_world_yz(local_point_yz):
    world_y = PAINT_Y_CENTER - local_point_yz[0] if TEXT_READABLE_FROM_ROBOT_SIDE else PAINT_Y_CENTER
    + local_point_yz[0]
    return np.array(
        [X_NOMINAL_BASE, world_y, PAINT_Z_CENTER + local_point_yz[1]],
        dtype=float,
    )

def world_to_paint_plane_yz(world_y, world_z):
    local_y = PAINT_Y_CENTER - world_y if TEXT_READABLE_FROM_ROBOT_SIDE else world_y - PAINT_Y_CENTER
    local_z = world_z - PAINT_Z_CENTER
    return np.array([local_y, local_z], dtype=float)

def ucla_letter_strokes():
    """Return pen-down 2D polylines for drawing UCLA in the wall's y-z plane."""
    w = LETTER_WIDTH
    h = LETTER_HEIGHT
    s = LETTER_SPACING
    total_width = 4 * w + 3 * s
    x0 = -0.5 * total_width
    z_bottom = -0.5 * h
    z_top = 0.5 * h
    z_mid = 0.02 * h

    def letter_origin(letter_index):
        return x0 + letter_index * (w + s)

    strokes = []

    # U
    u0 = letter_origin(0)
    strokes.append(np.array([[u0, z_top], [u0, z_bottom], [u0 + w, z_bottom], [u0 + w, z_top]],
        dtype=float))

    # C
    c0 = letter_origin(1)
    strokes.append(
        np.array(
            [[c0 + w, z_top], [c0, z_top], [c0, z_bottom], [c0 + w, z_bottom]],
            dtype=float,
        )
    )

    # L
    l0 = letter_origin(2)
    strokes.append(np.array([[l0, z_top], [l0, z_bottom], [l0 + w, z_bottom]], dtype=float))

```

```

# A outline and crossbar.
a0 = letter_origin(3)
strokes.append(np.array([[a0, z_bottom], [a0 + 0.5 * w, z_top], [a0 + w, z_bottom]], dtype=float))
strokes.append(np.array([[a0 + 0.25 * w, z_mid], [a0 + 0.75 * w, z_mid]], dtype=float))

return strokes

def polyline_length(points):
    if len(points) < 2:
        return 0.0
    return float(np.sum(np.linalg.norm(np.diff(points, axis=0), axis=1)))

def interpolate_polyline(points, distance_along):
    if len(points) == 1:
        return points[0].copy()

    remaining = float(np.clip(distance_along, 0.0, polyline_length(points)))
    for start, end in zip(points[:-1], points[1:]):
        segment = end - start
        segment_length = float(np.linalg.norm(segment))
        if segment_length < 1e-9:
            continue
        if remaining <= segment_length:
            return start + (remaining / segment_length) * segment
        remaining -= segment_length
    return points[-1].copy()

def build_paint_plan():
    """Build a timed path that paints UCLA with pen-up travel between strokes."""
    strokes = ucla_letter_strokes()
    segments = []
    cursor = None

    for stroke in strokes:
        if cursor is not None and np.linalg.norm(stroke[0] - cursor) > 1e-9:
            segments.append({"points": np.vstack([cursor, stroke[0]]), "pen_down": False, "speed":
PEN_UP_TRAVEL_SPEED})
        segments.append({"points": stroke, "pen_down": True, "speed": LETTER_STROKE_SPEED})
        cursor = stroke[-1].copy()

    cumulative_time = PAINT_START_DELAY
    timed_segments = []
    for segment in segments:
        length = polyline_length(segment["points"])
        duration = 0.0 if length < 1e-9 else length / segment["speed"]
        timed_segments.append(
            {
                "points": segment["points"],
                "pen_down": segment["pen_down"],
                "speed": segment["speed"],
                "length": length,
                "t_start": cumulative_time,
                "t_end": cumulative_time + duration,
            }
        )
        cumulative_time += duration

    total_duration = cumulative_time + PAINT_END_HOLD
    return {
        "segments": timed_segments,
        "total_duration": total_duration,
        "start_point": local_to_world_yz(strokes[0][0]),
        "end_point": local_to_world_yz(strokes[-1][-1]),
    }

PAINT_PLAN = build_paint_plan()

```

```

# %% Model construction
def build_simplified_piper_like_mjcf() -> str:
    """Return a self-contained MJCF string for a simple 6-DOF serial arm."""
    return dedent(
        f"""
        <mujoco model="piper_like_painting_arm">
          <compiler angle="degree" autolimits="true"/>
          <option timestep="{DT}" gravity="0 0 0" integrator="RK4" iterations="80"
ls_iterations="20"/>
          <size njmax="1000" nconmax="500"/>

          <visual>
            <headlight ambient="0.45 0.45 0.45" diffuse="0.85 0.85 0.85" specular="0.15 0.15 0.15"/>
            <rgba haze="0.98 0.99 1.0 1"/>
          </visual>

          <asset>
            <texture type="skybox" builtin="gradient" rgb1="0.92 0.96 1.0" rgb2="0.58 0.70 0.90"
width="256" height="256"/>
          </asset>

          <default>
            <joint damping="3.0" armature="0.02"/>
            <geom density="550"/>
          </default>

          <worldbody>
            <light pos="0.0 -1.0 1.6" dir="0 1 -1"/>
            <geom name="ground" type="plane" pos="0 0 0" size="2 2 0.1"
              rgba="0.96 0.96 0.96 1" contype="0" conaffinity="0"/>

            <body name="{WALL_BODY_NAME}" mocap="true" pos="{WALL_X_CENTER} 0 {PAINT_Z_CENTER}">
{wall_geoms_xml()}
{paint_mark_sites_xml()}
            </body>

            <body name="robot_base" pos="0 0 {PAINT_Z_CENTER}">
              <geom name="base_geom" type="cylinder" size="0.055 0.05"
                rgba="0.16 0.16 0.20 1" contype="0" conaffinity="0"/>

              <body name="link1" pos="0 0 0">
                <joint name="joint1" axis="0 0 1" range="-170 170"/>
                <geom name="link1_geom" type="capsule" fromto="0 0 0 0.14 0 0"
                  size="0.022" rgba="0.29 0.43 0.73 1" contype="0" conaffinity="0"/>

                <body name="link2" pos="0.14 0 0">
                  <joint name="joint2" axis="0 1 0" range="-110 110"/>
                  <geom name="link2_geom" type="capsule" fromto="0 0 0 0.12 0 0"
                    size="0.020" rgba="0.33 0.47 0.78 1" contype="0" conaffinity="0"/>

                  <body name="link3" pos="0.12 0 0">
                    <joint name="joint3" axis="0 1 0" range="-130 130"/>
                    <geom name="link3_geom" type="capsule" fromto="0 0 0 0.12 0 0"
                      size="0.019" rgba="0.37 0.51 0.82 1" contype="0" conaffinity="0"/>

                    <body name="link4" pos="0.12 0 0">
                      <joint name="joint4" axis="1 0 0" range="-180 180"/>
                      <geom name="link4_geom" type="capsule" fromto="0 0 0 0.10 0 0"
                        size="0.017" rgba="0.41 0.55 0.86 1" contype="0" conaffinity="0"/>

                      <body name="link5" pos="0.10 0 0">
                        <joint name="joint5" axis="0 1 0" range="-130 130"/>
                        <geom name="link5_geom" type="capsule" fromto="0 0 0 0.08 0 0"
                          size="0.015" rgba="0.46 0.60 0.90 1" contype="0" conaffinity="0"/>

                        <body name="link6" pos="0.08 0 0">
                          <joint name="joint6" axis="1 0 0" range="-180 180"/>
                          <geom name="link6_geom" type="capsule" fromto="0 0 0 0.06 0 0"

```

```
size="0.014" rgba="0.50 0.64 0.94 1" contype="0" conaffinity="0"/>
```

```
<body name="tool_mount" pos="0.06 0 0">
  <geom name="wrist_geom" type="sphere" size="0.022"
    rgba="0.30 0.30 0.34 1" contype="0" conaffinity="0"/>
  <geom name="brush_tip_geom" pos="0.02 0 0" type="sphere" size="
    rgba="0.95 0.28 0.20 1" friction="1.0 0.02 0.002"
    contype="1" conaffinity="1"/>
  <site name="{EE_SITE_NAME}" pos="0.02 0 0" size="0.010"
    rgba="1.0 0.0 0.0 1"/>
  <site name="brush_touch_site" pos="0.02 0 0" size="0.025"
    rgba="0.2 0.5 1.0 {0.20 if SHOW_TOUCH_SITE_VISUAL else 0.0}"/>
```

```
</body>
```

```
</body>
```

```
</body>
```

```
</body>
```

```
</body>
```

```
</body>
```

```
</body>
```

```
</worldbody>
```

```
<actuator>
```

```
  <position name="act1" joint="joint1" kp="180"/>
```

```
  <position name="act2" joint="joint2" kp="180"/>
```

```
  <position name="act3" joint="joint3" kp="160"/>
```

```
  <position name="act4" joint="joint4" kp="110"/>
```

```
  <position name="act5" joint="joint5" kp="100"/>
```

```
  <position name="act6" joint="joint6" kp="90"/>
```

```
</actuator>
```

```
<sensor>
```

```
  <touch name="{TOUCH_SENSOR_NAME}" site="brush_touch_site"/>
```

```
</sensor>
```

```
</mujoco>
```

```
"""
```

```
)
```

```
def maybe_clone_piper_repo(repo_root: Path):
```

```
    """Optionally clone the Piper ROS repository when running in Colab."""
```

```
    if repo_root.exists() or not AUTO_CLONE_PIPER_REPO:
```

```
        return repo_root if repo_root.exists() else None
```

```
    try:
```

```
        subprocess.check_call([
```

```
            [
```

```
                "git",
```

```
                "clone",
```

```
                "--depth",
```

```
                "1",
```

```
                "--branch",
```

```
                PIPER_REPO_BRANCH,
```

```
                PIPER_REPO_URL,
```

```
                str(repo_root),
```

```
            ]
```

```
        ]
```

```
        return repo_root
```

```
    except Exception as exc:
```

```
        print(f"Could not clone Piper repository automatically. Falling back if needed. Reason:
```

```
{exc}")
```

```
        return None
```

```
def candidate_piper_roots():
```

```
    """Common places where the Piper ROS repository may exist locally."""
```

```
    roots = []
```

```
    for env_name in ("PIPER_ASSET_REPO_DIR", "PIPER_ROS_DIR"):
```

```
        env_root = os.getenv(env_name)
```

```

if env_root:
    roots.append(Path(env_root))

cwd = Path.cwd()
roots.extend(
    [
        SCRIPT_DIR,
        SCRIPT_DIR / "piper_isaac_sim",
        SCRIPT_DIR / "piper_ros",
        SCRIPT_DIR.parent / "piper_isaac_sim",
        SCRIPT_DIR.parent / "piper_ros",
        cwd,
        cwd / "piper_isaac_sim",
        cwd / "piper_ros",
        cwd.parent / "piper_isaac_sim",
        cwd.parent / "piper_ros",
        Path("/content/piper_isaac_sim"),
        Path("/content/piper_ros"),
        Path.home() / "piper_isaac_sim",
        Path.home() / "piper_ros",
        Path.home() / "Desktop" / "piper_isaac_sim",
        Path.home() / "Desktop" / "piper_ros",
    ]
)

```

```

unique_roots = []
seen = set()
for root in roots:
    if root is None:
        continue
    resolved = root.expanduser()
    key = str(resolved)
    if key not in seen:
        seen.add(key)
        unique_roots.append(resolved)
return unique_roots

```

```

def find_local_piper_model():
    """Return the first discovered local Piper MuJoCo XML file."""
    for root in candidate_piper_roots():
        should_try_clone = str(root).endswith("/content/piper_isaac_sim") or
str(root).endswith("/content/piper_ros")
        actual_root = maybe_clone_piper_repo(root) if should_try_clone else root
        if actual_root is None:
            continue
        for relative_path in PIPER_MODEL_RELATIVE_CANDIDATES:
            candidate = actual_root / relative_path
            if candidate.exists():
                return candidate
    return None

```

```

def format_xyz(values):
    return " ".join(f"{float(v):.6f}" for v in values)

```

```

def absolutize_asset_paths(xml_root: ET.Element, source_dir: Path):
    for tag_name in ("mesh", "texture"):
        for element in xml_root.findall(f".//{tag_name}"):
            file_attr = element.attrib.get("file")
            if not file_attr:
                continue
            element.set("file", str((source_dir / file_attr).resolve()))

```

```

def ensure_xml_section(xml_root: ET.Element, section_name: str):
    section = xml_root.find(section_name)
    if section is None:
        section = ET.SubElement(xml_root, section_name)

```

```
return section
```

```
def find_ee_attachment_body(xml_root: ET.Element):
    for body_name in PIPER_ATTACHMENT_BODY_CANDIDATES:
        body = xml_root.find(f".//body[@name='{body_name}']")
        if body is not None:
            return body

    bodies = xml_root.findall(".//body")
    if not bodies:
        raise ValueError("No robot bodies were found in the external Piper XML.")
    return bodies[-1]
```

```
def ensure_position_actuators(xml_root: ET.Element):
    actuator = ensure_xml_section(xml_root, "actuator")
    existing_joint_actuators = {node.attrib.get("joint") for node in actuator}
    for joint_name in JOINT_NAMES:
        if joint_name in existing_joint_actuators:
            continue
        ET.SubElement(
            actuator,
            "position",
            attrib={
                "name": f"{joint_name}_paint_ctrl",
                "joint": joint_name,
                "kp": "2000",
            },
        )
```

```
def patch_external_piper_model(external_model_path: Path):
    """
    Patch the vendor Piper MuJoCo XML into a self-contained painting scene.
```

The vendor file provides the robot and actuators. This helper adds:

- the moving wall,
- a brush-tip body/geom/site,
- a touch sensor, and
- absolute mesh paths so the patched XML can be written anywhere.

```
tree = ET.parse(external_model_path)
root = tree.getroot()
source_dir = external_model_path.parent
```

```
absolutize_asset_paths(root, source_dir)
```

```
option = ensure_xml_section(root, "option")
option.set("timestep", str(DT))
```

```
worldbody = ensure_xml_section(root, "worldbody")
```

```
if root.find(".//geom[@name='ground']") is None:
    ET.SubElement(
        worldbody,
        "geom",
        attrib={
            "name": "ground",
            "type": "plane",
            "pos": "0 0 0",
            "size": "2 2 0.1",
            "rgba": "0.96 0.96 0.96 1",
            "contype": "0",
            "conaffinity": "0",
        },
    )
```

```
wall_body = root.find(f".//body[@name='{WALL_BODY_NAME}']")
if wall_body is None:
```

```

wall_body = ET.SubElement(
    worldbody,
    "body",
    attrib={
        "name": WALL_BODY_NAME,
        "mocap": "true",
        "pos": format_xyz([WALL_X_CENTER, 0.0, PAINT_Z_CENTER]),
    },
)
ET.SubElement(
    wall_body,
    "geom",
    attrib={
        "name": "paint_wall",
        "type": "box",
        "size": format_xyz([WALL_HALF_THICKNESS, WALL_HALF_WIDTH_Y, WALL_HALF_HEIGHT_Z]),
        "rgba": "0.75 0.82 0.92 1",
        "friction": "0.8 0.02 0.002",
        "contype": "1",
        "conaffinity": "1",
    },
)

for index in range(PAINT_MARK_COUNT):
    site_name = f"paint_mark_{index:04d}"
    if wall_body.find(f"//site[@name='{site_name}']") is None:
        ET.SubElement(
            wall_body,
            "site",
            attrib={
                "name": site_name,
                "pos": format_xyz([- (WALL_HALF_THICKNESS + PAINT_MARK_OFFSET),
PAINT_MARK_HIDDEN_Y, 0.0]),
                "size": f"{PAINT_MARK_RADIUS:.6f}",
                "rgba": "0.15 0.34 0.68 0",
            },
        )

attachment_body = find_ee_attachment_body(root)
if attachment_body.find(f"//site[@name='{EE_SITE_NAME}']") is None:
    tool_mount = ET.SubElement(
        attachment_body,
        "body",
        attrib={
            "name": "painting_tool_mount",
            "pos": format_xyz(EXTERNAL_BRUSH_OFFSET),
        },
    )
    ET.SubElement(
        tool_mount,
        "geom",
        attrib={
            "name": "brush_tip_geom",
            "type": "sphere",
            "size": f"{BRUSH_RADIUS:.6f}",
            "rgba": "0.95 0.28 0.20 1",
            "friction": "1.0 0.02 0.002",
            "contype": "1",
            "conaffinity": "1",
        },
    )
    ET.SubElement(
        tool_mount,
        "site",
        attrib={
            "name": EE_SITE_NAME,
            "size": "0.010",
            "rgba": "1.0 0.0 0.0 1",
        },
    )

```

```

    ET.SubElement(
        tool_mount,
        "site",
        attrib={
            "name": "brush_touch_site",
            "size": "0.025",
            "rgba": f"0.2 0.5 1.0 {0.20 if SHOW_TOUCH_SITE_VISUAL else 0.0}",
        },
    )

sensor = ensure_xml_section(root, "sensor")
existing_sensor_names = {node.attrib.get("name") for node in sensor}
if TOUCH_SENSOR_NAME not in existing_sensor_names:
    ET.SubElement(
        sensor,
        "touch",
        attrib={
            "name": TOUCH_SENSOR_NAME,
            "site": "brush_touch_site",
        },
    )

ensure_position_actuators(root)

patched_dir = Path(tempfile.gettempdir())
patched_path = patched_dir / "patched_piper_painting_scene.xml"
tree.write(patch_path, encoding="utf-8", xml_declaration=True)
return patched_path

def resolve_external_model_path():
    if EXTERNAL_MODEL_PATH is not None:
        candidate = Path(EXTERNAL_MODEL_PATH).expanduser()
        if candidate.is_dir():
            for relative_path in PIPER_MODEL_RELATIVE_CANDIDATES:
                nested = candidate / relative_path
                if nested.exists():
                    return nested, f"user-specified Piper repo: {nested}"
            elif candidate.exists():
                return candidate, f"user-specified model: {candidate}"

    if USE_REAL_PIPER_MODEL_IF_AVAILABLE and AUTO_FIND_LOCAL_PIPER_MODEL:
        candidate = find_local_piper_model()
        if candidate is not None:
            return candidate, f"auto-discovered Piper MuJoCo model: {candidate}"

    return None, None

def load_model():
    """Load an external model if provided, otherwise build the self-contained demo."""
    external_model_path, external_source = resolve_external_model_path()
    if external_model_path is not None:
        try:
            patched_model_path = patch_external_piper_model(external_model_path)
            model = mujoco.MjModel.from_xml_path(str(patched_model_path))
            source = f"{external_source} patched with wall and brush"
            return model, source
        except Exception as exc:
            print(f"Could not use the external Piper model at {external_model_path}. Falling back. Reason: {exc}")

    model = mujoco.MjModel.from_xml_string(build_simplified_piper_like_mjcf())
    source = "self-contained simplified 6-DOF Piper-like MJCF"
    return model, source

# %% Kinematics helpers
def get_joint_indices(model, joint_names):
    qpos_indices = []

```

```

dof_indices = []
lower = []
upper = []
for name in joint_names:
    joint_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_JOINT, name)
    if joint_id < 0:
        raise ValueError(
            f"Joint '{name}' was not found. Update JOINT_NAMES when swapping in a real robot
model."
        )
    qpos_indices.append(model.jnt_qposadr[joint_id])
    dof_indices.append(model.jnt_dofadr[joint_id])
    lower.append(model.jnt_range[joint_id, 0])
    upper.append(model.jnt_range[joint_id, 1])
return (
    np.array(qpos_indices, dtype=int),
    np.array(dof_indices, dtype=int),
    np.array(lower, dtype=float),
    np.array(upper, dtype=float),
)

def get_joint_actuator_indices(model, joint_names):
    """Map each commanded joint to its corresponding actuator index."""
    actuator_indices = []
    for joint_name in joint_names:
        joint_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_JOINT, joint_name)
        if joint_id < 0:
            raise ValueError(f"Joint '{joint_name}' was not found while searching for actuators.")

        matching_actuator_id = None
        for actuator_id in range(model.nu):
            transmission_target = int(model.actuator_trnid[actuator_id, 0])
            if transmission_target == joint_id:
                matching_actuator_id = actuator_id
                break

        if matching_actuator_id is None:
            raise ValueError(
                f"No actuator was found for joint '{joint_name}'. "
                "Check the external model actuators or update JOINT_NAMES."
            )
        actuator_indices.append(matching_actuator_id)

    return np.array(actuator_indices, dtype=int)

def get_ee_pose(model, data, site_id):
    """Return the current end-effector Cartesian position."""
    return data.site_xpos[site_id].copy()

def get_robot_chain_body_ids(model, joint_names):
    """Return body ids for the serial chain associated with the commanded joints."""
    body_ids = []
    for name in joint_names:
        joint_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_JOINT, name)
        if joint_id < 0:
            continue
        body_id = model.jnt_bodyid[joint_id]
        if body_id not in body_ids:
            body_ids.append(body_id)
    return body_ids

def get_robot_chain_points(model, data, body_ids, site_id):
    """Return a list of 3D points that sketch the robot chain for animation."""
    points = []
    for body_id in body_ids:
        points.append(data.xpos[body_id].copy())

```

```

points.append(data.site_xpos[site_id].copy())
return np.asarray(points, dtype=float)

```

```

def compute_jacobian(model, data, site_id):
    """Return translational and rotational Jacobians for the end-effector site."""
    jaccp = np.zeros((3, model.nv))
    jacr = np.zeros((3, model.nv))
    mujoco.mj_jacSite(model, data, jaccp, jacr, site_id)
    return jaccp, jacr

```

```

def get_site_linear_velocity(model, data, site_id):
    """Estimate site linear velocity from the translational Jacobian."""
    jaccp, _ = compute_jacobian(model, data, site_id)
    return jaccp @ data.qvel

```

```

def ik_damped_least_squares(
    model,
    ik_data,
    site_id,
    target_pos,
    q_seed,
    qpos_indices,
    dof_indices,
    joint_lower,
    joint_upper,
    damping=IK_DAMPING,
    max_iters=IK_MAX_ITERS,
    tol=IK_TOL,
):
    """
    Solve position-only inverse kinematics with a damped least-squares update.

    This keeps the code simple and easy to replace if you later use a different
    IK package or a real Piper model with task-space control.
    """
    q_trial = q_seed.copy()

    for _ in range(max_iters):
        ik_data.qpos[qpos_indices] = q_trial
        ik_data.qvel[:] = 0.0
        mujoco.mj_forward(model, ik_data)

        current_pos = ik_data.site_xpos[site_id].copy()
        position_error = target_pos - current_pos
        if np.linalg.norm(position_error) < tol:
            break

        jaccp = np.zeros((3, model.nv))
        jacr = np.zeros((3, model.nv))
        mujoco.mj_jacSite(model, ik_data, jaccp, jacr, site_id)
        J = jaccp[:, dof_indices]

        regularized = J @ J.T + (damping**2) * np.eye(3)
        dq = J.T @ np.linalg.solve(regularized, position_error)
        dq = np.clip(dq, -IK_STEP_LIMIT, IK_STEP_LIMIT)

        q_trial = np.clip(q_trial + dq, joint_lower, joint_upper)

    return q_trial

```

```

# %% Motion and control helpers
def smoothstep_fraction(t, t_start, duration):
    """Return a smooth 0->1 phase and its time derivative over the interval."""
    if duration <= 1e-9:
        return (1.0, 0.0) if t >= t_start else (0.0, 0.0)

```

```

if t <= t_start:
    return 0.0, 0.0
if t >= t_start + duration:
    return 1.0, 0.0

s = (t - t_start) / duration
phase = s * s * (3.0 - 2.0 * s)
phase_dot = (6.0 * s * (1.0 - s)) / duration
return float(phase), float(phase_dot)

```

```

def wall_motion(t):
    """
    Showcase-friendly wall motion along x.

    The wall starts at its nominal x location, moves toward the robot to
    increase contact force, then retreats partway while staying no farther away
    than its original x position. This makes the impedance response easy to see
    while keeping the painting task feasible.
    """
    if not WALL_MOTION_ENABLED:
        return float(WALL_X_CENTER), 0.0

    push_phase, push_phase_dot = smoothstep_fraction(t, WALL_MOTION_START, WALL_PUSH_DURATION)
    return_phase, return_phase_dot = smoothstep_fraction(
        t,
        WALL_MOTION_START + WALL_PUSH_DURATION,
        WALL_RETURN_DURATION,
    )

    x = WALL_X_CENTER - WALL_PUSH_DISTANCE * push_phase + WALL_RETREAT_DISTANCE * return_phase
    vx = -WALL_PUSH_DISTANCE * push_phase_dot + WALL_RETREAT_DISTANCE * return_phase_dot
    return float(x), float(vx)

```

```

def wall_surface_offset_for_yz(y, z):
    return wall_surface_offset_local(float(y), float(z))

```

```

def effective_sim_duration():
    return max(SIM_DURATION, PAINT_PLAN["total_duration"])

```

```

def painting_nominal_pose(t):
    """
    Return the nominal position and pen-down state for painting UCLA.

    When the brush is between strokes, pen_down=False so the controller can back
    away from the wall slightly instead of drawing connecting travel lines.
    """
    if t <= PAINT_START_DELAY:
        return PAINT_PLAN["start_point"].copy(), False

    for segment in PAINT_PLAN["segments"]:
        if segment["t_start"] <= t <= segment["t_end"] or (
            abs(t - segment["t_end"]) < 1e-9 and segment["t_end"] >= segment["t_start"]
        ):
            distance = segment["speed"] * max(0.0, t - segment["t_start"])
            local_point_yz = interpolate_polyline(segment["points"], distance)
            return local_to_world_yz(local_point_yz), segment["pen_down"]

    return PAINT_PLAN["end_point"].copy(), False

```

```

def get_paint_mark_site_ids(model):
    site_ids = []
    for index in range(PAINT_MARK_COUNT):
        site_name = f"paint_mark_{index:04d}"
        site_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_SITE, site_name)
        if site_id >= 0:

```

```
    site_ids.append(site_id)
    return site_ids
```

```
def reset_paint_marks(model, site_ids):
    hidden_pos = np.array([(WALL_HALF_THICKNESS + PAINT_MARK_OFFSET), PAINT_MARK_HIDDEN_Y, 0.0],
dtype=float)
    for site_id in site_ids:
        model.site_pos[site_id] = hidden_pos
        model.site_rgba[site_id] = np.array([PAINT_MARK_RGBA[0], PAINT_MARK_RGBA[1],
PAINT_MARK_RGBA[2], 0.0])
```

```
def maybe_add_paint_mark(model, site_ids, next_mark_index, ee_pos, last_mark_local_yz):
    if next_mark_index >= len(site_ids):
        return next_mark_index, last_mark_local_yz

    local_yz = world_to_paint_plane_yz(ee_pos[1], ee_pos[2])
    if abs(local_yz[0]) > WALL_HALF_WIDTH_Y or abs(local_yz[1]) > WALL_HALF_HEIGHT_Z:
        return next_mark_index, last_mark_local_yz
    if last_mark_local_yz is not None and np.linalg.norm(local_yz - last_mark_local_yz) <
PAINT_MARK_SPACING:
        return next_mark_index, last_mark_local_yz

    site_id = site_ids[next_mark_index]
    site_y = PAINT_Y_CENTER - local_yz[0] if TEXT_READABLE_FROM_ROBOT_SIDE else PAINT_Y_CENTER +
local_yz[0]
    model.site_pos[site_id] = np.array([(WALL_HALF_THICKNESS + PAINT_MARK_OFFSET), site_y,
local_yz[1]], dtype=float)
    model.site_rgba[site_id] = PAINT_MARK_RGBA
    return next_mark_index + 1, local_yz
```

```
def get_touch_sensor_value(model, data, sensor_name):
    sensor_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_SENSOR, sensor_name)
    if sensor_id < 0:
        return None
    adr = model.sensor_adr[sensor_id]
    dim = model.sensor_dim[sensor_id]
    values = data.sensordata[adr : adr + dim]
    if dim == 1:
        return float(values[0])
    return values.copy()
```

```
def contact_force(model, data, site_id, t):
    """
    Estimate the normal contact force.

    Preferred option in many MuJoCo demos is a touch sensor, but in this
    assignment the manual spring-damper estimate is often more stable and easier
    to interpret. The helper keeps both paths available.
    """
    ee_pos = get_ee_pose(model, data, site_id)
    ee_vel = get_site_linear_velocity(model, data, site_id)

    wall_x, wall_vx = wall_motion(t)
    wall_surface_x = wall_x - WALL_HALF_THICKNESS + wall_surface_offset_for_yz(ee_pos[1], ee_pos[2])

    penetration = ee_pos[0] + BRUSH_RADIUS - wall_surface_x
    closing_speed = ee_vel[0] - wall_vx

    if penetration > 0.0:
        force_manual = CONTACT_STIFFNESS * penetration + CONTACT_DAMPING * closing_speed
        force_manual = max(0.0, force_manual)
    else:
        force_manual = 0.0

    force_touch = get_touch_sensor_value(model, data, TOUCH_SENSOR_NAME)
    if force_touch is None:
```

```
force_touch = 0.0
```

```
force_used = float(force_touch) if USE_TOUCH_SENSOR else float(force_manual)
```

```
return {  
    "force_used": force_used,  
    "force_manual": float(force_manual),  
    "force_touch": float(force_touch),  
    "penetration": float(max(0.0, penetration)),  
    "wall_x": float(wall_x),  
    "wall_surface_x": float(wall_surface_x),  
    "wall_vx": float(wall_vx),  
    "ee_x": float(ee_pos[0]),  
}
```

```
# %% Plotting and rendering
```

```
def plot_results(log, output_dir):  
    time = np.asarray(log["time"])  
    wall_surface_x = np.asarray(log["wall_surface_x"])  
    ee_x = np.asarray(log["ee_x"])  
    x_cmd = np.asarray(log["x_cmd"])  
    force_meas = np.asarray(log["force_meas"])  
    force_des = np.asarray(log["force_des"])  
    contact_lost = np.asarray(log["contact_lost"], dtype=float)  
    ee_y = np.asarray(log["ee_y"])  
    ee_z = np.asarray(log["ee_z"])  
    pen_down = np.asarray(log["pen_down"], dtype=bool)  
    joint_positions = np.asarray(log["joint_positions"])  
    joint_commands = np.asarray(log["joint_commands"])  
  
    true_wall_surface_x = np.asarray(log["true_wall_surface_x"])  
    force_error = np.asarray(log["force_error"])  
    controller_state = np.asarray(log["controller_state"])  
    state_to_int = {"APPROACH": 0, "PAINTING": 1, "RECOVERY": 2, "SAFETY_STOP": 3}  
    state_values = np.array([state_to_int.get(s, -1) for s in controller_state], dtype=float)  
  
    fig1, axes = plt.subplots(5, 1, figsize=(10, 14), sharex=True)  
    axes[0].plot(time, state_values, linewidth=1.8, color="tab:blue")  
    axes[0].set_yticks([0, 1, 2, 3], ["APPROACH", "PAINTING", "RECOVERY", "SAFETY_STOP"])  
    axes[0].set_ylabel("state")  
    axes[0].set_title("Unknown-wall controller state")  
    axes[0].grid(True, alpha=0.3)  
  
    axes[1].plot(time, force_meas, label="Measured contact force", linewidth=2)  
    axes[1].plot(time, force_des, label="Desired contact force", linestyle="--")  
    axes[1].set_ylabel("Force [N]")  
    axes[1].set_title("Force tracking")  
    axes[1].grid(True, alpha=0.3)  
    axes[1].legend()  
  
    axes[2].plot(time, x_cmd, label="x_cmd", linewidth=2)  
    axes[2].plot(time, ee_x, label="ee_x", linewidth=2)  
    axes[2].plot(time, true_wall_surface_x, label="true_wall_surface_x (eval only)", linestyle="--")  
    axes[2].set_ylabel("x [m]")  
    axes[2].set_title("Command vs EE x vs true wall surface")  
    axes[2].grid(True, alpha=0.3)  
    axes[2].legend()  
  
    axes[3].plot(time, contact_lost, color="tab:red", linewidth=1.8, label="contact_lost")  
    axes[3].set_ylabel("flag")  
    axes[3].set_ylim(-0.1, 1.1)  
    axes[3].set_title("Contact-lost flag")  
    axes[3].grid(True, alpha=0.3)  
    axes[3].legend()  
  
    axes[4].plot(time, force_error, color="tab:purple", linewidth=1.8, label="force_error")  
    axes[4].set_xlabel("Time [s]")  
    axes[4].set_ylabel("N")  
    axes[4].set_title("Force error (F_DES - F_measured)")
```

```

axes[4].grid(True, alpha=0.3)
axes[4].legend()

fig1.tight_layout()
fig1_path = output_dir / "impedance_timeseries.png"
fig1.savefig(fig1_path, dpi=180, bbox_inches="tight")

fig2, axes2 = plt.subplots(1, 2, figsize=(13, 5))
painted_coords = np.array([world_to_paint_plane_yz(y, z) for y, z in zip(ee_y, ee_z)])
painted_y = painted_coords[:, 0]
painted_z = painted_coords[:, 1]
painted_y[~pen_down] = np.nan
painted_z[~pen_down] = np.nan
axes2[0].plot(painted_y, painted_z, label=f'Painted path: "{PAINT_TEXT}"', linewidth=2.5)
axes2[0].set_xlabel("wall-horizontal coordinate [m]")
axes2[0].set_ylabel("wall-vertical coordinate [m]")
axes2[0].set_title(f'Painting path in the wall plane: "{PAINT_TEXT}"')
axes2[0].axis("equal")
axes2[0].grid(True, alpha=0.3)
axes2[0].legend()

for i in range(joint_positions.shape[1]):
    axes2[1].plot(time, joint_positions[:, i], label=f"q{i+1}")
    axes2[1].plot(time, joint_commands[:, i], linestyle="--", alpha=0.55)
axes2[1].set_xlabel("Time [s]")
axes2[1].set_ylabel("Joint angle [rad]")
axes2[1].set_title("Joint trajectories (solid = actual, dashed = command)")
axes2[1].grid(True, alpha=0.3)
axes2[1].legend(ncol=2, fontsize=8)

fig2.tight_layout()
fig2_path = output_dir / "painting_path_and_joints.png"
fig2.savefig(fig2_path, dpi=180, bbox_inches="tight")

fig3 = plt.figure(figsize=(8, 6))
ax3 = fig3.add_subplot(111, projection="3d")
ax3.plot(np.asarray(log["ee_x"]), ee_y, ee_z, linewidth=2, label="Brush tip")
ax3.set_xlabel("x [m]")
ax3.set_ylabel("y [m]")
ax3.set_zlabel("z [m]")
ax3.set_title("3D end-effector trajectory")
ax3.legend()
fig3.tight_layout()
fig3_path = output_dir / "ee_3d_path.png"
fig3.savefig(fig3_path, dpi=180, bbox_inches="tight")

if "agg" not in plt.get_backend().lower():
    plt.show()
else:
    plt.close("all")
return fig1_path, fig2_path, fig3_path

```

```

def make_scene_option():
    option = mujoco.MjvOption()
    set_vis_flag(option, "mjVIS_SITE", True)
    set_vis_flag(option, "mjVIS_CONTACTPOINT", SHOW_CONTACT_VISUALS)
    set_vis_flag(option, "mjVIS_CONTACTFORCE", SHOW_CONTACT_VISUALS)
    return option

```

```

def make_renderer(model):
    renderer = mujoco.Renderer(model, height=RENDER_HEIGHT, width=RENDER_WIDTH)
    camera = mujoco.MjvCamera()
    camera.type = mujoco.mjtCamera.mjCAMERA_FREE
    camera.azimuth = CAMERA_AZIMUTH
    camera.elevation = CAMERA_ELEVATION
    camera.distance = CAMERA_DISTANCE
    camera.lookat[:] = CAMERA_LOOKAT
    return renderer, camera, make_scene_option()

```

```

def running_with_mjpython():
    return bool(os.getenv("MJPYTHON_BIN")) or "mjpython" in Path(sys.argv[0]).name.lower()

def should_launch_native_viewer():
    return SHOW_NATIVE_MUJOOCO_VIEWER and not IN_COLAB

def configure_native_viewer_camera(viewer_handle):
    with viewer_handle.lock():
        viewer_handle.cam.azimuth = CAMERA_AZIMUTH
        viewer_handle.cam.elevation = CAMERA_ELEVATION
        viewer_handle.cam.distance = CAMERA_DISTANCE
        viewer_handle.cam.lookat[:] = CAMERA_LOOKAT
        set_vis_flag(viewer_handle.opt, "mjVIS_SITE", True)
        set_vis_flag(viewer_handle.opt, "mjVIS_CONTACTPOINT", SHOW_CONTACT_VISUALS)
        set_vis_flag(viewer_handle.opt, "mjVIS_CONTACTFORCE", SHOW_CONTACT_VISUALS)

def maybe_launch_native_viewer(model, data):
    """
    Launch the MuJoCo mesh viewer when possible.

    Official MuJoCo docs note that on macOS `launch_passive` must be executed via
    the special `mjpython` launcher rather than plain `python`.
    """
    if not should_launch_native_viewer():
        return None

    if sys.platform == "darwin" and not running_with_mjpython():
        print(
            "Native MuJoCo viewer not launched because macOS passive viewer requires `mjpython`.\\n"
            "Run this script with:\\n"
            f" /opt/miniconda3/bin/mjpython {SCRIPT_DIR / 'MuJoCo_test.py'}"
        )
        return None

    viewer_handle = mujoco_viewer.launch_passive(
        model,
        data,
        show_left_ui=VIEWER_SHOW_LEFT_UI,
        show_right_ui=VIEWER_SHOW_RIGHT_UI,
    )
    configure_native_viewer_camera(viewer_handle)
    return viewer_handle

def should_attempt_rendering():
    """Attempt offscreen MuJoCo rendering whenever video export is enabled."""
    return RENDER_VIDEO

def save_render_outputs(frames, output_dir):
    if not frames:
        return None, None

    mp4_path = None
    gif_path = None

    if SAVE_RENDER_MP4:
        candidate_path = output_dir / "painting_demo.mp4"
        if imageio_ffmpeg is None:
            print("imageio-ffmpeg is not available, so MP4 export is skipped and GIF export will be used.")
        else:
            try:
                writer = imageio.get_writer(candidate_path, fps=RENDER_FPS, codec="libx264",
                macro_block_size=None)

```

```

        for frame in frames:
            writer.append_data(frame)
        writer.close()
        mp4_path = candidate_path
        print(f"Saved rendered video to: {mp4_path}")
    except Exception as exc:
        print(f"Could not save MP4 render. Falling back to GIF. Reason: {exc}")

```

```

if SAVE_RENDER_GIF:
    gif_path = output_dir / "painting_demo.gif"
    imageio.mimsave(gif_path, frames, fps=RENDER_FPS)
    print(f"Saved rendered GIF to: {gif_path}")
    if IPyImage is not None and display is not None:
        display(IPyImage(filename=str(gif_path)))

return mp4_path, gif_path

```

```

def wall_polygon(surface_x):
    z_min = PAINT_Z_CENTER - WALL_HALF_HEIGHT_Z
    z_max = PAINT_Z_CENTER + WALL_HALF_HEIGHT_Z
    y_min = -WALL_HALF_WIDTH_Y
    y_max = WALL_HALF_WIDTH_Y
    return np.array(
        [
            [surface_x, y_min, z_min],
            [surface_x, y_max, z_min],
            [surface_x, y_max, z_max],
            [surface_x, y_min, z_max],
        ],
        dtype=float,
    )

```

```

def save_robot_3d_animation(log, output_dir):
    """Save a matplotlib 3D animation that shows the robot painting the wall."""
    if not SAVE_ROBOT_3D_ANIMATION or not log["robot_chain_points"]:
        return None, None

    sample_indices = list(range(0, len(log["time"]), ROBOT_ANIMATION_EVERY_N_STEPS))
    if sample_indices[-1] != len(log["time"]) - 1:
        sample_indices.append(len(log["time"]) - 1)

    robot_points = np.asarray(log["robot_chain_points"])
    ee_positions = np.column_stack([log["ee_x"], log["ee_y"], log["ee_z"]])
    time = np.asarray(log["time"])
    force_meas = np.asarray(log["force_meas"])

    fig = plt.figure(figsize=(8, 7))
    ax = fig.add_subplot(111, projection="3d")
    ax.set_title("3D robot animation while painting")
    ax.set_xlabel("x [m]")
    ax.set_ylabel("y [m]")
    ax.set_zlabel("z [m]")
    ax.set_xlim(0.0, 0.80)
    ax.set_ylim(-0.35, 0.35)
    ax.set_zlim(0.0, 0.70)
    ax.view_init(elev=ROBOT_ANIMATION_ELEVATION, azimuth=ROBOT_ANIMATION_AZIMUTH)
    ax.grid(True, alpha=0.3)

    robot_line, = ax.plot([], [], [], "-o", color="tab:blue", linewidth=3, markersize=5,
label="Robot")
    ee_trail_line, = ax.plot([], [], [], color="tab:red", linewidth=1.8, alpha=0.8, label="Brush
trail")
    brush_marker, = ax.plot([], [], [], "o", color="crimson", markersize=7, label="Brush tip")
    wall_artist = {"poly": None}
    status_text = ax.text2D(0.02, 0.95, "", transform=ax.transAxes)
    ax.legend(loc="upper right")

def init():

```

```

robot_line.set_data([], [])
robot_line.set_3d_properties([])
ee_trail_line.set_data([], [])
ee_trail_line.set_3d_properties([])
brush_marker.set_data([], [])
brush_marker.set_3d_properties([])
status_text.set_text("")
return robot_line, ee_trail_line, brush_marker, status_text

```

```

def update(frame_number):
    idx = sample_indices[frame_number]
    points = robot_points[idx]
    trail = ee_positions[: idx + 1]

    robot_line.set_data(points[:, 0], points[:, 1])
    robot_line.set_3d_properties(points[:, 2])

    ee_trail_line.set_data(trail[:, 0], trail[:, 1])
    ee_trail_line.set_3d_properties(trail[:, 2])

    brush_marker.set_data([points[-1, 0]], [points[-1, 1]])
    brush_marker.set_3d_properties([points[-1, 2]])

    if wall_artist["poly"] is not None:
        wall_artist["poly"].remove()
    wall_vertices = [wall_polygon(log["wall_surface_x"][idx])]
    wall_artist["poly"] = Poly3DCollection(
        wall_vertices,
        facecolors=(0.55, 0.75, 0.95, 0.28),
        edgecolors=(0.25, 0.45, 0.75, 0.9),
        linewidths=1.0,
    )
    ax.add_collection3d(wall_artist["poly"])

    status_text.set_text(
        f"t = {time[idx]:.2f} s\n"
        f"F = {force_meas[idx]:.2f} N\n"
        f"x_cmd = {log['x_cmd'][idx]:.3f} m"
    )
    return robot_line, ee_trail_line, brush_marker, status_text, wall_artist["poly"]

```

```

animation_object = animation.FuncAnimation(
    fig,
    update,
    init_func=init,
    frames=len(sample_indices),
    interval=1000 / ROBOT_ANIMATION_FPS,
    blit=False,
)

```

```

gif_path = output_dir / "robot_painting_3d.gif"
animation_object.save(gif_path, writer=animation.PillowWriter(fps=ROBOT_ANIMATION_FPS), dpi=120)
plt.close(fig)

```

```

print(f"Saved matplotlib 3D animation to: {gif_path}")
if IPyImage is not None and display is not None:
    display(IPyImage(filename=str(gif_path)))

```

```

mp4_path = None
if imageio_ffmpeg is not None:
    try:

```

```

        mp4_path = output_dir / "robot_painting_3d.mp4"
        writer = imageio.get_writer(mp4_path, fps=ROBOT_ANIMATION_FPS, codec="libx264",
macro_block_size=None)
        reader = imageio.get_reader(gif_path)
        for frame in reader:
            writer.append_data(np.asarray(frame))
        reader.close()
        writer.close()
        print(f"Saved matplotlib 3D MP4 to: {mp4_path}")

```

```
except Exception as exc:
    print(f"Could not convert the 3D GIF into MP4. Reason: {exc}")
```

```
return gif_path, mp4_path
```

```
def get_output_dir():
    return choose_writable_dir(Script_DIR / "mujoco_painting_outputs", "mujoco_painting_outputs")
```

```
# %% Main simulation
```

```
def run_single_case(
    case_name="single",
    wall_retreat_distance=None,
    wall_profile_mode=None,
    concave_depth=None,
    concave_sigma=None,
    random_wall_x=False,
    wall_x_min=0.64,
    wall_x_max=0.69,
    fixed_wall_x_center=None,
    unknown_wall_preset="stable",
    wall_motion_enabled=True,
):
    global WALL_RETREAT_DISTANCE, WALL_PROFILE_MODE, CONCAVE_DEPTH, CONCAVE_SIGMA, WALL_X_CENTER,
    WALL_MOTION_ENABLED
    original_wall_retreat_distance = WALL_RETREAT_DISTANCE
    original_wall_profile_mode = WALL_PROFILE_MODE
    original_concave_depth = CONCAVE_DEPTH
    original_concave_sigma = CONCAVE_SIGMA
    original_wall_x_center = WALL_X_CENTER
    original_wall_motion_enabled = WALL_MOTION_ENABLED

    if wall_retreat_distance is not None:
        WALL_RETREAT_DISTANCE = float(wall_retreat_distance)
    if wall_profile_mode is not None:
        WALL_PROFILE_MODE = str(wall_profile_mode)
    if concave_depth is not None:
        CONCAVE_DEPTH = float(concave_depth)
    if concave_sigma is not None:
        CONCAVE_SIGMA = float(concave_sigma)
    if fixed_wall_x_center is not None:
        WALL_X_CENTER = float(fixed_wall_x_center)
    elif random_wall_x:
        WALL_X_CENTER = float(np.random.uniform(wall_x_min, wall_x_max))
    WALL_MOTION_ENABLED = bool(wall_motion_enabled)

    control_mode = "unknown_wall"
    unknown_params = UNKNOWN_WALL_PRESETS.get(unknown_wall_preset,
    UNKNOWN_WALL_PRESETS["stable"]).copy()
    contact_threshold = float(unknown_params["contact_threshold"])
    contact_lost_threshold = float(unknown_params["contact_lost_threshold"])
    f_max_safe = float(unknown_params["f_max_safe"])
    v_search = float(unknown_params["v_search"])
    v_recovery = float(unknown_params["v_recovery"])
    k_force = float(unknown_params["k_force"])
    max_dx_per_step = float(unknown_params["max_dx_per_step"])
    max_approach_distance = float(unknown_params["max_approach_distance"])
    max_recovery_distance = float(unknown_params["max_recovery_distance"])
    contact_confirm_steps = int(unknown_params["contact_confirm_steps"])
    contact_lost_confirm_steps = int(unknown_params["contact_lost_confirm_steps"])
    unknown_xcmd_lpf_alpha = float(unknown_params["unknown_xcmd_lpf_alpha"])

    print(f"Python interpreter: {sys.executable}")
    print(f"Script directory: {Script_DIR}")
    print(
        f"Running case: {case_name} | mode={control_mode} | wall_profile={WALL_PROFILE_MODE} "
        f"| wall_motion={'moving' if WALL_MOTION_ENABLED else 'static'} | WALL_X_CENTER="
        f"{WALL_X_CENTER:.4f} "
        f"| unknown_wall_preset={unknown_wall_preset}"
    )
```

```

)

model, model_source = load_model()
data = mujoco.MjData(model)
ik_data = mujoco.MjData(model)

site_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_SITE, EE_SITE_NAME)
if site_id < 0:
    raise ValueError(
        f"Site '{EE_SITE_NAME}' was not found. Update EE_SITE_NAME after swapping in a real robot
model."
    )

wall_body_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_BODY, WALL_BODY_NAME)
if wall_body_id < 0:
    raise ValueError(
        f"Wall body '{WALL_BODY_NAME}' was not found. Keep the moving wall name consistent with
this script."
    )
mocap_id = model.body_mocapid[wall_body_id]
if mocap_id < 0:
    raise ValueError("The wall body is expected to be a mocap body in this demo.")

paint_mark_site_ids = get_paint_mark_site_ids(model)
reset_paint_marks(model, paint_mark_site_ids)

qpos_indices, dof_indices, joint_lower, joint_upper = get_joint_indices(model, JOINT_NAMES)
actuator_indices = get_joint_actuator_indices(model, JOINT_NAMES)
robot_chain_body_ids = get_robot_chain_body_ids(model, JOINT_NAMES)

# Initialize the wall pose.
initial_wall_x, _ = wall_motion(0.0)
data.mocap_pos[mocap_id] = np.array([initial_wall_x, 0.0, PAINT_Z_CENTER])
data.mocap_quat[mocap_id] = np.array([1.0, 0.0, 0.0, 0.0])
ik_data.mocap_pos[mocap_id] = data.mocap_pos[mocap_id].copy()
ik_data.mocap_quat[mocap_id] = data.mocap_quat[mocap_id].copy()

# Use IK to find a reasonable starting joint configuration at the initial painting pose.
q_seed = np.zeros(len(JOINT_NAMES))
initial_target, _ = painting_nominal_pose(0.0)
initial_target[0] = X_NOMINAL_BASE - PEN_UP_CLEARANCE
q_init = ik_damped_least_squares(
    model,
    ik_data,
    site_id,
    initial_target,
    q_seed,
    qpos_indices,
    dof_indices,
    joint_lower,
    joint_upper,
)

data.qpos[qpos_indices] = q_init
data.qvel[:] = 0.0
data.ctrl[:] = 0.0
data.ctrl[actuator_indices] = q_init
mujoco.mj_forward(model, data)

ik_data.qpos[qpos_indices] = q_init
ik_data.qvel[:] = 0.0
ik_data.ctrl[:] = 0.0
ik_data.ctrl[actuator_indices] = q_init
mujoco.mj_forward(model, ik_data)

renderer = None
camera = None
scene_option = None
frames = []
viewer_handle = maybe_launch_native_viewer(model, data)

```

```

if should_attempt_rendering():
    try:
        renderer, camera, scene_option = make_renderer(model)
    except Exception as exc:
        print(f"Renderer could not be created. Continuing without video. Reason: {exc}")

log = {
    "control_mode": control_mode,
    "time": [],
    "wall_x": [],
    "wall_surface_x": [],
    "true_wall_surface_x": [],
    "ee_x": [],
    "ee_y": [],
    "ee_z": [],
    "x_cmd_raw": [],
    "x_cmd": [],
    "force_meas": [],
    "force_manual": [],
    "force_touch": [],
    "force_des": [],
    "force_error": [],
    "controller_state": [],
    "contact_confirm_count": [],
    "contact_lost_count": [],
    "x_search_start": [],
    "x_recovery_start": [],
    "x_contact_est": [],
    "joint_positions": [],
    "joint_commands": [],
    "contact_lost": [],
    "robot_chain_points": [],
    "pen_down": [],
    "penetration": [],
}

q_command = q_init.copy()
sim_duration_budget = effective_sim_duration()
if control_mode == "unknown_wall":
    sim_duration_budget *= UNKNOWN_WALL_PATH_TIME_BUDGET_SCALE
num_steps = int(sim_duration_budget / DT)
next_paint_mark_index = 0
last_paint_mark_local_yz = None
previous_pen_down = False
pen_down_elapsed = 0.0
controller_state = "APPROACH"
contact_confirm_count = 0
contact_lost_count = 0
x_search_start = None
x_recovery_start = None
x_contact_est = None
unknown_safety_warned = False
yz_hold = None
path_time = 0.0
path_time_can_advance = True
initial_ee_x = float(get_ee_pose(model, data, site_id)[0])
x_unknown_min = initial_ee_x - 0.03
x_unknown_max = initial_ee_x + 0.08

try:
    for step in range(num_steps):
        if viewer_handle is not None and not viewer_handle.is_running():
            print("Viewer window was closed early; ending simulation loop.")
            break

        step_wallclock_start = time.time()
        t = step * DT

        wall_x, _ = wall_motion(t)
        data.mocap_pos[mocap_id] = np.array([wall_x, 0.0, PAINT_Z_CENTER])

```

```
data.mocap_quat[mocap_id] = np.array([1.0, 0.0, 0.0, 0.0])
mujoco.mj_forward(model, data)
```

```
contact = contact_force(model, data, site_id, t)
if control_mode == "unknown_wall":
    if step > 0 and path_time_can_advance:
        path_time += DT
    path_time = min(path_time, effective_sim_duration())
    p_nominal, pen_down = painting_nominal_pose(path_time)
else:
    p_nominal, pen_down = painting_nominal_pose(t)
yz_tracking_enabled = True

if pen_down and not previous_pen_down:
    pen_down_elapsed = 0.0
    controller_state = "APPROACH"
    contact_confirm_count = 0
    contact_lost_count = 0
    x_search_start = None
    x_recovery_start = None
    ee_pos_now = get_ee_pose(model, data, site_id)
    yz_hold = np.array([float(ee_pos_now[1]), float(ee_pos_now[2])], dtype=float)

if pen_down:
    force_measured = contact["force_used"]
    force_desired = F_DES
    previous_x_cmd = log["x_cmd"][-1] if len(log["x_cmd"]) > 0 else float(contact["ee_x"])
    ee_vx = float(get_site_linear_velocity(model, data, site_id)[0])

    ee_pos = get_ee_pose(model, data, site_id)
    x_cmd_raw = previous_x_cmd
    force_error = F_DES - force_measured

    if force_measured > f_max_safe:
        controller_state = "SAFETY_STOP"

    if controller_state == "APPROACH":
        if x_search_start is None:
            x_search_start = previous_x_cmd
        x_cmd_raw = previous_x_cmd + v_search * DT
        if (x_cmd_raw - x_search_start) > max_approach_distance:
            controller_state = "SAFETY_STOP"
            x_cmd_raw = previous_x_cmd
        if force_measured > contact_threshold:
            contact_confirm_count += 1
        else:
            contact_confirm_count = 0
        if contact_confirm_count >= contact_confirm_steps:
            controller_state = "PAINTING"
            x_contact_est = float(ee_pos[0])
            x_cmd_raw = previous_x_cmd
            contact_lost_count = 0
    elif controller_state == "PAINTING":
        dx = k_force * force_error * DT
        dx = float(np.clip(dx, -max_dx_per_step, max_dx_per_step))
        x_cmd_raw = previous_x_cmd + dx
        if force_measured < contact_lost_threshold:
            contact_lost_count += 1
        else:
            contact_lost_count = 0
        if contact_lost_count >= contact_lost_confirm_steps:
            controller_state = "RECOVERY"
            x_recovery_start = previous_x_cmd
    elif controller_state == "RECOVERY":
        x_cmd_raw = previous_x_cmd + v_recovery * DT
        if x_recovery_start is None:
            x_recovery_start = previous_x_cmd
        if (x_cmd_raw - x_recovery_start) > max_recovery_distance:
            controller_state = "SAFETY_STOP"
            x_cmd_raw = previous_x_cmd
```

```

        if force_measured > contact_threshold:
            contact_confirm_count += 1
        else:
            contact_confirm_count = 0
        if contact_confirm_count >= contact_confirm_steps:
            controller_state = "PAINTING"
            x_contact_est = float(ee_pos[0])
            x_cmd_raw = previous_x_cmd
            contact_lost_count = 0
    else:
        x_cmd_raw = previous_x_cmd
        if not unknown_safety_warned:
            print("Safety stop: wall not found, contact lost too long, or force exceeded
limit.")

            unknown_safety_warned = True

x_cmd_target = float(np.clip(x_cmd_raw, x_unknown_min, x_unknown_max))
max_step = UNKNOWN_MAX_XCMD_RATE * DT
x_cmd_rate_limited = float(
    np.clip(x_cmd_target, previous_x_cmd - max_step, previous_x_cmd + max_step)
)
x_cmd = float(
    (1.0 - unknown_xcmd_lpf_alpha) * previous_x_cmd
    + unknown_xcmd_lpf_alpha * x_cmd_rate_limited
)
x_cmd = float(np.clip(x_cmd, x_unknown_min, x_unknown_max))
in_contact = force_measured > contact_threshold
yz_tracking_enabled = (controller_state == "PAINTING") and in_contact
if yz_tracking_enabled:
    yz_hold = np.array([float(p_nominal[1]), float(p_nominal[2])], dtype=float)
elif yz_hold is None:
    yz_hold = np.array([float(ee_pos[1]), float(ee_pos[2])], dtype=float)
else:
    force_measured = contact["force_used"]
    in_contact = False
    pen_down_elapsed = 0.0
    force_desired = 0.0
    previous_x_cmd = log["x_cmd"][-1] if len(log["x_cmd"]) > 0 else float(contact["ee_x"])
    x_cmd_raw = float(min(previous_x_cmd, X_NOMINAL_BASE - PEN_UP_CLEARANCE))
    x_cmd_target = float(np.clip(x_cmd_raw, x_unknown_min, x_unknown_max))
    max_step = UNKNOWN_MAX_XCMD_RATE * DT
    x_cmd_rate_limited = float(
        np.clip(x_cmd_target, previous_x_cmd - max_step, previous_x_cmd + max_step)
    )
    x_cmd = float(
        (1.0 - unknown_xcmd_lpf_alpha) * previous_x_cmd
        + unknown_xcmd_lpf_alpha * x_cmd_rate_limited
    )
    x_cmd = float(np.clip(x_cmd, x_unknown_min, x_unknown_max))
    controller_state = "APPROACH"
    contact_confirm_count = 0
    contact_lost_count = 0
    x_search_start = None
    x_recovery_start = None
    force_error = 0.0
    yz_tracking_enabled = False

path_time_can_advance = (not pen_down) or (controller_state in ("PAINTING", "RECOVERY"))

p_command = p_nominal.copy()
p_command[0] = x_cmd
if not yz_tracking_enabled and yz_hold is not None:
    p_command[1] = float(yz_hold[0])
    p_command[2] = float(yz_hold[1])

q_current = data.qpos[qpos_indices].copy()
q_command = ik_damped_least_squares(
    model,
    ik_data,
    site_id,

```

```

        p_command,
        q_current,
        qpos_indices,
        dof_indices,
        joint_lower,
        joint_upper,
    )
    data.ctrl[actuator_indices] = q_command

    if USE_QUASI_STATIC_SERVO:
        data.qpos[qpos_indices] = q_command
        data.qvel[dof_indices] = 0.0
        mujoco.mj_forward(model, data)

    ee_pos = get_ee_pose(model, data, site_id)
    robot_chain_points = get_robot_chain_points(model, data, robot_chain_body_ids, site_id)

    paint_allowed = (
        pen_down
        and in_contact
        and controller_state == "PAINTING"
        and controller_state != "SAFETY_STOP"
        and force_measured > contact_threshold
    )
    if paint_allowed:
        next_paint_mark_index, last_paint_mark_local_yz = maybe_add_paint_mark(
            model,
            paint_mark_site_ids,
            next_paint_mark_index,
            ee_pos,
            last_paint_mark_local_yz,
        )

    log["time"].append(t)
    log["wall_x"].append(contact["wall_x"])
    log["wall_surface_x"].append(contact["wall_surface_x"])
    log["true_wall_surface_x"].append(contact["wall_surface_x"])
    log["ee_x"].append(ee_pos[0])
    log["ee_y"].append(ee_pos[1])
    log["ee_z"].append(ee_pos[2])
    log["x_cmd_raw"].append(x_cmd_raw)
    log["x_cmd"].append(x_cmd)
    log["force_meas"].append(force_measured)
    log["force_manual"].append(contact["force_manual"])
    log["force_touch"].append(contact["force_touch"])
    log["force_des"].append(force_desired)
    log["force_error"].append(force_error)
    log["controller_state"].append(controller_state)
    log["contact_confirm_count"].append(contact_confirm_count)
    log["contact_lost_count"].append(contact_lost_count)
    log["x_search_start"].append(np.nan if x_search_start is None else float(x_search_start))
    log["x_recovery_start"].append(np.nan if x_recovery_start is None else
float(x_recovery_start))
    log["x_contact_est"].append(np.nan if x_contact_est is None else float(x_contact_est))
    log["joint_positions"].append(q_current.copy())
    log["joint_commands"].append(q_command.copy())
    log["contact_lost"].append(0 if in_contact else 1)
    log["robot_chain_points"].append(robot_chain_points)
    log["pen_down"].append(1 if pen_down else 0)
    log["penetration"].append(contact["penetration"])
    if pen_down:
        pen_down_elapsed += DT
    previous_pen_down = pen_down

    if renderer is not None and step % RENDER_EVERY_N_STEPS == 0:
        renderer.update_scene(data, camera=camera, scene_option=scene_option)
        pixels = renderer.render()
        frames.append(np.asarray(pixels).copy())

    if viewer_handle is not None:

```

```
viewer_handle.sync()
```

```
mujoco.mj_step(model, data)
```

```
if viewer_handle is not None and VIEWER_REALTIME:  
    elapsed = time.time() - step_wallclock_start  
    remaining = DT - elapsed  
    if remaining > 0:  
        time.sleep(remaining)
```

```
if viewer_handle is not None and KEEP_VIEWER_OPEN_AFTER_SIM and viewer_handle.is_running():  
    print("Simulation finished. Close the MuJoCo viewer window when you are done inspecting
```

```
it.")
```

```
while viewer_handle.is_running():  
    viewer_handle.sync()  
    time.sleep(0.02)
```

```
finally:
```

```
    WALL_RETREAT_DISTANCE = original_wall_retreat_distance  
    WALL_PROFILE_MODE = original_wall_profile_mode  
    CONCAVE_DEPTH = original_concave_depth  
    CONCAVE_SIGMA = original_concave_sigma  
    WALL_X_CENTER = original_wall_x_center  
    WALL_MOTION_ENABLED = original_wall_motion_enabled  
    if renderer is not None:  
        renderer.close()  
    if viewer_handle is not None:  
        viewer_handle.close()
```

```
output_dir = get_output_dir() / case_name  
output_dir.mkdir(parents=True, exist_ok=True)
```

```
fig_paths = plot_results(log, output_dir)  
mp4_path, gif_path = save_render_outputs(frames, output_dir) if frames else (None, None)  
robot_animation_gif_path, robot_animation_mp4_path = save_robot_3d_animation(log, output_dir)
```

```
sim_time = np.asarray(log["time"])  
wall_x_log = np.asarray(log["wall_x"])  
force_meas = np.asarray(log["force_meas"])  
x_cmd_log = np.asarray(log["x_cmd"])  
ee_x_log = np.asarray(log["ee_x"])  
pen_down_mask = np.asarray(log["pen_down"], dtype=bool)  
pen_down_force = force_meas[pen_down_mask]  
painting_mask = pen_down_mask  
if control_mode == "unknown_wall":  
    painting_mask = pen_down_mask & (np.asarray(log["controller_state"]) == "PAINTING")  
pen_down_contact_lost = np.asarray(log["contact_lost"])[pen_down_mask]  
contact_loss_fraction = float(np.mean(pen_down_contact_lost)) if pen_down_contact_lost.size > 0  
else float("nan")  
mean_force_pen_down = float(np.mean(pen_down_force)) if pen_down_force.size > 0 else float("nan")  
max_force_pen_down = float(np.max(pen_down_force)) if pen_down_force.size > 0 else float("nan")  
mean_abs_force_error_pen_down = (  
    float(np.mean(np.abs(force_meas[painting_mask] - F_DES))) if np.any(painting_mask) else  
    float("nan")  
)  
min_force_pen_down = float(np.min(force_meas[painting_mask])) if np.any(painting_mask) else  
float("nan")  
max_abs_cmd_correction = float(np.max(np.abs(x_cmd_log - ee_x_log))) if x_cmd_log.size > 0 else  
float("nan")
```

```
longest_loss_steps = 0  
current_loss_steps = 0  
for pd, lost in zip(log["pen_down"], log["contact_lost"]):  
    if pd and lost:  
        current_loss_steps += 1  
        longest_loss_steps = max(longest_loss_steps, current_loss_steps)
```

```
    elif pd:  
        current_loss_steps = 0
```

```
longest_contact_loss_duration = longest_loss_steps * DT  
safety_stop_count = int(np.sum(np.asarray(log["controller_state"]) == "SAFETY_STOP"))  
recovery_events = 0
```

```

previous_state = None
for s in log["controller_state"]:
    if s == "RECOVERY" and previous_state != "RECOVERY":
        recovery_events += 1
    previous_state = s
first_contact_indices = np.where((pen_down_mask) & (force_meas > contact_threshold))[0]
time_to_first_contact = float(sim_time[first_contact_indices[0]]) if first_contact_indices.size >
0 else float("nan")

```

```

print("\n=== Simulation summary ===")
print(f"Controller mode: {control_mode}")
print(f"Model source: {model_source}")
print(f"Simulated time: {sim_time[-1]:.2f} s")
print(f"Wall x range: [{np.min(wall_x_log):.3f}, {np.max(wall_x_log):.3f}] m")
print(f"Mean measured force (pen-down): {mean_force_pen_down:.2f} N")
print(f"Max measured force (pen-down): {max_force_pen_down:.2f} N")
print(f"Mean |force error| (pen-down): {mean_abs_force_error_pen_down:.2f} N")
print(f"Minimum measured force (pen-down): {min_force_pen_down:.2f} N")
print(f"Contact-lost fraction (pen-down): {100.0 * contact_loss_fraction:.1f}%")
print(f"Longest continuous contact-loss (pen-down): {longest_contact_loss_duration:.3f} s")
print(f"Number of safety stops: {safety_stop_count}")
print(f"Time to first contact: {time_to_first_contact:.3f} s")
print(f"Number of recovery events: {recovery_events}")
print(f"Max |x_cmd - ee_x| commanded correction: {max_abs_cmd_correction:.4f} m")
print(f"Plots saved to: {output_dir}")
print(f'Painted text: "{PAINT_TEXT}"')

```

```

if mp4_path is not None:
    print(f"Rendered MP4 saved to: {mp4_path}")
if gif_path is not None:
    print(f"Rendered GIF saved to: {gif_path}")
if robot_animation_gif_path is not None:
    print(f"3D robot animation GIF saved to: {robot_animation_gif_path}")
if robot_animation_mp4_path is not None:
    print(f"3D robot animation MP4 saved to: {robot_animation_mp4_path}")

```

```

print("\nInterpretation:")

```

```

print("- This demo uses a one-sided wall motion: the wall first advances toward the robot, then
relaxes back while staying no farther than its starting x position.")
print("- When the wall moves toward the robot, the brush-wall penetration increases, so the
measured contact force rises.")
print("- The selected controller adjusts commanded x to regulate normal contact while painting.")
print("- Unknown-wall mode does not use true wall retreat or surface x for x_cmd control.")
print("- It uses only measured force and EE state to approach, paint, recover, and safety-stop.")

```

```

print("\nSaved figures:")

```

```

for path in fig_paths:
    print(f"- {path}")

```

```

return {
    "case_name": case_name,
    "control_mode": control_mode,
    "contact_lost_pct": 100.0 * contact_loss_fraction,
    "longest_contact_loss_s": longest_contact_loss_duration,
    "mean_force_error_pen_down": mean_abs_force_error_pen_down,
    "min_force_pen_down": min_force_pen_down,
    "safety_stops": safety_stop_count,
    "time_to_first_contact_s": time_to_first_contact,
    "recovery_events": recovery_events,
    "output_dir": str(output_dir),
}

```

```

def parse_cli_args():
    parser = argparse.ArgumentParser(description="MuJoCo painting demo with unknown-wall force
control.")
    parser.add_argument("--case-name", type=str, default="single")
    parser.add_argument(
        "--scenario",
        type=str,
        default="all",
        choices=["all", "flat_static", "bumpy_static", "flat_moving", "bumpy_moving"],
    )

```

```

        help="Choose one scenario or run all four.",
    )
    parser.add_argument("--random-wall-x", action="store_true", help="Randomize true wall x center for each run.")
    parser.add_argument("--wall-x-min", type=float, default=0.64)
    parser.add_argument("--wall-x-max", type=float, default=0.69)
    parser.add_argument(
        "--unknown-wall-preset",
        type=str,
        default="stable",
        choices=list(UNKNOWN_WALL_PRESETS.keys()),
        help="Preset for unknown-wall controller tuning.",
    )
    args, unknown = parser.parse_known_args()
    if unknown:
        print(f"Ignoring unknown CLI args: {unknown}")
    return args

def run_four_scenarios(random_wall_x=False, wall_x_min=0.64, wall_x_max=0.69,
unknown_wall_preset="stable"):
    scenarios = [
        ("flat_static", "rigid", False),
        ("bumpy_static", "bumpy", False),
        ("flat_moving", "rigid", True),
        ("bumpy_moving", "bumpy", True),
    ]
    results = []
    for case_name, wall_profile_mode, wall_motion_enabled in scenarios:
        result = run_single_case(
            case_name=case_name,
            wall_profile_mode=wall_profile_mode,
            wall_motion_enabled=wall_motion_enabled,
            random_wall_x=bool(random_wall_x),
            wall_x_min=float(wall_x_min),
            wall_x_max=float(wall_x_max),
            unknown_wall_preset=str(unknown_wall_preset),
        )
        results.append(result)

    print("\n=== Four Scenario Summary ===")
    for result in results:
        print(
            f"{result['case_name']}: lost={result['contact_lost_pct']:.2f}% | "
            f"mean|err|={result['mean_force_error_pen_down']:.3f} N | "
            f"minF={result['min_force_pen_down']:.3f} N | "
            f"safety_stops={result['safety_stops']} | out={result['output_dir']}"
        )

# %% Standalone 0524 task configuration and controller
from typing import List

def configure_task(base):
    # Use the existing Piper model + wall patch, and set a 15 cm brush.
    base.EXTERNAL_BRUSH_OFFSET = base.np.array([0.0, 0.0, 0.15], dtype=float)
    base.USE_TOUCH_SENSOR = False

    # Moving rigid wall. The MuJoCo mocap body is synchronized in the run loop.
    base.WALL_PROFILE_MODE = "rigid"
    base.WALL_MOTION_ENABLED = True
    base.WALL_PUSH_DISTANCE = 0.008
    base.WALL_RETREAT_DISTANCE = 0.006

    # Contact mechanics assumptions (brush + wall).
    base.CONTACT_STIFFNESS = 3000.0 # N/m
    base.CONTACT_DAMPING = 60.0 # N*s/m
    base.F_DES = 10.0 # N

    # UCLA painting path.
    base.PAINT_TEXT = "UCLA"

```

```

base.LETTER_WIDTH = 0.055
base.LETTER_HEIGHT = 0.095
base.LETTER_SPACING = 0.015
base.LETTER_STROKE_SPEED = 0.040
base.CA_STROKE_SPEED_SCALE = 0.50
base.PEN_UP_TRAVEL_SPEED = 0.18
base.PRE_STROKE_SETTLE_TIME = 0.20
base.CORNER_HOLD_C_TIME = 0.08
base.CORNER_HOLD_A_TIME = 0.08
base.SIM_DURATION = 20.0
base.PAINT_START_DELAY = 0.8
base.PAINT_END_HOLD = 1.0
# Make text readable from the viewer/user side in your current camera setup.
base.TEXT_READABLE_FROM_ROBOT_SIDE = True
# Keep base plan for compatibility; runtime uses a local strict plan in this file.
base.PAINT_PLAN = base.build_paint_plan()
# Improve Cartesian tracking fidelity.
base.IK_MAX_ITERS = 60
base.IK_DAMPING = 1.5e-3

```

```

def build_local_ucla_plan(base):
    w = base.LETTER_WIDTH
    h = base.LETTER_HEIGHT
    s = base.LETTER_SPACING
    total_width = 4 * w + 3 * s
    x0 = -0.5 * total_width
    z_bottom = -0.5 * h
    z_top = 0.5 * h
    z_mid = 0.02 * h

    settle_before_stroke_s = float(getattr(base, "PRE_STROKE_SETTLE_TIME", 0.40))
    corner_hold_c_s = float(getattr(base, "CORNER_HOLD_C_TIME", 0.14))
    corner_hold_a_s = float(getattr(base, "CORNER_HOLD_A_TIME", 0.14))
    c_a_speed_scale = float(getattr(base, "CA_STROKE_SPEED_SCALE", 0.35))
    pen_up_clearance = float(getattr(base, "PEN_UP_Z_OFFSET", 0.03))
    base.PEN_UP_Z_OFFSET = pen_up_clearance

    def letter_origin(letter_index):
        return x0 + letter_index * (w + s)

    # Each stroke spec: points, speed_scale, optional corner_holds(after stroke end)
    letters: List[List[object]] = []

    u0 = letter_origin(0)
    letters.append(
        [
            {
                "points": base.numpy.array([[u0, z_top], [u0, z_bottom], [u0 + w, z_bottom], [u0 + w,
z_top]], dtype=float),
                "speed_scale": 1.0,
                "corner_holds": [],
            }
        ]
    )

    c0 = letter_origin(1)
    letters.append(
        [
            {
                # C one-stroke path: top-right -> top-left -> down-left -> bottom-right
                "points": base.numpy.array(
                    [
                        [c0 + w, z_top],
                        [c0 + 0.65 * w, z_top],
                        [c0 + 0.30 * w, z_top],
                        [c0, z_top],
                        [c0, 0.50 * z_top],
                        [c0, 0.0],
                        [c0, 0.50 * z_bottom],
                        [c0, z_bottom],
                    ]
                )
            }
        ]
    )

```

```

        [c0 + 0.30 * w, z_bottom],
        [c0 + 0.65 * w, z_bottom],
        [c0 + w, z_bottom],
    ],
    dtype=float,
),
"speed_scale": c_a_speed_scale,
"corner_holds": [corner_hold_c_s, corner_hold_c_s],
},
]
)

l0 = letter_origin(2)
letters.append(
    [
        {
            "points": base.numpy.array([[l0, z_top], [l0, z_bottom], [l0 + w, z_bottom]]),
dtype=float),
            "speed_scale": 1.0,
            "corner_holds": [],
        }
    ]
)

a0 = letter_origin(3)
letters.append(
    [
        {
            # A one-stroke path: left leg up -> right leg down -> move to crossbar and draw
            "points": base.numpy.array(
                [
                    [a0, z_bottom],
                    [a0 + 0.25 * w, 0.5 * z_top],
                    [a0 + 0.5 * w, z_top],
                    [a0 + 0.75 * w, 0.5 * z_top],
                    [a0 + w, z_bottom],
                    [a0 + 0.75 * w, z_mid],
                    [a0 + 0.25 * w, z_mid],
                ]
            ),
            dtype=float,
        },
        {
            "speed_scale": c_a_speed_scale,
            "corner_holds": [corner_hold_a_s],
        }
    ]
)

segments = []
cursor = None
for letter_idx, letter_strokes in enumerate(letters):
    first_stroke = letter_strokes[0]["points"]
    if cursor is not None and base.numpy.linalg.norm(first_stroke[0] - cursor) > 1e-9:
        safe_z = max(float(cursor[1]), float(first_stroke[0, 1])) + pen_up_clearance
        lift_point = base.numpy.array([cursor[0], safe_z], dtype=float)
        travel_point = base.numpy.array([first_stroke[0, 0], safe_z], dtype=float)
        descend_point = first_stroke[0].copy()

        segments.append({"points": base.numpy.vstack([cursor, lift_point]), "pen_down": False,
"speed": base.PEN_UP_TRAVEL_SPEED, "letter_idx": letter_idx})
        segments.append({"points": base.numpy.vstack([lift_point, travel_point]), "pen_down": False,
"speed": base.PEN_UP_TRAVEL_SPEED, "letter_idx": letter_idx})
        segments.append({"points": base.numpy.vstack([travel_point, descend_point]), "pen_down":
False, "speed": base.PEN_UP_TRAVEL_SPEED, "letter_idx": letter_idx})
        cursor = descend_point.copy()

    for stroke_spec in letter_strokes:
        stroke = stroke_spec["points"]
        stroke_speed = base.LETTER_STROKE_SPEED * float(stroke_spec.get("speed_scale", 1.0))
        if cursor is not None and base.numpy.linalg.norm(stroke[0] - cursor) > 1e-9:
            segments.append({"points": base.numpy.vstack([cursor, stroke[0]]), "pen_down": False,

```

```

speed": base.PEN_UP_TRAVEL_SPEED, "letter_idx": letter_idx})

# Pre-stroke settle hold (pen-up) so controller can converge before contact.
segments.append(
    {
        "points": base.np.vstack([stroke[0], stroke[0]]),
        "pen_down": False,
        "speed": base.PEN_UP_TRAVEL_SPEED,
        "duration_override": settle_before_stroke_s,
        "letter_idx": letter_idx,
    }
)

# Actual writing segment.
segments.append({"points": stroke, "pen_down": True, "speed": stroke_speed, "letter_idx":
letter_idx})

# Optional short corner hold(s) for sharp turns.
for hold_dt in stroke_spec.get("corner_holds", []):
    segments.append(
        {
            "points": base.np.vstack([stroke[-1], stroke[-1]]),
            "pen_down": True,
            "speed": stroke_speed,
            "duration_override": float(hold_dt),
            "letter_idx": letter_idx,
        }
    )
    cursor = stroke[-1].copy()

cumulative_time = base.PAINT_START_DELAY
timed = []
for segment in segments:
    length = base.polyline_length(segment["points"])
    if "duration_override" in segment:
        duration = float(segment["duration_override"])
    else:
        duration = 0.0 if length < 1e-9 else length / segment["speed"]
    timed.append(
        {
            "points": segment["points"],
            "pen_down": segment["pen_down"],
            "speed": segment["speed"],
            "letter_idx": int(segment.get("letter_idx", -1)),
            "t_start": cumulative_time,
            "t_end": cumulative_time + duration,
            "length": length,
        }
    )
    cumulative_time += duration

def local_to_world(local_yz):
    y = base.PAINT_Y_CENTER - local_yz[0] if base.TEXT_READABLE_FROM_ROBOT_SIDE else
base.PAINT_Y_CENTER + local_yz[0]
    return base.np.array([base.X_NOMINAL_BASE, y, base.PAINT_Z_CENTER + local_yz[1]], dtype=float)

total_duration = cumulative_time + base.PAINT_END_HOLD
first_local_point = letters[0][0]["points"][0]
last_local_point = letters[-1][-1]["points"][-1]
local_plan = {
    "segments": timed,
    "total_duration": total_duration,
    "start_point": local_to_world(first_local_point),
    "end_point": local_to_world(last_local_point),
}
if bool(getattr(base, "PLAN_DEBUG_PRINT", True)):
    print("\n=== Local UCLA Plan Segments ===")
    for i, seg in enumerate(local_plan["segments"]):
        p0 = seg["points"][0]
        p1 = seg["points"][-1]

```

```

        print(
            f"[{i:02d}] letter={seg['letter_idx']} pen_down={seg['pen_down']} "
            f"t=[{seg['t_start']:.3f}, {seg['t_end']:.3f}] "
            f"start=({p0[0]:.4f},{p0[1]:.4f}) end=({p1[0]:.4f},{p1[1]:.4f})"
        )
    return local_plan

def painting_nominal_pose_local(base, plan, t):
    if t <= base.PAINT_START_DELAY:
        return plan["start_point"].copy(), False, -1

    for segment in plan["segments"]:
        if segment["t_start"] <= t <= segment["t_end"] or (abs(t - segment["t_end"]) < 1e-9 and
segment["t_end"] >= segment["t_start"]):
            distance = segment["speed"] * max(0.0, t - segment["t_start"])
            local_point_yz = base.interpolate_polyline(segment["points"], distance)
            world_y = base.PAINT_Y_CENTER - local_point_yz[0] if base.TEXT_READABLE_FROM_ROBOT_SIDE
else base.PAINT_Y_CENTER + local_point_yz[0]
            return (
                base.numpy.array([base.X_NOMINAL_BASE, world_y, base.PAINT_Z_CENTER + local_point_yz[1]],
dtype=float),
                segment["pen_down"],
                int(segment.get("letter_idx", -1)),
            )

    return plan["end_point"].copy(), False, -1

def tool_axis_error(base, current_rotation, target_tool_axis):
    current_tool_axis = current_rotation[:, 2]
    return base.numpy.cross(current_tool_axis, target_tool_axis)

def ik_pose_with_posture(
    base,
    model,
    ik_data,
    site_id,
    target_pos,
    target_tool_axis,
    q_seed,
    q_nominal,
    qpos_idx,
    dof_idx,
    joint_lower,
    joint_upper,
):
    q_trial = q_seed.copy()
    orientation_length_scale = 0.05
    posture_gain = 0.01
    posture_weights = base.numpy.array(
        [0.05, 0.05, 0.05, 0.25, 0.50, 1.00],
        dtype=float,
    )

    for _ in range(base.IK_MAX_ITERS):
        ik_data.qpos[qpos_idx] = q_trial
        ik_data.qvel[:] = 0.0
        base.mujoco.mj_forward(model, ik_data)

        current_pos = ik_data.site_xpos[site_id].copy()
        current_rotation = ik_data.site_xmat[site_id].reshape(3, 3).copy()
        position_error = target_pos - current_pos
        orientation_error = tool_axis_error(
            base,
            current_rotation,
            target_tool_axis,
        )
        if (
            base.numpy.linalg.norm(position_error) < base.IK_TOL
            and base.numpy.linalg.norm(orientation_error) < 2.0e-3
        ):

```

break

```
jacp = base.np.zeros((3, model.nv))
jacr = base.np.zeros((3, model.nv))
base.mujoco.mj_jacSite(model, ik_data, jacp, jacr, site_id)
jacobian = base.np.vstack(
    [
        jacp[:, dof_idx],
        orientation_length_scale * jacr[:, dof_idx],
    ]
)
task_error = base.np.concatenate(
    [
        position_error,
        orientation_length_scale * orientation_error,
    ]
)

regularized = (
    jacobian @ jacobian.T
    + (base.IK_DAMPING**2) * base.np.eye(6)
)
jacobian_pinv = jacobian.T @ base.np.linalg.solve(
    regularized,
    base.np.eye(6),
)
dq_primary = jacobian_pinv @ task_error
nullspace = base.np.eye(len(q_trial)) - jacobian_pinv @ jacobian
posture_step = posture_weights * (q_nominal - q_trial)
dq = dq_primary + posture_gain * (nullspace @ posture_step)
dq = base.np.clip(dq, -base.IK_STEP_LIMIT, base.IK_STEP_LIMIT)
q_trial = base.np.clip(q_trial + dq, joint_lower, joint_upper)
```

return q_trial

```
def run_impedance_case(base):
    model, model_source = base.load_model()
    data = base.mujoco.MjData(model)
    ik_data = base.mujoco.MjData(model)

    site_id = base.mujoco.mj_name2id(model, base.mujoco.mjtObj.mjOBJ_SITE, base.EE_SITE_NAME)
    if site_id < 0:
        raise RuntimeError(f"Site '{base.EE_SITE_NAME}' not found in model.")

    wall_body_id = base.mujoco.mj_name2id(
        model,
        base.mujoco.mjtObj.mjOBJ_BODY,
        base.WALL_BODY_NAME,
    )
    if wall_body_id < 0:
        raise RuntimeError(f"Wall body '{base.WALL_BODY_NAME}' not found in model.")
    wall_mocap_id = int(model.body_mocapid[wall_body_id])
    if wall_mocap_id < 0:
        raise RuntimeError("The wall body must be a MuJoCo mocap body.")

    initial_wall_x, _ = base.wall_motion(0.0)
    initial_wall_pos = base.np.array(
        [initial_wall_x, 0.0, base.PAINT_Z_CENTER],
        dtype=float,
    )
    initial_wall_quat = base.np.array([1.0, 0.0, 0.0, 0.0], dtype=float)
    data.mocap_pos[wall_mocap_id] = initial_wall_pos
    data.mocap_quat[wall_mocap_id] = initial_wall_quat
    ik_data.mocap_pos[wall_mocap_id] = initial_wall_pos
    ik_data.mocap_quat[wall_mocap_id] = initial_wall_quat

    qpos_idx, dof_idx, joint_lower, joint_upper = base.get_joint_indices(model, base.JOINT_NAMES)
    act_idx = base.get_joint_actuator_indices(model, base.JOINT_NAMES)

    # Cartesian normal-direction impedance params (stabilized for position-actuated robot).
```

```

M_imp = 1.6
B_imp = 145.0
K_imp = 560.0
pen_up_clearance = 0.006
ex_limit = 0.015
v_limit = 0.12
x_cmd_rate_limit = 0.08 # m/s
force_lpf_alpha = 0.25
yz_track_kp = 0.65
yz_corr_limit = 0.020
yz_track_kp_c = 1.20
yz_corr_limit_c = 0.032
q_step_limit = base.numpy.array([0.03, 0.03, 0.03, 0.04, 0.04, 0.025], dtype=float)

# Initial pose.
q_seed = base.numpy.zeros(len(base.JOINT_NAMES))
local_plan = build_local_ucla_plan(base)
p0, _, _ = painting_nominal_pose_local(base, local_plan, 0.0)
q_init = base.ik_damped_least_squares(
    model,
    ik_data,
    site_id,
    p0,
    q_seed,
    qpos_idx,
    dof_idx,
    joint_lower,
    joint_upper,
)
data.qpos[qpos_idx] = q_init
data.qvel[:] = 0.0
base.mujoco.mj_forward(model, data)
target_ee_rotation = data.site_xmat[site_id].reshape(3, 3).copy()
target_tool_axis = target_ee_rotation[:, 2].copy()
paint_mark_site_ids = base.get_paint_mark_site_ids(model)
base.reset_paint_marks(model, paint_mark_site_ids)
next_paint_mark_index = 0
last_paint_mark_local_yz = None
q_ref = q_init.copy()

sim_time = max(base.SIM_DURATION, float(local_plan["total_duration"]))
steps = int(sim_time / base.DT)
frames = []
renderer = None
camera = None
scene_option = None
if base.should_attempt_rendering():
    renderer, camera, scene_option = base.make_renderer(model)

x_imp = float(data.site_xpos[site_id][0])
x_imp_dot = 0.0
desired_penetration = base.F_DES / base.CONTACT_STIFFNESS
fixed_contact_equilibrium = (
    base.WALL_X_CENTER
    - base.WALL_HALF_THICKNESS
    - base.BRUSH_RADIUS
    + desired_penetration
)
x_ref_prev = x_imp
x_cmd_prev = x_imp
f_meas_filt = 0.0
log_t, log_f, log_fdes = [], [], []
log_xcmd, log_xee, log_xref = [], [], []
log_wall_surface, log_contact_gap = [], []
log_y_nom, log_z_nom, log_y_ee, log_z_ee = [], [], [], []
log_pen_down = []
log_orientation_error = []
log_q = []

for i in range(steps):

```

```

t = i * base.DT

wall_x, _ = base.wall_motion(t)
data.mocap_pos[wall_mocap_id] = base.numpy.array(
    [wall_x, 0.0, base.PAINT_Z_CENTER],
    dtype=float,
)
data.mocap_quat[wall_mocap_id] = initial_wall_quat
base.mujoco.mj_forward(model, data)

nominal_pos, pen_down, letter_idx = painting_nominal_pose_local(base, local_plan, t)
ee_pos = base.get_ee_pose(model, data, site_id)
contact = base.contact_force(model, data, site_id, t)

wall_surface_x = contact["wall_surface_x"]
x_ref = (
    fixed_contact_equilibrium
    if pen_down
    else fixed_contact_equilibrium - pen_up_clearance
)

f_des = base.F_DES if pen_down else 0.0
f_meas_raw = float(contact["force_used"])
if i == 0:
    f_meas_filt = f_meas_raw
else:
    f_meas_filt = force_lpf_alpha * f_meas_raw + (1.0 - force_lpf_alpha) * f_meas_filt
f_error = f_des - f_meas_filt

# True impedance dynamics in wall-normal direction:
#  $M\ddot{e} + B\dot{e} + Ke = F_{des} - F_{meas}$ 
x_ref_dot = 0.0 if i == 0 else float((x_ref - x_ref_prev) / base.DT)
e = x_imp - x_ref
e_dot = x_imp_dot - x_ref_dot
e_ddot = (f_error - B_imp * e_dot - K_imp * e) / M_imp

x_imp_dot = float(base.numpy.clip(x_imp_dot + e_ddot * base.DT, -v_limit, v_limit))
x_imp = float(x_imp + x_imp_dot * base.DT)
x_imp = float(base.numpy.clip(x_imp, x_ref - ex_limit, x_ref + ex_limit))

x_cmd_target = x_imp
max_step = x_cmd_rate_limit * base.DT
x_cmd = float(base.numpy.clip(x_cmd_target, x_cmd_prev - max_step, x_cmd_prev + max_step))
x_ref_prev = x_ref
x_cmd_prev = x_cmd
target = nominal_pos.copy()
target[0] = x_cmd
# Shape-first correction: keep y-z close to writing trajectory while x follows impedance.
yz_err = nominal_pos[1:3] - ee_pos[1:3]
if base.USE_QUASI_STATIC_SERVO:
    yz_corr = base.numpy.zeros(2, dtype=float)
elif pen_down and letter_idx == 1:
    yz_corr = base.numpy.clip(yz_track_kp_c * yz_err, -yz_corr_limit_c, yz_corr_limit_c)
else:
    yz_corr = base.numpy.clip(yz_track_kp * yz_err, -yz_corr_limit, yz_corr_limit)
target[1:3] = target[1:3] + yz_corr

q_curr = data.qpos[qpos_idx].copy()
q_cmd = ik_pose_with_posture(
    base,
    model,
    ik_data,
    site_id,
    target,
    target_tool_axis,
    q_curr,
    q_ref,
    qpos_idx,
    dof_idx,
    joint_lower,

```

```

        joint_upper,
    )
    q_cmd = q_curr + base.np.clip(q_cmd - q_curr, -q_step_limit, q_step_limit)
    q_cmd[-1] = q_ref[-1]
    data.ctrl[act_idx] = q_cmd
    if base.USE_QUASI_STATIC_SERVO:
        data.qpos[qpos_idx] = q_cmd
        data.qvel[dof_idx] = 0.0
        base.mujoco.mj_forward(model, data)

    log_t.append(t)
    log_f.append(f_meas_filt)
    log_fdes.append(f_des)
    log_xcmd.append(x_cmd)
    log_xee.append(float(ee_pos[0]))
    log_xref.append(x_ref)
    log_wall_surface.append(float(wall_surface_x))
    log_contact_gap.append(
        float(wall_surface_x - (ee_pos[0] + base.BRUSH_RADIUS))
    )
    log_y_nom.append(float(nominal_pos[1]))
    log_z_nom.append(float(nominal_pos[2]))
    log_y_ee.append(float(ee_pos[1]))
    log_z_ee.append(float(ee_pos[2]))
    log_pen_down.append(bool(pen_down))
    current_rotation = data.site_xmat[site_id].reshape(3, 3)
    log_orientation_error.append(
        float(base.np.linalg.norm(tool_axis_error(base, current_rotation, target_tool_axis)))
    )
    log_q.append(data.qpos[qpos_idx].copy())

    if renderer is not None and (i % base.RENDER_EVERY_N_STEPS == 0):
        renderer.update_scene(data, camera=camera, scene_option=scene_option)
        pixels = renderer.render()
        frames.append(base.np.asarray(pixels).copy())

    base.mujoco.mj_step(model, data)
    if pen_down:
        ee_pos_mark = base.get_ee_pose(model, data, site_id)
        next_paint_mark_index, last_paint_mark_local_yz = base.maybe_add_paint_mark(
            model,
            paint_mark_site_ids,
            next_paint_mark_index,
            ee_pos_mark,
            last_paint_mark_local_yz,
        )

    out_dir = base.get_output_dir() / "0524_force_impedance"
    out_dir.mkdir(parents=True, exist_ok=True)

    base.plt.figure(figsize=(10, 7))
    ax1 = base.plt.subplot(2, 1, 1)
    ax1.plot(log_t, log_f, label="measured force")
    ax1.plot(log_t, log_fdes, "--", label="desired force")
    ax1.set_ylabel("Force [N]")
    ax1.grid(True, alpha=0.3)
    ax1.legend()

    ax2 = base.plt.subplot(2, 1, 2)
    ax2.plot(log_t, log_xcmd, label="x_cmd")
    ax2.plot(log_t, log_xee, label="ee_x")
    ax2.plot(log_t, log_xref, "--", label="fixed equilibrium")
    ax2.plot(log_t, log_wall_surface, ":", label="wall surface")
    ax2.set_xlabel("Time [s]")
    ax2.set_ylabel("X [m]")
    ax2.grid(True, alpha=0.3)
    ax2.legend()
    base.plt.tight_layout()
    fig_path = out_dir / "impedance_response.png"
    base.plt.savefig(fig_path, dpi=150)

```

```
base=plt.close()
```

```
retreat_start = base.WALL_MOTION_START + base.WALL_PUSH_DURATION
```

```
retreat_end = retreat_start + base.WALL_RETURN_DURATION
```

```
retreat_mask = (  
    (base.numpy.asarray(log_t) >= retreat_start)  
    & (base.numpy.asarray(log_t) <= retreat_end)  
)
```

```
retreat_wall = base.numpy.asarray(log_wall_surface)[retreat_mask]
```

```
retreat_ee = base.numpy.asarray(log_xee)[retreat_mask]
```

```
retreat_force = base.numpy.asarray(log_f)[retreat_mask]
```

```
retreat_gap = base.numpy.asarray(log_contact_gap)[retreat_mask]
```

```
fig_follow = base.pyplot.figure(figsize=(10, 7))
```

```
follow_x = fig_follow.add_subplot(2, 1, 1)
```

```
follow_x.plot(log_t, log_wall_surface, label="wall surface")
```

```
follow_x.plot(  
    log_t,  
    base.numpy.asarray(log_xee) + base.BRUSH_RADIUS,  
    label="brush front",  
)
```

```
follow_x.set_ylabel("X [m]")
```

```
follow_x.grid(True, alpha=0.3)
```

```
follow_x.legend()
```

```
follow_gap = fig_follow.add_subplot(2, 1, 2)
```

```
follow_gap.plot(log_t, 1000.0 * base.numpy.asarray(log_contact_gap), label="contact gap")
```

```
follow_gap.axhline(0.0, color="black", linestyle="--", linewidth=1.0)
```

```
follow_gap.set_xlabel("Time [s]")
```

```
follow_gap.set_ylabel("Gap [mm]")
```

```
follow_gap.grid(True, alpha=0.3)
```

```
follow_gap.legend()
```

```
fig_follow.tight_layout()
```

```
follow_path = out_dir / "wall_retreat_following.png"
```

```
fig_follow.savefig(follow_path, dpi=160)
```

```
base.pyplot.close(fig_follow)
```

```
# Trajectory accuracy diagnostics in wall tangential plane (y-z).
```

```
t_arr = base.numpy.asarray(log_t)
```

```
y_nom = base.numpy.asarray(log_y_nom)
```

```
z_nom = base.numpy.asarray(log_z_nom)
```

```
y_ee = base.numpy.asarray(log_y_ee)
```

```
z_ee = base.numpy.asarray(log_z_ee)
```

```
pen_down_arr = base.numpy.asarray(log_pen_down, dtype=bool)
```

```
yz_err = base.numpy.sqrt((y_ee - y_nom) ** 2 + (z_ee - z_nom) ** 2)
```

```
yz_err_mm = 1000.0 * yz_err
```

```
if base.numpy.any(pen_down_arr):
```

```
    yz_err_pen_mm = yz_err_mm[pen_down_arr]
```

```
    mean_err_mm = float(base.numpy.mean(yz_err_pen_mm))
```

```
    max_err_mm = float(base.numpy.max(yz_err_pen_mm))
```

```
else:
```

```
    mean_err_mm = float("nan")
```

```
    max_err_mm = float("nan")
```

```
fig_err = base.pyplot.figure(figsize=(11, 5))
```

```
ax_traj = fig_err.add_subplot(1, 2, 1)
```

```
ax_traj.plot(y_nom, z_nom, "--", linewidth=2.0, label="Nominal YZ path")
```

```
ax_traj.plot(y_ee, z_ee, linewidth=2.0, label="Actual YZ path")
```

```
ax_traj.set_xlabel("y [m]")
```

```
ax_traj.set_ylabel("z [m]")
```

```
ax_traj.set_title("Path Shape: Nominal vs Actual")
```

```
ax_traj.grid(True, alpha=0.3)
```

```
ax_traj.legend()
```

```
ax_traj.axis("equal")
```

```
ax_err = fig_err.add_subplot(1, 2, 2)
```

```
ax_err.plot(t_arr, yz_err_mm, linewidth=2.0, label="YZ tracking error")
```

```
if base.numpy.any(pen_down_arr):
```

```
    ax_err.plot(t_arr[pen_down_arr], yz_err_mm[pen_down_arr], ".", markersize=2.0, label="Pen-down
```

```

samples")
ax_err.set_xlabel("time [s]")
ax_err.set_ylabel("error [mm]")
ax_err.set_title("Tangential Tracking Error")
ax_err.grid(True, alpha=0.3)
ax_err.legend()
fig_err.tight_layout()
fig_err_path = out_dir / "yz_tracking_error.png"
fig_err.savefig(fig_err_path, dpi=160)
base.plt.close(fig_err)
mp4_path, gif_path = (None, None)
if frames:
    mp4_path, gif_path = base.save_render_outputs(frames, out_dir)
if renderer is not None:
    renderer.close()

print("=== True Impedance Control Run Complete ===")
print(f"Model source: {model_source}")
print("Controller:  $M\ddot{e} + B\dot{e} + Ke = F_{des} - F_{meas}$ ")
print(f"Output dir: {out_dir}")
print(f"Figure: {fig_path}")
print(f"Wall following figure: {follow_path}")
print(f"YZ error figure: {fig_err_path}")
print(f"Mean YZ error (pen-down): {mean_err_mm:.2f} mm")
print(f"Max YZ error (pen-down): {max_err_mm:.2f} mm")
print(
    "Mean tool-axis drift: "
    f"{base.np.degrees(base.np.mean(log_orientation_error)):.2f} deg"
)
print(
    "Max tool-axis drift: "
    f"{base.np.degrees(base.np.max(log_orientation_error)):.2f} deg"
)
q_range_deg = base.np.degrees(base.np.ptp(base.np.asarray(log_q), axis=0))
print(
    "Wrist joint ranges (q4/q5/q6): "
    f"{q_range_deg[3]:.2f} / {q_range_deg[4]:.2f} / {q_range_deg[5]:.2f} deg"
)
metrics = {
    "planned_duration_s": float(local_plan["total_duration"]),
    "simulation_duration_s": float(sim_time),
    "video_fps": int(base.RENDER_FPS),
    "desired_force_n": float(base.F_DES),
    "mean_yz_error_pen_down_mm": mean_err_mm,
    "max_yz_error_pen_down_mm": max_err_mm,
    "mean_tool_axis_drift_deg": float(
        base.np.degrees(base.np.mean(log_orientation_error))
    ),
    "max_tool_axis_drift_deg": float(
        base.np.degrees(base.np.max(log_orientation_error))
    ),
    "wrist_joint_ranges_deg": {
        "q4": float(q_range_deg[3]),
        "q5": float(q_range_deg[4]),
        "q6": float(q_range_deg[5]),
    },
}
force_arr = base.np.asarray(log_f)
force_des_arr = base.np.asarray(log_fdes)
if base.np.any(pen_down_arr):
    force_error_pen = force_arr[pen_down_arr] - force_des_arr[pen_down_arr]
    metrics.update(
        {
            "mean_force_pen_down_n": float(base.np.mean(force_arr[pen_down_arr])),
            "force_mae_pen_down_n": float(base.np.mean(base.np.abs(force_error_pen))),
            "force_rmse_pen_down_n": float(
                base.np.sqrt(base.np.mean(force_error_pen**2))
            ),
            "force_std_pen_down_n": float(base.np.std(force_arr[pen_down_arr])),
            "peak_force_pen_down_n": float(base.np.max(force_arr[pen_down_arr])),
        }
    )

```

```

        "minimum_force_pen_down_n": float(base.np.min(force_arr[pen_down_arr])),
    }
)

if retreat_wall.size > 1:
    wall_retreat_mm = 1000.0 * float(retreat_wall[-1] - retreat_wall[0])
    brush_follow_mm = 1000.0 * float(retreat_ee[-1] - retreat_ee[0])
    contact_ratio = 100.0 * float(base.np.mean(retreat_gap <= 0.0))
    mean_retreat_force = float(base.np.mean(retreat_force))
    follow_ratio = 100.0 * brush_follow_mm / wall_retreat_mm
    metrics.update(
        {
            "wall_retreat_mm": wall_retreat_mm,
            "brush_follow_mm": brush_follow_mm,
            "brush_follow_ratio_pct": follow_ratio,
            "retreat_contact_maintained_pct": contact_ratio,
            "mean_retreat_force_n": mean_retreat_force,
            "retreat_force_rmse_n": float(
                base.np.sqrt(base.np.mean((retreat_force - base.F_DES) ** 2))
            ),
            "maximum_retreat_gap_mm": float(1000.0 * base.np.max(retreat_gap)),
            "minimum_retreat_gap_mm": float(1000.0 * base.np.min(retreat_gap)),
        }
    )
    print(f"Wall retreat distance: {wall_retreat_mm:.2f} mm")
    print(f"Brush follow distance: {brush_follow_mm:.2f} mm")
    print(f"Brush follow ratio: {follow_ratio:.1f}%")
    print(f"Retreat contact maintained: {contact_ratio:.1f}%")
    print(f"Mean retreat force: {mean_retreat_force:.2f} N")
summary_path = out_dir / "performance_summary.json"
summary_path.write_text(
    json.dumps(metrics, indent=2, ensure_ascii=False),
    encoding="utf-8",
)
print(f"Performance summary: {summary_path}")
if mp4_path is not None:
    print(f"MP4: {mp4_path}")
if gif_path is not None:
    print(f"GIF: {gif_path}")

def main():
    base = sys.modules[__name__]
    configure_task(base)
    run_impedance_case(base)

if __name__ == "__main__":
    main()

```

RoboPainting

Final Presentation

(Built on indirect force control)

Group 13
Ruohan He
Shixuan Sun
Jiawen Wang
Xiaoyang Zhao



Overview



GOAL:

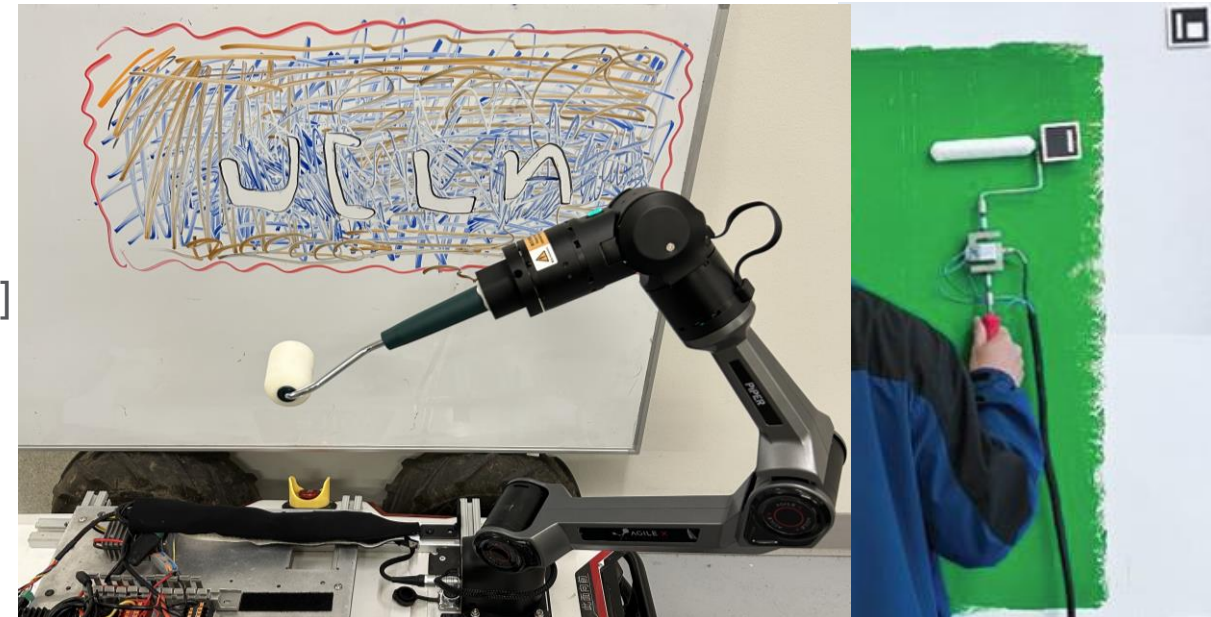
Stable brush–wall contact for robot painting

MOTIVATION:

- Contact-rich manipulation often requires force control [1]
- Contact force directly affects painting quality and safety. [2]
- Adaptive control is needed to handle surface variations [3]

OBJECTIVES OF CONTROLLER DESIGN :

Maintain stable brush–wall contact through real-time adaptive control



[1] M. Suomalainen, Y. Karayiannidis, and V. Kyrki, “A survey of robot manipulation in contact,” *Robotics and Autonomous Systems*, vol. 156, Art. no. 104224, 2022, doi: 10.1016/j.robot.2022.104224.

[2] K. Bansal, Y. Du, E. Palan, M. Quadir, and M. K. Jawed, “Robotic painting: mimicking human applicators,” *Journal of Coatings Technology and Research*, vol. 20, pp. 1369–1381, 2023, doi: 10.1007/s11998-022-00750-7.

[3] F. J. Abu-Dakka and M. Saveriano, “Variable impedance control and learning—A review,” *Frontiers in Robotics and AI*, vol. 7, Art. no. 590681, 2020, doi: 10.3389/frobt.2020.590681.



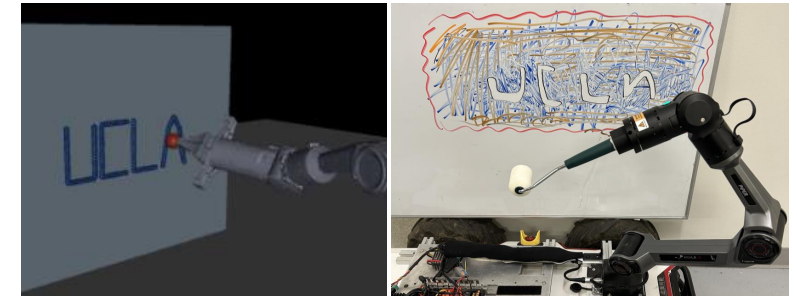
Methods and Control Architecture

SIMULATION:

$$M\ddot{e} + B\dot{e} + Ke = F_{\text{des}} - F_{\text{meas}}$$

$$e = x_{\text{imp}} - x_{\text{ref}} \quad e_F = F_{\text{des}} - F_{\text{meas}}$$

$$F_{\text{des}} - F_{\text{meas}} \rightarrow x_{\text{imp}} \rightarrow x_{\text{cmd}} \rightarrow \mathbf{x}_d \rightarrow \text{IK} \rightarrow q_{\text{cmd}}$$



REAL WORLD: JOINT-LEVEL IMPEDANCE CONTROL

$$\tau_i \approx Kp_i(q_{\text{ref}i} - q_i) + Kd_i(-\dot{q}_i).$$

MIT-based

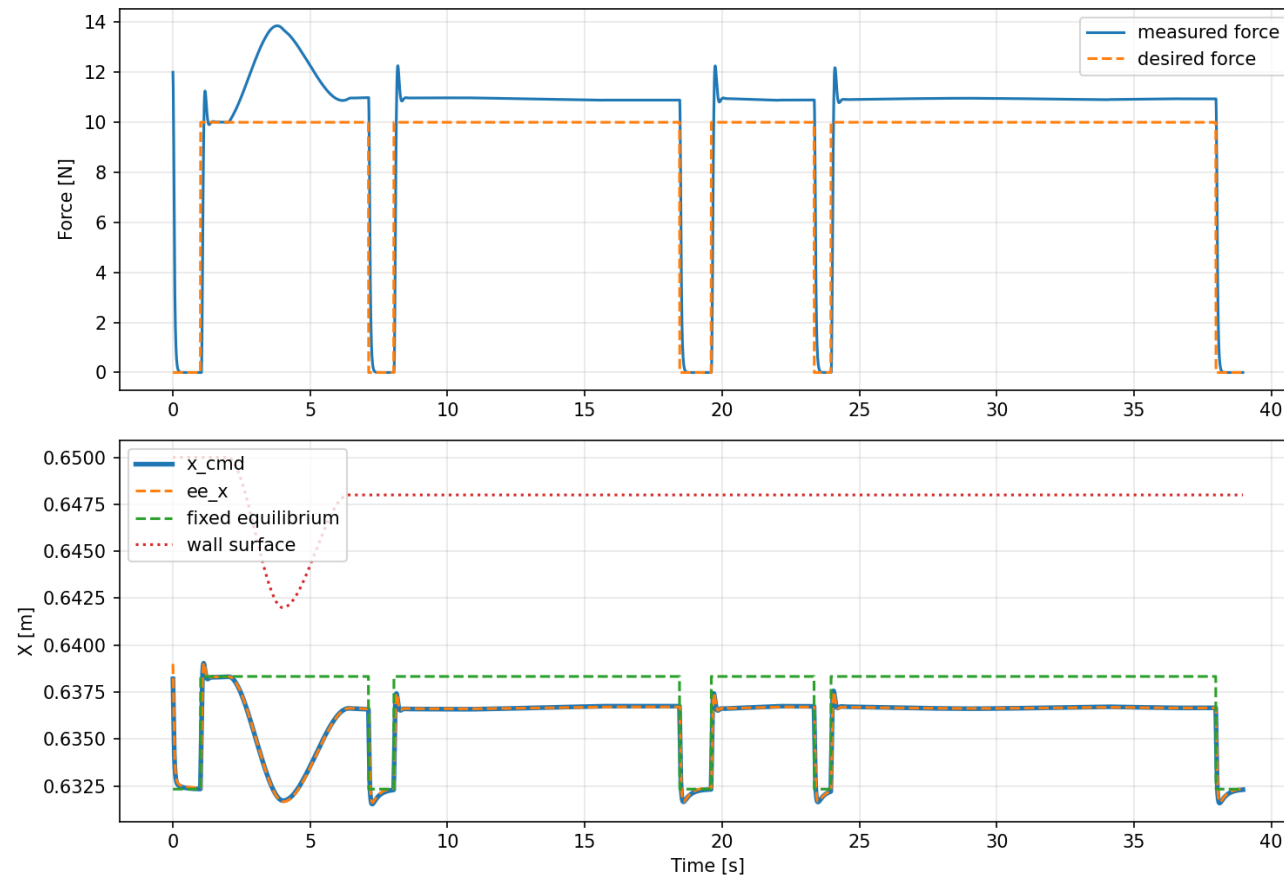
Each joint behaves as a virtual spring–damper.



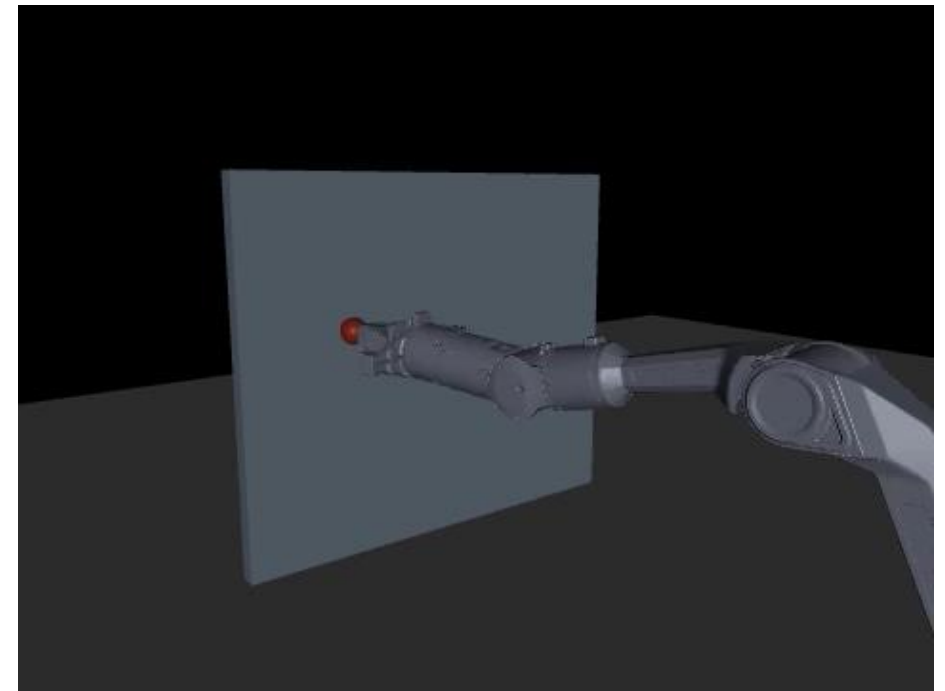
Results



SIMULATION:



$$M = 1.6 \text{ kg}, \quad B = 145 \text{ N} \cdot \text{s/m}, \quad K = 560 \text{ N/m}$$
$$F_{\text{des}} = 10 \text{ N}$$



Results

REAL WORLD:

- Control rate: 200 Hz
- Ramp time: 2 s
- Joint error limit: $0.35 \text{ rad} \approx 20^\circ$
- Letter speed: 0.02 m/s
- Stiffness scale: 0.3

$$K_p = [10, 8, 8, 8, 6, 6]$$

$$K_d = [0.8, 0.6, 0.6, 0.6, 0.5, 0.5]$$

$$K_p^{eff} = 0.3K_p = [3.0, 2.4, 2.4, 2.4, 1.8, 1.8]$$

$$K_d^{eff} = \sqrt{0.3}K_d \approx [0.44, 0.33, 0.33, 0.33, 0.27, 0.27]$$



Discussions

GAIN EFFECT TABLE

Gain	Increase effect	Risk
M (sim)	smoother command	slower response
B (sim)	less oscillation	more lag
K (sim)	less position drift	less compliance
Stiffness (real world)	Better tracking	Risk of damage

LIMITATIONS & CHALLENGES

Virtual path must stay behind the wall → limited workspace

Friction strongly affects low-stiffness impedance

High stiffness improves tracking but may damage the end effector



Future Directions

1. Track targets behind the wall under a maximum contact-force limit
2. Validate safety, tracking accuracy, and painting quality on real surfaces

